

代码解析

1	系统入口：main.cpp	6
1.1	1、LIVMapper mapper(nh);	6
1.1.1	1、初始化激光雷达到 IMU，激光雷达到相机的外参变量	6
1.1.2	2、初始化 lidar 数据预处理器 p_pre，和 imu 数据处理器 p_imu	6
1.1.2.1	1、Preprocess::Preprocess() : feature_enabled(0), lidar_type(AVIA), blind(0.01), point_filter_num(1)	6
1.1.2.2	2、ImuProcess::ImuProcess() : Eye3d(M3D::Identity()), Zero3d(0, 0, 0), b_first_frame(true), imu_need_init(true)	6
1.1.3	3、读取加载到 ros 节点中的参数	7
1.1.3.1	1、来自 yaml 文件	7
1.1.3.1.1	1、common	7
1.1.3.1.2	2、vio	7
1.1.3.1.3	3、time_offset	7
1.1.3.1.4	4、uav	7
1.1.3.1.5	5、evo	7
1.1.3.1.6	6、imu	7
1.1.3.1.7	7、preprocess	7
1.1.3.1.8	8、pcd_save	7
1.1.3.1.9	9、extrin_calib	8
1.1.3.1.10	11、publish	8
1.1.3.1.11	12、盲区平方（仅用于特征提取）	8
1.1.3.2	2、yaml 文件中未定义的参数	8
1.1.3.2.1	1、yaml 中 vio 项未定义	8
1.1.3.2.2	2、yaml 中 time_offset 项未定义	8
1.1.3.2.3	3、yaml 中 preprocess 项未定义	8
1.1.3.2.4	4、yaml 文件未定义 debug 项	8
1.1.4	4、赋值体素地图配置变量 voxel_config	8
1.1.4.1	1、publish	8
1.1.4.2	2、lio	8
1.1.4.3	3、local_map	8
1.1.5	5、对相关点云指针进行重置	9
1.1.6	6、初始化体素地图管理器	9
1.1.6.1	1、根据体素地图配置和八叉树节点初始化体素地图管理器	9
1.1.6.2	2、初始化当前帧 id 为 0	9
1.1.6.3	3、将体素地图管理器中的点云指针进行初始化	9
1.1.7	7、初始化 VIO 管理器	9
1.1.8	8、根路径 root_dir = ROOT_DIR; 来自 CmakeLists.txt	9
1.1.9	9、初始化 PCD 和 Colmap 输出的保存路径及文件，初始化状态预测及更新对应文件 9	9
1.1.9.1	1、判断是否同时保存 pcd 和 colmap	9
1.1.9.1.1	1、找到 colmap 格式输出脚本	9
1.1.9.1.2	2、在终端执行脚本命令行	9
1.1.9.1.3	3、获得脚本执行权限的状态码	9
1.1.9.1.4	4、获得脚本执行成功的状态码	9
1.1.9.2	2、判断是否保存 colmap 打开文件	9
1.1.9.3	3、判断是否分割帧保存 打开文件	9
1.1.9.4	4、打开文件 将 LIO 和 VIO 状态更新之前以及更新之后的分别输出	10
1.1.10	10、初始化组件	10
1.1.10.1	1、设置体素滤波器的大小 0.1x0.1x0.1	10

1.1.10.2	2、将 std 类型的外参转为 Eigen::vector 和 mat 类型并给 mapper 和 voxelmap_manager.....	10
1.1.10.3	3、viki 封装了从 ROS 节点中加载相机模型配置的函数，并在加载失败时抛出异常。 10	
1.1.10.4	4、设置 vio_manager 的参数	10
1.1.10.4.1	1、获得 IMU 到激光雷达的外参	10
1.1.10.4.2	2、获得激光雷达到相机的外参	10
1.1.10.4.3	3、对 vio_manager 进行初始化	10
1.1.10.4.3.1	1、初始化一个视觉子地图的对象	11
1.1.10.4.3.2	2、获得相机的内参	11
1.1.10.4.3.3	3、获得图像的宽高	11
1.1.10.4.3.4	4、获得 IMU 相对于相机的外参:	11
1.1.10.4.3.5	5、获得相机到 IMU 的外参	11
1.1.10.4.3.6	6、获得外参平移的反对称阵，乘以负的外参旋转	11
1.1.10.4.3.7	7、判断 如果设置的每个栅格边长超过 10 像素 if (grid_size > 10): 直接将图像高宽除以 该数 作为图像宽高方向上的栅格个数	12
1.1.10.4.3.8	8、否则: 重新计算单个栅格边长，用图像的高除以默认设置的图像高方向上的栅格个数 17。图像宽高方向上的栅格个数。	12
1.1.10.4.3.9	9、计算 一幅图像可分为多少个栅格	12
1.1.10.4.3.10	10、判断是否进行体素投射	12
1.1.10.4.3.11	11、判断是否 colmap 输出使能: if(colmap_output_en), 按照 colmap 格式将相机相关参数输入到 images.txt 文件中。	13
1.1.10.4.3.12	12、按照栅格的数量对相关容器大小进行重置。grid_num, map_index, map_dist, update_flag, scan_value。	13
1.1.10.4.3.13	13、图像块所拥有的像素总数	13
1.1.10.4.3.14	14、图像块边长的一半	13
1.1.10.4.3.15	15、按图像块像素总数对块缓存器大小重置:	13
1.1.10.4.3.16	16、金字塔所有层的图像块的像素总和:	14
1.1.10.4.3.17	17、边界计算	14
1.1.10.4.3.18	18、按照栅格的数量重置候选体素点容器的大小: retrieve_voxel_points.reserve(length);	14
1.1.10.4.3.19	19、按照栅格的数量重置增添体素点容器大小: append_voxel_points.reserve(length);	14
1.1.10.4.3.20	20、清空子特征图	14
1.1.10.5	5、对 p_imu 的相关参数进行设置	14
1.1.10.5.1	1、设置 lidar 到 imu 外参	14
1.1.10.5.2	2、陀螺仪协方差	14
1.1.10.5.3	3、加速度协方差	14
1.1.10.5.4	4、逆曝光时间	14
1.1.10.5.5	5、陀螺仪偏差的协方差	14
1.1.10.5.6	6、加速度偏差的协方差	14
1.1.10.5.7	7、imu 初始化所需帧数	14
1.1.10.6	6、根据相关项是否使能，对一些项进行设置。	14
1.1.10.7	7、根据可用的传感器判断 使用什么模式。（本解析默认 LIVO 模式） ..	14
1.2	2、mapper.initializeSubscribersAndPublishers(nh, it)	14
1.2.1	1、lidar 消息订阅及回调函数	15
1.2.1.1	1、未使用雷达返回即可 if (!lidar_en) return;	15
1.2.1.2	2、获取互斥锁 mtx_buffer.lock(); 这里并没有多线程	15
1.2.1.3	3、重置一个新的 avia 自定义消息	15
1.2.1.4	4、判断激光雷达与 IMU 时间差	15
1.2.1.5	5、记录当前雷达时间戳并打印出来	15

1.2.1.6	6、判断雷达是否出现时间戳回滚 if (cur_head_time < last_timestamp_lidar).....	15
1.2.1.7	7、时间没有什么问题后 lidar 数据处理函数调用处理函数.....	15
1.2.1.7.1	1、针对 avia 雷达点云的处理.....	15
1.2.1.7.1.1	1、清空三种点云类型：平面，角点，所有点.....	15
1.2.1.7.1.2	记录当前时刻，double t1 = omp_get_wtime();.....	15
1.2.1.7.1.3	2、当前帧点的数量并打印 int plsize = msg->point_num;.....	15
1.2.1.7.1.4	3、根据点数设置点云容量.....	15
1.2.1.7.1.5	4、根据点云线数对每个线设置等同的容量 pl_buff 每个线束的点云，分别存入。.....	15
1.2.1.7.1.6	5、有效线数 uint valid_num = 0;.....	15
1.2.1.7.1.7	6、判断是否进行特征提取 if (feature_enabled)---可以略过.....	15
1.2.1.7.1.8	8、不进行特征提取 else，则循环当前帧点云所有点，并判断点对应的线束是否小于总线束。.....	27
1.2.1.7.2	2、将存入平面点云容器的数据给 pcl_out, *pcl_out = pl_surf;.....	27
1.2.1.8	8、如果点云没有则解开互斥锁，之间返回 空.....	27
1.2.1.9	9、将当前帧处理后的点云 推入 雷达原始数据缓存器.....	27
1.2.1.10	10、将每帧点云的帧头时间都推入雷达帧头时间缓存器.....	27
1.2.1.11	11、更新当前 lidar 帧头时间到最近的 lidar 时间戳.....	27
1.2.1.12	12、解开互斥锁并通知所有线程.....	27
1.2.2	2、Imu 消息订阅及回调函数.....	28
1.2.2.1	1、若未使用 IMU 则直接返回 if (!imu_en) return;.....	28
1.2.2.2	2、最近的雷达时间戳小于 0 直接返回 if (last_timestamp_lidar < 0.0) return;.....	28
1.2.2.3	3、将 Imu 时间戳对齐到 lidar 时间戳.....	28
1.2.2.4	记录时间戳 double timestamp = msg->header.stamp.toSec();.....	28
1.2.2.5	4、判断是否于激光雷达 同步 不能超过 0.5 秒.....	28
1.2.2.6	5、若使用 ros 驱动修复使能 ros_driver_fix_en，将多出来的时间差四舍五入给时间戳。.....	28
1.2.2.7	6、时间没用问题后 进行互斥锁 mtx_buffer.lock();.....	28
1.2.2.8	7、imu 时间回滚。当前时间戳小于上一个时间戳.....	28
1.2.2.9	8、更新上个 IMU 时间戳，并推入 imu 消息到 imu 缓存器中。解开互斥锁.....	28
1.2.2.10	9、判断 imu 频率里程计 imu_rate_odom (imu_prop_enable) 使能 if (imu_prop_enable)。默认是关着的--可略过.....	28
1.2.2.10.1	1、上 mtx_buffer_imu_prop 的互斥锁，mtx_buffer_imu_prop.lock();..	28
1.2.2.10.2	2、判断 imu 需初始化否（不使用 IMU 和迭代次数够时则置为 false）（将 imu 消息推入 prop_imu_buffer ） if (imu_prop_enable && !p_imu->imu_need_init) { prop_imu_buffer.push_back(*msg); }	28
1.2.2.10.3	3、获得最新的 IMU 数据，标签置真：newest_imu = *msg; new_imu = true;.....	28
1.2.2.10.4	4、解锁 mtx_buffer_imu_prop.unlock();.....	28
1.2.2.11	10、通知所有线程 sig_buffer.notify_all();.....	28
1.2.3	3、image 消息订阅及回调函数.....	28
1.2.3.1	1、若不使用相机直接返回 if (!img_en) return;.....	28
1.2.3.2	2、声明一个新的 图像消息指针 sensor_msgs::Image::Ptr msg(new sensor_msgs::Image(*msg_in));.....	28
1.2.3.3	3、将补偿 img 时间戳 double msg_header_time = msg->header.stamp.toSec() + img_time_offset;.....	28
1.2.3.4	4、判断时间回滚.....	28
1.2.3.5	5、加互斥锁 mtx_buffer.lock();.....	29
1.2.3.6	6、记录当前相机时间戳 double img_time_correct = msg_header_time;.....	29
1.2.3.7	7、如果与上帧时间过近则 直接返回.....	29
1.2.3.8	8、从消息中获取 cv 格式的图片 并 推入缓存器.....	29
1.2.3.9	9、更新 上个图像时间戳 并解锁.....	29

1.2.4	4、发布者设置，有*号表示在系统中会执行的，其余的表示未被调用。	29
1.3	3、mapper.run();	29
1.3.1	1、设置循环体的运行频率	29
1.3.2	2、按设置频率进行循环	29
1.3.2.1	1、ros::spinOnce();	29
1.3.2.2	2、if(!sync_packages(LidarMeasures))。	30
1.3.2.2.1	1、判断使用的传感器缓存区是否有数据，否则直接返回 false	30
1.3.2.2.2	2、使用 switch 来按照此时的 SLAM 模式进行同步 switch (slam_mode_)	30
1.3.2.2.2.1	1、case ONLY_LIO: 激光惯性里程计模式---可以略过	30
1.3.2.2.2.2	2、case LIVO: 激光雷达，惯性，相机里程计模式	32
1.3.2.2.2.3	3、case ONLY_LO:{xxx}----可以略过	35
1.3.2.3	3、handleFirstFrame();	36
1.3.2.3.1	判断 是否第一帧 if(!is_first_frame)。获得第一帧激光雷达的帧头时间，将这个时间给 imu 处理器中的第一帧激光雷达时间 p_imu->first_lidar_time = _first_lidar_time; 修改第一帧标志位。	36
1.3.2.4	4、processImu();	36
1.3.2.4.1	1、p_imu->Process2(LidarMeasures, _state, feats_undistort);	36
1.3.2.4.1.1	1、三个时间记录量 double t1, t2, t3;	36
1.3.2.4.1.2	2、检查点云空不空	37
1.3.2.4.1.3	3、判断是否使用了 IMU。如果不使用 IMU 则进行无 imu 的前向传播后返回。-----可略过	37
1.3.2.4.1.4	4、获得激光雷达测量组中最新的 IMU 队列	40
1.3.2.4.1.5	5、判断是否需要 imu 初始化 该函数仅执行一次。	40
1.3.2.4.1.6	6、UndistortPcl(lidar_meas, stat, *cur_pcl_un);	43
1.3.2.4.2	2、if (gravity_align_en) gravityAlignment(); ---可略过	52
1.3.2.4.2.1	1、如果 imu 不需要初始化了且重量对齐未完成 if(!p_imu->imu_need_init && !gravity_align_finished)，获得状态重力与地球重力的旋转阵，将状态都旋转到对齐了地球重力的方向上。完成对齐。	52
1.3.2.4.3	3、记录状态，并将先验状态_state 和去畸变点云 feats_undistort 给体素地图管理器 voxelmap_manager	52
1.3.2.5	5、stateEstimationAndMapping();	52
1.3.2.5.1	1、void LIVMapper::handleLIO()	52
1.3.2.5.1.1	1、数据有效判断	52
1.3.2.5.1.2	记录时间 double t0 = omp_get_wtime();	53
1.3.2.5.1.3	2、点云降采样	53
1.3.2.5.1.4	记录时间 double t_down = omp_get_wtime();	53
1.3.2.5.1.5	3、获得各系下的体素点云并构建体素地图	53
1.3.2.5.1.6	记录时间 double t1 = omp_get_wtime();	62
1.3.2.5.1.7	4、用体素管理器进行状态估计	62
1.3.2.5.1.8	记录时间 double t2 = omp_get_wtime();	74
1.3.2.5.1.9	5、根据估计的状态决定是否高频发布，评估，里程计发布	74
1.3.2.5.1.10	记录时间 double t3 = omp_get_wtime();	75
1.3.2.5.1.11	6、根据后验状态对点的不确定性列表进行更新、体素地图（平面属性）更新、复制列表给_pv_list	75
1.3.2.5.1.12	记录时间：double t4 = omp_get_wtime();	79
1.3.2.5.1.13	7、当前帧转到世界系并准备发布：pcl_w_wait_pub	79
1.3.2.5.1.14	8、发布各类型点云、路径	81
1.3.2.5.1.15	9、打印 LIO 整体信息	81
1.3.2.5.2	2、void LIVMapper::handleVIO()	81
1.3.2.5.2.1	1、获得状态的旋转欧拉角 euler_cur 同时将上次 LIO 获得的状态	

	输出到文件 fout_pre	82
1.3.2.5.2.2	2、输出待发布的世界点的点数:	82
1.3.2.5.2.3	3、判断 这一帧图像捕获时间与第一帧点云帧头时间差再减去-10 的绝对值是否 小于 0.3 (帧计数 $6/2 * 0.1$) 。-----可略过	82
1.3.2.5.2.4	4、vio_manager->processFrame(LidarMeasures.measures.back().img, _pv_list, voxelmap_manager->voxel_map_, LidarMeasures.last_lid_update_time - _first_lidar_time);.....	82
1.3.2.5.2.5	5、判断是否以 IMU 频率传播使能: if (imu_prop_enable)----可略过 123	
1.3.2.5.2.6	6、发布有 RGB 颜色的点云:	123
1.3.2.5.2.7	7、发布带有标注点的图像 img_cp :	123
1.3.2.5.2.8	8、将姿态存入 fout_out 文件中	124
1.3.3	3、系统运行结束后将缓存的点云保存为 PCD	124

1 系统入口：main.cpp

int main(int argc, char **argv){ ros::init(argc, argv, "laserMapping"); //初始化 ros 节点 ros::NodeHandle nh; //获得 ros 节点的句柄, 这是与 ROS 主系统进行通信的主要接口。 image_transport::ImageTransport it(nh); //是 ROS 中专门用于高效传输图像消息的工具类。 LIVMapper mapper(nh); ※ // 初始化建图器 (负责整个系统流程) mapper.initializeSubscribersAndPublishers(nh, it); ※ // 初始化系统的订阅器和发布者 mapper.run(); ※ // 系统运行 return 0;} 可以看到, 主函数包含了三个关键元素 (※), 笔者将对关键元素进行展开解析。

1.1 1、LIVMapper mapper(nh);

声明于 LIVMapper.h, 定义于 LIVMapper.cpp. 初始化对象的同时会调用 LIVMapper 中相应的构造函数: LIVMapper::LIVMapper(ros::NodeHandle &nh) : extT(0, 0, 0), extR(M3D::Identity()) 此处对构造函数解析。主要做了以下几个事。1、初始化传感器外参; 2、初始化 lidar 和 Imu 的数据处理器 p_pre, p_imu; 3、加载 ros 节点中的参数; 4、体素地图配置 voxel_config 加载; 5、mapper 的相关点云指针进行重置; 6、初始化体素地图管理器; 7、初始化 VIO 管理器 vio_manager (实际什么也没做); 8、获得根路径; 9、初始化 PCD 和 Colmap 输出的保存路径及文件, 初始化状态预测及更新对应文件; 10、初始化组件 (这里才是 vio_manager 初始化的地方)

1.1.1 1、初始化激光雷达到 IMU, 激光雷达到相机的外参变量

extrinT.assign(3, 0.0); extrinR.assign(9, 0.0); cameraextrinT.assign(3, 0.0); cameraextrinR.assign(9, 0.0); 以上变量均在 LIVMapper.h 中的 class LIVMapper 中完成声明。

1.1.2 2、初始化 lidar 数据预处理器 p_pre, 和 imu 数据处理器 p_imu

p_pre.reset(new Preprocess()); // PreprocessPtr p_pre; p_imu.reset(new ImuProcess()); // ImuProcessPtr p_imu; 以上变量均在 LIVMapper.h 中的 class LIVMapper 中完成声明。初始化时会调用 p_pre 和 p_imu 对应的构造函数。由于还未加载真实参数, p_pre 和 p_imu 在这里都只是对相关变量进行重置。注意: Fast-LIVO2 采用的是直接法, 所以关于特征提取的变量可忽略。

1.1.2.1 1、Preprocess::Preprocess() : feature_enabled(0), lidar_type(AVIA), blind(0.01), point_filter_num(1)

声明于 preprocess.h, 定义于 preprocess.cpp feature_enabled(0), lidar_type(AVIA), blind(0.01), point_filter_num(1) // 是否进行特征提取, 激光雷达类型, 以及盲区, 跳过的点数 (相当于降采样) inf_bound = 10; // 用于判断边缘的距离上限 (需要提取特征的时候才会使用) N_SCANS = 6; // 激光雷达的线束 group_size = 8; // 每组点的数量下限 (需要提取特征的时候才会使用) disA = 0.01; // 用于计算每组允许的最远距离 (需要提取特征的时候才会使用) disA = 0.1; // B? 应该是 B, Fast-lio 系列代码有。用于计算每组允许的最远距离 (需要提取特征的时候才会使用) p2l_ratio = 225; // 点到线的距离的比 h > AC/15 用于判断平面点的。 (需要提取特征的时候才会使用) limit_maxmid = 6.25; // 组内头部与中间距离差的比上限 (需要提取特征的时候才会使用) limit_midmin = 6.25; // 组内中间与尾部距离差的比上限 (需要提取特征的时候才会使用) limit_maxmin = 3.24; // 组内头部与尾部距离差的比上限 (需要提取特征的时候才会使用) jump_up_limit = 170.0; // 用于判断激光射线方向与邻点方向的夹角 (需要提取特征的时候才会使用) jump_down_limit = 8.0; // 用于判断激光射线方向与邻点方向的夹角 (需要提取特征的时候才会使用) cos160 = 160.0; // 用于判断两个邻点向量之间的夹角 (需要提取特征的时候才会使用) edgea = 2; // 距离差倍数, 用于判断边缘跳跃点 (需要提取特征的时候才会使用) edgeb = 0.1; // 距离差的差, 用于判断边缘跳跃点 (需要提取特征的时候才会使用) smallp_intersect = 172.5; // 两个邻点向量之间的夹角上限 (需要提取特征的时候才会使用) smallp_ratio = 1.2; // 两个邻点距离差的比上限, (需要提取特征的时候才会使用) given_offset_time = false; // 是否给定偏移时间, 用于 velody 等无时间偏移量的雷达 jump_up_limit = cos(jump_up_limit / 180 * M_PI); // 计算对应的余弦值 jump_down_limit = cos(jump_down_limit / 180 * M_PI); // 计算对应的余弦值 cos160 = cos(cos160 / 180 * M_PI); // 计算对应的余弦值 smallp_intersect = cos(smallp_intersect / 180 * M_PI); // 计算对应的余弦值

1.1.2.2 2、ImuProcess::ImuProcess() : Eye3d(M3D::Identity()), Zero3d(0, 0, 0), b_first_frame(true), imu_need_init(true)

声明于 "IMU_Processing.h" 定义于 "IMU_Processing.cpp" Eye3d(M3D::Identity()), Zero3d(0, 0, 0), b_first_frame(true), imu_need_init(true) // 设置单位阵, 0 向量, 第一帧标志位(真), imu 需

```

初始化标志位(真) init_iter_num = 1; // 初始化的迭代次数为 1 cov_acc = V3D(0.1, 0.1, 0.1); //
加速度协方差 cov_gyr = V3D(0.1, 0.1, 0.1); //陀螺仪协方差 cov_bias_gyr = V3D(0.1, 0.1, 0.1);
//陀螺仪偏差的协方差 cov_bias_acc = V3D(0.1, 0.1, 0.1); //加速度偏差的协方差
cov_inv_expo = 0.2; //逆曝光时间协方差 mean_acc = V3D(0, 0, -1.0); //加速度均值 mean_gyr
= V3D(0, 0, 0); //陀螺仪均值 angvel_last = Zero3d; //上次角速度 acc_s_last = Zero3d; // 上次
加速度 Lid_offset_to_IMU = Zero3d; // 激光雷达到 imu 的偏移 Lid_rot_to_IMU = Eye3d; //
激光雷达到 IMU 的旋转 last_imu.reset(new sensor_msgs::Imu()); //对最近 IMU 消息进行重置
cur_pcl_un_.reset(new PointCloudXYZI()); //对当前无畸变点云进行重置

```

1.1.3 3、读取加载到 ros 节点中的参数

readParameters(nh);声明于 LIVMapper.h, 定义于 LIVMapper.cpp 对应函数: void

LIVMapper::readParameters(ros::NodeHandle &nh)将 yaml 文件中的参数读出并赋给 LIVMapper.h 中声明的变量 (大部分参数含义已在预热阶段注释)。注意, 有些参数并未在 yaml 文件中定义。

1.1.3.1 1、来自 yaml 文件

1.1.3.1.1 1、common

```

nh.param<string>("common/lid_topic", lid_topic,
"/livox/lidar");nh.param<string>("common/imu_topic", imu_topic,
"/livox/imu");nh.param<bool>("common/ros_driver_bug_fix", ros_driver_fix_en,
false);nh.param<int>("common/img_en", img_en, 1);nh.param<int>("common/lidar_en",
lidar_en, 1);nh.param<string>("common/img_topic", img_topic, "/left_camera/image");

```

1.1.3.1.2 2、vio

```

nh.param<bool>("vio/normal_en", normal_en,
true);nh.param<bool>("vio/inverse_composition_en", inverse_composition_en,
false);nh.param<int>("vio/max_iterations", max_iterations,
5);nh.param<double>("vio/img_point_cov", IMG_POINT_COV,
100);nh.param<bool>("vio/raycast_en", raycast_en,
false);nh.param<bool>("vio/exposure_estimate_en", exposure_estimate_en,
true);nh.param<double>("vio/inv_expo_cov", inv_expo_cov,
0.2);nh.param<int>("vio/patch_pyramid_level", patch_pyramid_level,
3);nh.param<int>("vio/patch_size", patch_size, 8);nh.param<double>("vio/outlier_threshold",
outlier_threshold, 1000);

```

1.1.3.1.3 3、time_offset

```

nh.param<double>("time_offset/exposure_time_init", exposure_time_init,
0.0);nh.param<double>("time_offset/img_time_offset", img_time_offset,
0.0);nh.param<double>("time_offset/imu_time_offset", imu_time_offset, 0.0);

```

1.1.3.1.4 4、uav

```

nh.param<bool>("uav/imu_rate_odom", imu_prop_enable,
false);nh.param<bool>("uav/gravity_align_en", gravity_align_en, false);

```

1.1.3.1.5 5、evo

```

nh.param<string>("evo/seq_name", seq_name, "01");nh.param<bool>("evo/pose_output_en",
pose_output_en, false);

```

1.1.3.1.6 6、imu

```

nh.param<double>("imu/gyr_cov", gyr_cov, 1.0);nh.param<double>("imu/acc_cov", acc_cov,
1.0);nh.param<int>("imu/imu_int_frame", imu_int_frame, 3);nh.param<bool>("imu/imu_en",
imu_en, false);nh.param<bool>("imu/gravity_est_en", gravity_est_en,
true);nh.param<bool>("imu/ba_bg_est_en", ba_bg_est_en, true);

```

1.1.3.1.7 7、preprocess

```

nh.param<double>("preprocess/blind", p_pre->blind,
0.01);nh.param<double>("preprocess/filter_size_surf", filter_size_surf_min,
0.5);nh.param<bool>("preprocess/hilti_en", hilti_en,
false);nh.param<int>("preprocess/lidar_type", p_pre->lidar_type,
AVIA);nh.param<int>("preprocess/scan_line", p_pre->N_SCANS,
6);nh.param<int>("preprocess/point_filter_num", p_pre->point_filter_num, 3);

```

1.1.3.1.8 8、pcd_save

```

nh.param<int>("pcd_save/interval", pcd_save_interval, -
1);nh.param<bool>("pcd_save/pcd_save_en", pcd_save_en,
false);nh.param<bool>("pcd_save/colmap_output_en", colmap_output_en,

```

```
false);nh.param<double>("pcd_save/filter_size_pcd", filter_size_pcd, 0.5);
```

1.1.3.1.9 9、extrin_calib

```
nh.param<vector<double>>("extrin_calib/extrinsic_T", extrinT,
vector<double>());nh.param<vector<double>>("extrin_calib/extrinsic_R", extrinR,
vector<double>());nh.param<vector<double>>("extrin_calib/Pcl", cameraextrinT,
vector<double>());nh.param<vector<double>>("extrin_calib/Rcl", cameraextrinR,
vector<double>());
```

1.1.3.1.10 11、publish

```
nh.param<double>("publish/blind_rgb_points", blind_rgb_points,
0.01);nh.param<int>("publish/pub_scan_num", pub_scan_num,
1);nh.param<bool>("publish/pub_effect_point_en", pub_effect_point_en,
false);nh.param<bool>("publish/dense_map_en", dense_map_en, false);
```

1.1.3.1.11 12、盲区平方（仅用于特征提取）

```
p_pre->blind_sqr = p_pre->blind * p_pre->blind;
```

1.1.3.2 2、yaml 文件中未定义的参数

1.1.3.2.1 1、yaml 中 vio 项未定义

```
nh.param<int>("vio/grid_size", grid_size, 5); // 每个栅格单元的大小
nh.param<int>("vio/grid_n_height", grid_n_height, 17); // 在图像的高度方向上有多少个栅格。
```

1.1.3.2.2 2、yaml 中 time_offset 项未定义

```
nh.param<double>("time_offset/lidar_time_offset", lidar_time_offset, 0.0);
```

1.1.3.2.3 3、yaml 中 preprocess 项未定义

```
nh.param<bool>("preprocess/feature_extract_enabled", p_pre->feature_enabled, false); // 是否
进行特征提取，默认不使用
```

1.1.3.2.4 4、yaml 文件未定义 debug 项

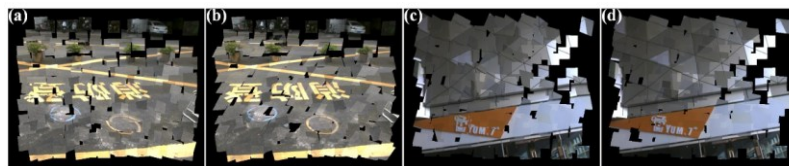


Fig. S1: (a) and (c) depict the warped patches derived from the constant depth assumption, while (b) and (d) represent those based on the plane prior.

```
nh.param<double>("debug/plot_time", plot_time, -10);nh.param<int>("debug/frame_cnt",
frame_cnt, 6); // 这两个参数是 Fast-LIVO2 做实验用的，将当前块和参考块绘制到一个
图像并观察是否对齐。
```

1.1.4 4、赋值体素地图配置变量 voxel_config

VoxelMapConfig voxel_config;loadVoxelConfig(nh, voxel_config); //VoxelMapConfig 声明于 voxel_map.h 对应函数：void loadVoxelConfig(ros::NodeHandle &nh, VoxelMapConfig &voxel_config)，声明于 voxel_map.h，定义于 voxel_map.cpp。将 yaml 文件中的参数读出并赋给 voxel_config 中的变量（大部分参数含义已在预热阶段注释）。

1.1.4.1 1、publish

```
nh.param<bool>("publish/pub_plane_en", voxel_config.is_pub_plane_map_, false); //是否发布平面
visualization_msgs::MarkerArray 格式的
```

1.1.4.2 2、lio

```
nh.param<int>("lio/max_layer", voxel_config.max_layer_, 1);nh.param<double>("lio/voxel_size",
voxel_config.max_voxel_size_, 0.5);nh.param<double>("lio/min_eigen_value",
voxel_config.planner_threshold_, 0.01);nh.param<double>("lio/sigma_num",
voxel_config.sigma_num_, 3); // 作为一种系数与激光雷达观测方程的协方差相乘，作为一个
平面判断条件。nh.param<double>("lio/beam_err", voxel_config.beam_err_, 0.02);
nh.param<double>("lio/dept_err", voxel_config.dept_err_,
0.05);nh.param<vector<int>>("lio/layer_init_num", voxel_config.layer_init_num_,
vector<int>{5,5,5,5,5});nh.param<int>("lio/max_points_num", voxel_config.max_points_num_,
50);nh.param<int>("lio/max_iterations", voxel_config.max_iterations_, 5);
```

1.1.4.3 3、local_map

```
nh.param<bool>("local_map/map_sliding_en", voxel_config.map_sliding_en,
false);nh.param<int>("local_map/half_map_size", voxel_config.half_map_size,
```



```
100);nh.param<double>("local_map/sliding_thresh", voxel_config.sliding_thresh, 8);
```

1.1.5 5、对相关点云指针进行重置

```
visual_sub_map.reset(new PointCloudXYZI()); //视觉子地图点云 feats_undistort.reset(new  
PointCloudXYZI()); //去畸变后的点云 feats_down_body.reset(new PointCloudXYZI()); //降采样后  
的载体系点云 feats_down_world.reset(new PointCloudXYZI()); //降采样后的世界系点云  
pcl_w_wait_pub.reset(new PointCloudXYZI()); //待发布的世界系点云 pcl_wait_pub.reset(new  
PointCloudXYZI()); //待发布的载体系点云 pcl_wait_save.reset(new PointCloudXYZRGB()); //待保  
存为 PCD 的 RGB 值点云 pcl_wait_save_intensity.reset(new PointCloudXYZI()); //待保存为 PCD 的  
强度值点云
```

1.1.6 6、初始化体素地图管理器

```
voxelmap_manager.reset(new VoxelMapManager(voxel_config, voxel_map)); 其中  
VoxelMapManagerPtr voxelmap_manager; //体素地图管理器  
std::unordered_map<VOXEL_LOCATION, VoxelOctoTree *> voxel_map; //八叉树根节点均声明于  
LIVMapper.h; voxelmap_manager 在初始化时调用对应构造函数:  
VoxelMapManager(VoxelMapConfig &config_setting, std::unordered_map<VOXEL_LOCATION,  
VoxelOctoTree *> &voxel_map): config_setting_(config_setting), voxel_map_(voxel_map)声明及定  
义于 voxel_map.h
```

1.1.6.1 1、根据体素地图配置和八叉树节点初始化体素地图管理器

```
config_setting_(config_setting), voxel_map_(voxel_map)
```

1.1.6.2 2、初始化当前帧 id 为 0

```
current_frame_id_ = 0;
```

1.1.6.3 3、将体素地图管理器中的点云指针进行初始化

```
feats_undistort_.reset(new PointCloudXYZI()); //去畸变点云 feats_down_body_.reset(new  
PointCloudXYZI()); //降采样后的载体点云 feats_down_world_.reset(new PointCloudXYZI()); //  
降采样后的世界点云
```

1.1.7 7、初始化 VIO 管理器

VIOManagerPtr vio_manager; 声明于 LIVMapper.h 其对应的构造函数:
VIOManager::VIOManager(){} 声明于 vio.h, 定义于 vio.cpp 函数是空的, 什么也不执行。实际
上的初始化在 initializeVIO, 不用急, 看到第 10 项就能了解了。

1.1.8 8、根路径 root_dir = ROOT_DIR; 来自 CmakeLists.txt

```
string root_dir; // LIVMapper.h
```

1.1.9 9、初始化 PCD 和 Colmap 输出的保存路径及文件, 初始化状态预测及更新对应文件

initializeFiles();对应函数: void LIVMapper::initializeFiles()声明于 LIVMapper.h, 定义于
LIVMapper.cpp

1.1.9.1 1、判断是否同时保存 pcd 和 colmap

```
if (pcd_save_en && colmap_output_en)用于执行对应的脚本生成相应文件
```

1.1.9.1.1 1、找到 colmap 格式输出脚本

```
const std::string folderPath = std::string(ROOT_DIR) + "/scripts/colmap_output.sh";
```

1.1.9.1.2 2、在终端执行脚本命令行

```
std::string chmodCommand = "chmod +x " + folderPath;
```

1.1.9.1.3 3、获得脚本执行权限的状态码

```
int chmodRet = system(chmodCommand.c_str());失败则: std::cerr << "Failed to set execute  
permissions for the script." << std::endl;return;
```

1.1.9.1.4 4、获得脚本执行成功的状态码

```
int executionRet = system(folderPath.c_str());失败则: std::cerr << "Failed to execute the  
script." << std::endl;return;
```

1.1.9.2 2、判断是否保存 colmap 打开文件

```
if(colmap_output_en) {fout_points.open(std::string(ROOT_DIR) +  
"Log/Colmap/sparse/0/points3D.txt", std::ios::out);}打开 colmap 保存路径下的文件 fout_points
```

1.1.9.3 3、判断是否分割帧保存 打开文件

```
if(pcd_save_interval > 0) {fout_pcd_pos.open(std::string(ROOT_DIR) +
```

```
"Log/PCD/scans_pos.json", std::ios::out);}
```

1.1.9.4 4、打开文件 将LIO 和VIO 状态更新之前以及更新之后的分别输出

```
fout_pre.open(DEBUG_FILE_DIR("mat_pre.txt"),  
std::ios::out);fout_out.open(DEBUG_FILE_DIR("mat_out.txt"), std::ios::out);
```

1.1.10 10、初始化组件

initializeComponents();对应函数：void LIVMapper::initializeComponents()声明于 LIVMapper.h, 定义于 LIVMapper.cpp1、设置了体素滤波器；2、获得 Eigen 格式的传感器外参给 mapper 和 voxelmap_manager；3、加载相机参数；4、设置 vio_manager 的参数（栅格、图像块、状态、边界，视觉子地图等等）；5、设置 p_imu 相关参数（外参，协方差等）；6、相关使能项（是否使用 Imu,是否估计偏差等）；7、设置本次系统执行的模式（默认为 LIVO）

1.1.10.1 1、设置体素滤波器的大小 0.1x0.1x0.1

```
downSizeFilterSurf.setLeafSize(filter_size_surf_min, filter_size_surf_min,  
filter_size_surf_min);pcl::VoxelGrid<PointType> downSizeFilterSurf; 声明于 LIVMapper.h 笔者  
再解释一下，这里的体素大小是对每一帧点云体素滤波用的，经过状态更新整理为地图时，  
会用 0.5x0.5x0.5（voxel_size）的体素块来分割地图。
```

1.1.10.2 2、将std 类型的外参转为Eigen::vector 和mat 类型并给 mapper 和 voxelmap_manager

```
extT << VEC_FROM_ARRAY(extrinT); // V3D extT;声明于 LIVMapper.h extR <<  
MAT_FROM_ARRAY(extrinR); // M3D extR; 声明于 LIVMapper.h voxelmap_manager->extT_ <<  
VEC_FROM_ARRAY(extrinT);voxelmap_manager->extR_ <<  
MAT_FROM_ARRAY(extrinR);
```

1.1.10.3 3、viki 封装了从 ROS 节点中加载相机模型配置的函数，并在加载失败时抛出异常。

```
if (!vk::camera_loader::loadFromRosNs("laserMapping", vio_manager->cam)) throw  
std::runtime_error("Camera model not correctly specified.");loadFromRosNs 需要 ros 节点句柄，  
这用来实现读取 camera_pinhole.yaml 中的参数，注意：要按要求的格式进行参数命名
```

1.1.10.4 4、设置vio_manager 的参数

```
vio_manager->grid_size = grid_size; // (yaml 文件未定义) 单个栅格边的像素数量  
5vio_manager->patch_size = patch_size; // 图像块 边长大小 8 vio_manager->outlier_threshold =  
outlier_threshold; // 块像素光度误差离群阈值 vio_manager->setImuToLidarExtrinsic(extT,  
extR); // IMU 到激光雷达的外参 vio_manager->setLidarToCameraExtrinsic(cameraextrinR,  
cameraextrinT); // 激光雷达到相机的外参 vio_manager->state = &_state; //当前系统的状态  
量（更新）vio_manager->state_propagat = &state_propagat; //状态的传播量（预测）  
vio_manager->max_iterations = max_iterations; //最大的迭代次数 5vio_manager->  
img_point_cov = IMG_POINT_COV; //图像像素点的协方差 100（也是 vio 观测方程的协方  
差）vio_manager->normal_en = normal_en; // 是否有法向量 vio_manager->  
inverse_composition_en = inverse_composition_en; //是否逆合成 vio_manager->raycast_en =  
raycast_en; //是否体素射线投射 vio_manager->grid_n_width = grid_n_width; //栅格单元在图像  
宽度方向上的个数 vio_manager->grid_n_height = grid_n_height; // 栅格单元在图像高度方向  
上的个数 vio_manager->patch_pyramid_level = patch_pyramid_level; //图像块金字塔层级  
vio_manager->exposure_estimate_en = exposure_estimate_en; //是否估计曝光时间 vio_manager->  
colmap_output_en = colmap_output_en; //是否输出 colmap 格式文件 vio_manager->  
initializeVIO(); // 对 VIO 进行初始化：初始化了一个视觉子地图 visual_submap，获得相  
机相关参数，计算栅格数，进行体素射线投射（默认不使用，读者可以跳过），保存相机  
参数至 colmap 路径，关于栅格相关属性的容器，构建图像块大小，边界设置，容器大小和  
子特征图 sub_feat_map 清空
```

1.1.10.4.1 1、获得 IMU 到激光雷达的外参

```
void VIOManager::setImuToLidarExtrinsic(const V3D &transl, const M3D &rot){ Pli = -  
rot.transpose() * transl; Rli = rot.transpose();}
```

1.1.10.4.2 2、获得激光雷达到相机的外参

```
void VIOManager::setLidarToCameraExtrinsic(vector<double> &R, vector<double>  
&P){ Rcl << MAT_FROM_ARRAY(R); Pcl << VEC_FROM_ARRAY(P);}
```

1.1.10.4.3 3、对vio_manager 进行初始化

```
vio_manager->initializeVIO();void VIOManager::initializeVIO() 声明于 vio.h, 定义于
```

vio.cpp 初始化了一个视觉子地图 visual_submap，获得相机相关参数，计算栅格数，进行体素射线投射，保存相机参数至 colmap 路径，关于栅格相关属性的容器，构建图像块大小，边界设置，容器大小和子特征图 sub_feat_map 清空

1.1.10.4.3.1 1、初始化一个视觉子地图的对象

visual_submap = new SubSparseMap; 声明于 vio.h: SubSparseMap *visual_submap; 对应构造函数 SubSparseMap() 声明并定义于 vio.h 视觉子地图的含义可以通过论文解析获得，或者先不急着想明白它，跟着代码解析到后面就知道它是干啥的了。

1.1.10.4.3.1.1 1、传播误差容器大小设置

propa_errors.reserve(SIZE_LARGE); 声明于 vio.h 在状态预测阶段，保存图像每个栅格单元对应视觉点的图像块光度误差

1.1.10.4.3.1.2 2、误差容器大小设置

errors.reserve(SIZE_LARGE); 声明于 vio.h 在状态更新阶段，保存图像每个栅格单元对应视觉点的图像块光度误差

1.1.10.4.3.1.3 3、仿射块容器大小设置

warp_patch.reserve(SIZE_LARGE); 声明于 vio.h 保存每个栅格单元对应视觉点的图像块投影到参考帧上后的图像块

1.1.10.4.3.1.4 4、最佳搜索层级容器大小设置

search_levels.reserve(SIZE_LARGE); 声明于 vio.h 保存每个栅格单元对应视觉点的最佳搜索层级

1.1.10.4.3.1.5 5、体素点容器大小设置

voxel_points.reserve(SIZE_LARGE); 声明于 vio.h 保存每个栅格单元对应视觉点到该视觉子地图中

1.1.10.4.3.1.6 6、逆曝光容器设置

inv_expo_list.reserve(SIZE_LARGE); 声明于 vio.h 保存每个栅格单元对应视觉点到的参考图像块逆曝光时间

1.1.10.4.3.1.7 7、添加体素地图中点容器大小设置

add_from_voxel_map.reserve(SIZE_SMALL); 声明于 vio.h 保存从全局体素点中抽取出来的体素点。

1.1.10.4.3.2 2、获得相机的内参

fx = cam->fx(); fy = cam->fy(); cx = cam->cx(); cy = cam->cy(); image_resize_factor = cam->scale();

1.1.10.4.3.3 3、获得图像的宽高

width = cam->width(); height = cam->height();

1.1.10.4.3.4 4、获得 IMU 相对于相机的外参：

Rci = Rcl * Rli; Pci = Rcl * Pli + Pcl;

1.1.10.4.3.5 5、获得相机到 IMU 的外参

V3D Pic; M3D tmp; Jdphi_dR = Rci; // 为什么是 Imu 到相机的旋转外参 Pic = -

Rci.transpose() * Pci; // 这一步是根据 Imu 到相机的外参得到相机到 imu 的平移外参。

1.1.10.4.3.6 6、获得外参平移的反对称阵，乘以负的外参旋转

tmp << SKEW_SYM_MATRX(Pic); Jdp_dR = -Rci * tmp; 这里不明白没有关系，知

道这里是在为光度误差的雅可比阵做准备即可。

- 1.1.10.4.3.7 7、判断 如果设置的每个栅格边长超过 10 像素 if (grid_size > 10): 直接将图像高宽除以该数 作为图像宽高方向上的栅格个数

```
grid_n_width = ceil(static_cast<double>(width / grid_size)); grid_n_height =  
ceil(static_cast<double>(height / grid_size));
```

- 1.1.10.4.3.8 8、否则: 重新计算单个栅格边长, 用图像的高除以默认设置的图像高方向上的栅格个数 17。图像宽高方向上的栅格个数。

```
grid_size = static_cast<int>(height / grid_n_height); grid_n_height =  
ceil(static_cast<double>(height / grid_size)); grid_n_width =  
ceil(static_cast<double>(width / grid_size));
```

- 1.1.10.4.3.9 9、计算 一幅图像可分为多少个栅格

```
length = grid_n_width * grid_n_height;
```

- 1.1.10.4.3.10 10、判断是否进行体素投射

if(raycast_en)将图片划分为 nxm 的栅格, 将所有栅格中心像素的射线采样点都保存下来 (这些点是固定不变的, 与相机坐标系固连), 仅执行一次即可。

1.1.10.4.3.10.1 1、设置 栅格数量大小的边界标志容器

```
border_flag.resize(length, 0);用于判断哪个栅格是在图像边缘。
```

1.1.10.4.3.10.2 2、所有栅格对应的采样点容器 (带有采样点的射线) :

```
std::vector<std::vector<V3D>>().swap(rays_with_sample_points);rays_with_sample_  
points.reserve(length);
```

1.1.10.4.3.10.3 3、声明一些变量最小深度 d_min , 最大深度 d_max , 步长 step

```
float d_min = 0.1;float d_max = 3.0;float step = 0.2;
```

1.1.10.4.3.10.4 4、循环每个栅格, 按行循环。

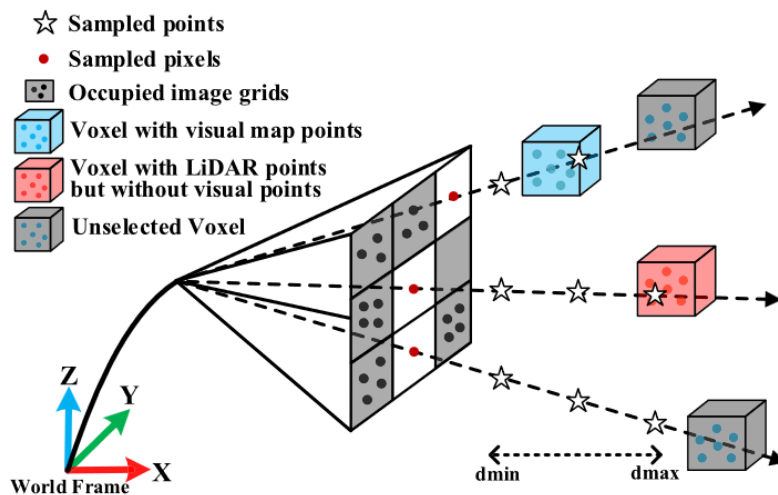


Fig. 7. Illustration of on-demand voxel raycasting.

```
for (int grid_row = 1; grid_row <= grid_n_height; grid_row++) {for (int grid_col =  
1; grid_col <= grid_n_width; grid_col++) {每个栅格单元的中心像素发出射线,  
并在射线上按固定间隔进行采样
```

1.1.10.4.3.10.4.1 1、声明一个向量容器存储每个栅格的采样点:

```
std::vector<V3D> SamplePointsEachGrid;
```

1.1.10.4.3.10.4.2 2、每个栅格左上角第一个点的索引：

```
int index = (grid_row - 1) * grid_n_width + grid_col - 1;
```

1.1.10.4.3.10.4.3 3、栅格的中心像素坐标

```
int u = grid_size / 2 + (grid_col - 1) * grid_size; int v = grid_size / 2 + (grid_row - 1) * grid_size;
```

1.1.10.4.3.10.4.4 4、按照步长，从最小深度循环之最大深度：

```
for (float d_temp = d_min; d_temp <= d_max; d_temp += step)
```

1.1.10.4.3.10.4.4.11、一个向量存放三维点：V3D xyz;

1.1.10.4.3.10.4.4.22、相机到世界系的变换（像素到三维点）

```
vikit xyz = cam->cam2world(u, v);
```

1.1.10.4.3.10.4.4.33、将单位深度按 step 恢复到绝对深度，每个栅格点中心像素会产生 (d_max - d_min) / step 个三维点（都在一条线上）

1.1.10.4.3.10.4.4.44、将这个栅格中心像素的射线采样点都存入 SamplePointsEachGrid

```
SamplePointsEachGrid.push_back(xyz);
```

1.1.10.4.3.10.4.5 5、将所有栅格中心像素产生的射线采样点存入

```
rays_with_sample_points
```

```
rays_with_sample_points.push_back(SamplePointsEachGrid);
```

1.1.10.4.3.11 11、判断是否 colmap 输出使能：if(colmap_output_en)，按照 colmap 格式将相机相关参数输入到 images.txt 文件中。

```
pinhole_cam = dynamic_cast<vk::PinholeCamera*>(cam);
fout_colmap.open(DEBUG_FILE_DIR("Colmap/sparse/0/images.txt"),
ios::out);fout_colmap << "# Image list with two lines of data per image:\n";fout_colmap
<< "# IMAGE_ID, QW, QX, QY, QZ, TX, TY, TZ, CAMERA_ID,
NAME\n";fout_colmap << "# POINTS2D[] as (X, Y,
POINT3D_ID)\n";fout_camera.open(DEBUG_FILE_DIR("Colmap/sparse/0/cameras.txt"
), ios::out);fout_camera << "# Camera list with one line of data per
camera:\n";fout_camera << "# CAMERA_ID, MODEL, WIDTH, HEIGHT,
PARAMS[]\n";fout_camera << "1 PINHOLE " << width << " " << height << " " <<
std::fixed << std::setprecision(6) // 控制浮点数精度为 10 位 << fx << " " << fy << " " <<
cx << " " << cy << std::endl;fout_camera.close();
```

1.1.10.4.3.12 12、按照栅格的数量对相关容器大小进行重置。grid_num, map_index, map_dist, update_flag, scan_value。

```
grid_num.resize(length); // 判断当前栅格的类型 map_index.resize(length); //似乎并未
用到 map_dist.resize(length); // 图像到视觉点的距离 update_flag.resize(length); //给当
前视觉点生成新图像块的标志。scan_value.resize(length); //视觉点对应图像像素的角
点分数
```

1.1.10.4.3.13 13、图像块所拥有的像素总数。

```
patch_size_total = patch_size * patch_size;
```

1.1.10.4.3.14 14、图像块边长的一半

```
: patch_size_half = static_cast<int>(patch_size / 2);
```

1.1.10.4.3.15 15、按图像块像素总数对块缓存器大小重置：

- patch_buffer.resize(patch_size_total);
- 1.1.10.4.3.16 16、金字塔所有层的图像块的像素总和：
warp_len = patch_size_total * patch_pyramid_level;
- 1.1.10.4.3.17 17、边界计算
border = (patch_size_half + 1) * (1 << patch_pyramid_level);在某个金字塔级别下，处理块时需要的边界范围，确保在操作时不会超出图像或数据的有效区域。（设块的中心为参考点的位置，将最高层的金字塔边缘的图像块中心投影到原图像后，其投影中心点到原图像边的距离为 border，也就是说在[border, w-border],[border, h-border]范围内选中心点，其对应的图像块投影到原图像上才不会越界。）
- 1.1.10.4.3.18 18、按照栅格的数量重置候选体素点容器的大小：retrieve_voxel_points.reserve(length);
- 1.1.10.4.3.19 19、按照栅格的数量重置增添体素点容器大小：append_voxel_points.reserve(length);
- 1.1.10.4.3.20 20、清空子特征图
sub_feat_map.clear();判断该体素点对应的体素是否位于视野内。
- 1.1.10.5 5、对p_imu的相关参数进行设置
p_imu->set_extrinsic(extT, extR); // lidar 到 IMU 外参 p_imu->set_gyr_cov_scale(V3D(gyr_cov, gyr_cov, gyr_cov)); // 陀螺仪协方差 p_imu->set_acc_cov_scale(V3D(acc_cov, acc_cov, acc_cov)); // 加速度协方差 p_imu->set_inv_expo_cov(inv_expo_cov); // 逆曝光时间 p_imu->set_gyr_bias_cov(V3D(0.0001, 0.0001, 0.0001)); // 陀螺仪偏差的协方差 p_imu->set_acc_bias_cov(V3D(0.0001, 0.0001, 0.0001)); // 加速度偏差的协方差 p_imu->set_imu_init_frame_num(imu_init_frame); // imu 初始化所需帧数以上函数均声明于 IMU_Processing.h，定义于 IMU_Processing.cpp
- 1.1.10.5.1 1、设置 lidar 到 imu 外参
void ImuProcess::set_extrinsic(const V3D &transl, const M3D &rot){Lid_offset_to_IMU = transl;Lid_rot_to_IMU = rot;}
- 1.1.10.5.2 2、陀螺仪协方差
void ImuProcess::set_gyr_cov_scale(const V3D &scaler) { cov_gyr = scaler; }
- 1.1.10.5.3 3、加速度协方差
void ImuProcess::set_acc_cov_scale(const V3D &scaler) { cov_acc = scaler; }
- 1.1.10.5.4 4、逆曝光时间
void ImuProcess::set_inv_expo_cov(const double &inv_expo) { cov_inv_expo = inv_expo; }
- 1.1.10.5.5 5、陀螺仪偏差的协方差
void ImuProcess::set_gyr_bias_cov(const V3D &b_g) { cov_bias_gyr = b_g; }
- 1.1.10.5.6 6、加速度偏差的协方差
void ImuProcess::set_acc_bias_cov(const V3D &b_a) { cov_bias_acc = b_a; }
- 1.1.10.5.7 7、imu 初始化所需帧数
void ImuProcess::set_imu_init_frame_num(const int &num) { MAX_INI_COUNT = num; }
- 1.1.10.6 6、根据相关项是否使能，对一些项进行设置。
if (!imu_en) p_imu->disable_imu(); // 不用 imuif (!gravity_est_en) p_imu->disable_gravity_est();
// 不估计重力 if (!ba_bg_est_en) p_imu->disable_bias_est(); // 不估计偏差 if (!exposure_estimate_en) p_imu->disable_exposure_est(); // 不估计曝光默认配置下，以上条件均不执行。笔者疑惑的是，若 imu_en=false，则执行 p_imu->disable_imu()，而 p_imu->disable_imu()中又执行了 imu_en = false;（可以注释掉）还有 imu_need_init = false。
- 1.1.10.7 7、根据可用的传感器判断使用什么模式。（本解析默认 LIVO 模式）
slam_mode_ = (img_en && lidar_en) ? LIVO : imu_en ? ONLY_LIO : ONLY_LO;

1.2 2、mapper.initializeSubscribersAndPublishers(nh, it)

声明于 LIVMapper.h，定义于 LIVMapper.cpp.对应函数为：void LIVMapper::initializeSubscribersAndPublishers(ros::NodeHandle &nh, image_transport::ImageTransport &it)此处对订阅器和发布者初始化进行解析。主要做了以下几个事。1、将 lidar 点云转换为 pcl 格式后，将点云（每个点均有偏移时间）及帧头时间推入对应的缓存器 lid_raw_data_buffer 和 lid_header_time_buffer。2、将对齐到 lidar 帧头时间的 imu 推入其缓存器 imu_buffer，最近的 imu 时间戳更新 last_timestamp_imu。3、修正图像实际的触发时间，并将其转为 cv 格式分布存入对应的缓存器 img_buffer，img_time_buffer。

1.2.1 1、lidar 消息订阅及回调函数

sub_pcl = p_pre->lidar_type == AVIA ? nh.subscribe(lid_topic, 200000, &LIVMapper::livox_pcl_cbk, this): // ※nh.subscribe(lid_topic, 200000, &LIVMapper::standard_pcl_cbk, this);本解析以 avia 的回调函数为例。对应函数为: void LIVMapper::livox_pcl_cbk(const livox_ros_driver::CustomMsg::ConstPtr &msg_in)声明于 LIVMapper.h, 定义于 LIVMapper.cpp 将 lidar 点云转换为 pcl 格式后, 将点云 (每个点均有偏移时间) 及帧头时间推入对应的缓存器。

1.2.1.1 1、未使用雷达返回即可 if (!lidar_en) return;

1.2.1.2 2、获取互斥锁 mtx_buffer.lock(); 这里并没有多线程

尝试获取 mtx_buffer 的锁。如果锁当前是空闲的 (未被其他线程持有), 调用线程会成功获取锁, 并继续执行后续代码。如果锁已经被其他线程持有, 当前线程会被阻塞, 直到锁被释放。

1.2.1.3 3、重置一个新的 avia 自定义消息

livox_ros_driver::CustomMsg::Ptr msg(new livox_ros_driver::CustomMsg(*msg_in));

1.2.1.4 4、判断激光雷达与 IMU 时间差

if (abs(last_timestamp_imu - msg->header.stamp.toSec()) > 1.0 && !imu_buffer.empty())最近一次的 IMU 时间戳与激光雷达时间戳差值大于 1S, 且 IMU 不空计算 imu 对雷达的时间戳偏移并用亮紫色打印出来。double timediff_imu_wrt_lidar = last_timestamp_imu - msg->header.stamp.toSec();printf("\033[95mSelf sync IMU and LiDAR, HARD time lag is %.10lf\n\033[0m", timediff_imu_wrt_lidar - 0.100);

1.2.1.5 5、记录当前雷达时间戳并打印出来

double cur_head_time = msg->header.stamp.toSec();ROS_INFO("Get LiDAR, its header time: %.6f", cur_head_time);

1.2.1.6 6、判断雷达是否出现时间戳回滚 if (cur_head_time < last_timestamp_lidar)

打印并清空激光雷达数据缓存器 lid_raw_data_buffer.clear()

1.2.1.7 7、时间没有什么问题后 lidar 数据处理器调用处理函数

p_pre->process(msg, ptr); //PointCloudXYZI::Ptr ptr(new PointCloudXYZI()); 对应函数: void Preprocess::process(const livox_ros_driver::CustomMsg::ConstPtr &msg, PointCloudXYZI::Ptr &pcl_out)声明于 preprocess.h, 定义于 preprocess.cpp 目的: 将自定义消息类型转为 pcl 格式, 并将非盲区的 pcl 点给 ptr, 由于 Fast-LIVO2 不涉及特征提取, 因此特征提取部分读者可以略过。

1.2.1.7.1 1、针对 avia 雷达点云的处理

avia_handler(msg)对应函数: void Preprocess::avia_handler(const livox_ros_driver::CustomMsg::ConstPtr &msg)声明于 "preprocess.h" 在 preprocess.cpp 完成定义

1.2.1.7.1.1 1、清空三种点云类型: 平面, 角点, 所有点

PointCloudXYZI pl_full, pl_corn, pl_surf;pl_surf.clear(); //声明于 "preprocess.h"pl_corn.clear(); //声明于 "preprocess.h"pl_full.clear(); //声明于 "preprocess.h"

1.2.1.7.1.2 记录当前时刻, double t1 = omp_get_wtime();

1.2.1.7.1.3 2、当前帧点的数量并打印 int plsize = msg->point_num;

printf("[Preprocess] Input point number: %d\n", plsize);

1.2.1.7.1.4 3、根据点数设置点云容量

pl_corn.reserve(plsize);pl_surf.reserve(plsize);pl_full.resize(plsize);

1.2.1.7.1.5 4、根据点云线数对每个线设置等同的容量 pl_buff 每个线束的点云, 分别存入。

for (int i = 0; i < N_SCANS; i++){ pl_buff[i].clear(); pl_buff[i].reserve(plsize);} avia 是 6 线, 相当于将每一线当成一帧点云来看待

1.2.1.7.1.6 5、有效线数 uint valid_num = 0;

1.2.1.7.1.7 6、判断是否进行特征提取 if (feature_enabled)---可以略过

然而在 yaml 文件中并没有设置这个参数变量, 因此采用默认值 falsenh.param<bool>("preprocess/feature_extract_enabled", p_pre->feature_enabled, false);

1.2.1.7.1.7.1 1、从第二个点循环该帧点数 `for (uint i = 1; i < plsize; i++)` 每个线束的点云，分别存入 `pl_buff` 中。

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
保留位		回波序号：		基于强度的点属性：		基于空间位置的点属性：	
		00: 第 0 个回波		00 正常点		00 正常点	
		01: 第 1 个回波		01 回波能量噪点置信度高		01 空间噪点置信度高	
		10: 第 2 个回波		10 回波能量噪点置信度中		10 空间噪点置信度中	
		11: 第 3 个回波		11 保留位		11 空间噪点置信度低	

判断当前点的线号超出否并且当前点标志属于什么 `if ((msg->points[i].line < N_SCANS) && ((msg->points[i].tag & 0x30) == 0x10))` 只有 00010000 与上 00110000 == 00010000，即只考虑第一次回波点。

1.2.1.7.1.7.1.1 1、取出消息中点的属性：位置，反射强度，时间戳

```
pl_full[i].x = msg->points[i].x; pl_full[i].y = msg->points[i].y; pl_full[i].z = msg->points[i].z;
pl_full[i].intensity = msg->points[i].reflectivity; pl_full[i].curvature = msg->points[i].offset_time / float(1000000);
// use curvature as time of each laser points curvature 单位是 ms
```

1.2.1.7.1.7.1.2 2、`bool is_new = false;` 未用到

1.2.1.7.1.7.1.3 3、与上一个点距离满足一定条件再讲点存入点云缓存器对应的线容器中

```
if ((abs(pl_full[i].x - pl_full[i - 1].x) > 1e-7) || (abs(pl_full[i].y - pl_full[i - 1].y) > 1e-7) ||
    (abs(pl_full[i].z - pl_full[i - 1].z) > 1e-7)) { pl_buff[msg->points[i].line].push_back(pl_full[i]); }
```

1.2.1.7.1.7.2 2、静态变量 `static int count = 0;` `static double time = 0.0;`

每次调用 函数 时，静态的值都会保留，并且不会因为函数退出而丢失。
`count++; double t0 = omp_get_wtime();`

1.2.1.7.1.7.3 3、循环 整个点云的每个线束 `for (int j = 0; j < N_SCANS; j++)`

获得每个线束点云的点特征，并存入相应的点云容器中：`pl_surf;` `pl_corn。`

1.2.1.7.1.7.3.1 1、如果该线束上的点不足 6 个，处理下一个线束 `if (pl_buff[j].size() <= 5)`
`continue;`

1.2.1.7.1.7.3.2 2、引用该线束上的所有点 `pcl::PointCloud<PointType> &pl = pl_buff[j];`
`plsize = pl.size();`

1.2.1.7.1.7.3.3 3、初始化一下该线束上的 org 类型（原始点属性？）`vector<orgtype> &types = typess[j];`

```
types.clear(); types.resize(plsize); 其中 typess 定义于 "preprocess.h"
vector<orgtype> typess[128]; // maximum 128 line lidar
```

1.2.1.7.1.7.3.3.1 关于 org 属性结构体 `struct orgtype` 位于论文的雷达点处理部分

```
{double range; // 测距 double dista; // 与下一个点的欧式距离 double
```



```
angle[2]; // 激光射线方向与邻点向量角度 cosdouble intersect; // 邻点向量
间角度 cosE_jump edj[2]; // 盲区点, 无限点, 近 180, 近 0, 近 90。
LiDARFeature ftype;orgtype() {range = 0;edj[Prev] = Nr_nor;edj[Next] =
Nr_nor;ftype = Nor;intersect = 2; };
```

1.2.1.7.1.7.3.4 4、plsize--; 循环当前线束上的所有点 for (uint i = 0; i < plsize; i++)

```
types[i].range = pl[i].x * pl[i].x + pl[i].y * pl[i].y; // 当前点的测距值 vx =
pl[i].x - pl[i + 1].x; // 与下一个点的 x 距离 vy = pl[i].y - pl[i + 1].y; // 与下
一个点的 y 距离 vz = pl[i].z - pl[i + 1].z; // 与下一个点的 z 距离
types[i].dista = vx * vx + vy * vy + vz * vz; // 与下一个点的欧式距离
```

1.2.1.7.1.7.3.5 5、最后一个点的测距值

```
types[plsize].range = pl[plsize].x * pl[plsize].x + pl[plsize].y * pl[plsize].y;
```

1.2.1.7.1.7.3.6 6、输入当前线数点云和对应属性给出特征点 give_feature(pl, types);

```
void Preprocess::give_feature(pcl::PointCloud<PointType> &pl, vector<orgtype>
&types)定义于 preprocess.cpp 目的: 获得 enum LiDARFeature 中设置的各种
特征: 无特征, 可能平面, 真平面, 边缘跳跃, 边缘平面, 导线。并存入
相应的点云容器中: pl_surf; pl_corn;
```

1.2.1.7.1.7.3.6.1 1、获得当前线束点云的数量 int plsize = pl.size(); int plsize2;

```
没有点云时返回 (之前的判断应该能够避免这个问题) if (plsize == 0)
{printf("something wrong\n");return; }
```

1.2.1.7.1.7.3.6.2 2、判断每个点的测距值是否小于盲区平方, 直至找到盲区外的第一个激光点

```
uint head = 0;while (types[head].range < blind_sqr) { head++; }
```

1.2.1.7.1.7.3.6.3 3、设置 plsize2 的数量为 plsize1 减去 group_size 的或 0 **plsize2 = (plsize > group_size) ? (plsize - group_size) : 0; group_size=8**。特殊含义: 在后续判断平面 plane_judge 中需要从当前点往后加 8 个点为一组, 所以需留出最后八个点。注意, group_size=8 只是一组点的数量下限, 只要不超出组内两点最大距离, 便可视为组内点。

```
int group_size;定义于定义于 preprocess.h
```

1.2.1.7.1.7.3.6.4 4、当前方向和上次方向 初始化 (指的是当前点与组内最远点构成的向量方向)

```
Eigen::Vector3d curr_direct(Eigen::Vector3d::Zero());Eigen::Vector3d
last_direct(Eigen::Vector3d::Zero());
```

1.2.1.7.1.7.3.6.5 5、相关状态变量, 每个变量含义?

```
uint i_nex = 0, i2; // 当前点对应的最远点索引, i2 用于保存当前循环点
索引 uint last_i = 0; // 上一个循环点索引 (每次循环是跳着来的, 跳过
该组所有点) uint last_i_nex = 0; // 上个循环点对应的最远点索引 int
last_state = 0; // 上一个循环点的平面状态 int plane_type; // 平面类型 0 1
2. 只有 1 代表平面点。
```

1.2.1.7.1.7.3.6.6 6、从第一个非盲区点循环至 plsize2。 for (uint i = head; i < plsize2; i++)

目的：根据当前点获得一组点，判断组内点是否为真平面 Real_Plane，以及当前点是真平面还是边缘平面 Edge_Plane，还是无平面特征 Nor。

1.2.1.7.1.7.3.6.7 1、怎么又作了一次盲区判断 if (types[i].range < blind_sqr) { continue; }

前面的操作已经避免了盲区点吧

1.2.1.7.1.7.3.6.8 2、当前循环点在 types 中的索引给 i2

1.2.1.7.1.7.3.6.9 3、判断平面，传入当前线束点云，类型，当前点索引，下一个点索引，当前方向等变量。plane_type = plane_judge(pl, types, i, i_nex, curr_direct); 组内有盲区点则直接退出，返回 2 或 0。平面点则返回 1。这里只能判断可能为平面点。

int Preprocess::plane_judge(const PointCloudXYZI &pl, vector<orgtype> &types, uint i_cur, uint &i_nex, Eigen::Vector3d &curr_direct)定义于 preprocess.cpp 绿色标注表示为平面点评判标准：1、找到的最大法向量模不能太大（构成的平行四边形的高不能太大）；2、组内的点间距离变化不能太大。

1.2.1.7.1.7.3.6.10 1、计算组内两点最大距离限制 double group_dis = disA * types[i_cur].range + disB; group_dis = group_dis * group_dis; 一个线性公式。根据当前点的测距计算出包含该点的点云组中，两点间最大距离应是多少。当前点测距值越远，组内两点最大距离就越大。

然而在 Preprocess::Preprocess()初始化中，并没有 disB，并且重复定义了两个 disA。 disA = 0.01; disA = 0.1; // B?

1.2.1.7.1.7.3.6.11 2、定义两个点距离 two_dis 以及距离容器 disarr，大小 20。

double two_dis; vector<double> disarr;disarr.reserve(20);

1.2.1.7.1.7.3.6.12 3、从当前点循环至当前点后 8 个点。即一组 8 个点 for (i_nex = i_cur; i_nex < i_cur + group_size; i_nex++) group_size=8。 返回 2.

判断盲区，则返回 2.非盲区则将这一组内的点到下一个点距离推入 距离容器 if (types[i_nex].range < blind_sqr) {curr_direct.setZero();return 2; }disarr.push_back(types[i_nex].dista);

1.2.1.7.1.7.3.6.13 4、无限循环 for (;;)

接着组外继续找点，直至这个点 i_nex 超出了组内两点最大距离。

1.2.1.7.1.7.3.6.14 1、如果当前点索引/下个点索引为点云大小则退出该循环。

1.2.1.7.1.7.3.6.15 2、判读盲区

if (types[i_nex].range < blind_sqr) {curr_direct.setZero();return 2; }

1.2.1.7.1.7.3.6.16 3、计算 i_nex 到 i_cur 的欧式距离，判断是否超出最大组内距离，

vx = pl[i_nex].x - pl[i_cur].x; vy = pl[i_nex].y - pl[i_cur].y; vz =

```
pl[i_nex].z - pl[i_cur].z; two_dis = vx * vx + vy * vy + vz * vz; // 当前点到最远点的模平方 if (two_dis >= group_dis) { break; }
```

1.2.1.7.1.7.3.6.17 4、将下一个点推入距离容器中

```
disarr.push_back(types[i_nex].dista); i_nex++;
```

1.2.1.7.1.7.3.6.18 5、初始化了长宽量和距离差数组

```
double leng_wid = 0; 法向量最长的模 double v1[3], v2[3]; // 当前点到某点向量, 法向量假设 i_cur 点为 A, j 点为 B, i_nex 点为 C
```

1.2.1.7.1.7.3.6.19 6、从当前点的下一个开始循环一直到组内最后一点。for (uint j = i_cur + 1; j < i_nex; j++)

计算组内最远点-当前点 向量 叉乘 组内某点-当前点向量 得到 法向量, 并获取最长的模值。leng_wid 此时获得的两个向量的角度是最接近垂直的 (越垂直值越大) 。

1.2.1.7.1.7.3.6.20 1、判断是否超出该线束点云上限 if ((j >= pl.size()) || (i_cur >= pl.size())) break;

1.2.1.7.1.7.3.6.21 2、计算组内每个点到当前点的各轴距离或者向量

```
v1[0] = pl[j].x - pl[i_cur].x; v1[1] = pl[j].y - pl[i_cur].y; v1[2] = pl[j].z - pl[i_cur].z;
```

1.2.1.7.1.7.3.6.22 3、某点到当前点的向量与最远点到当前点向量的叉乘。V2 是法向量

```
v2[0] = v1[1] * vz - vy * v1[2]; v2[1] = v1[2] * vx - v1[0] * vz; v2[2] = v1[0] * vy - vx * v1[1]; 结果是一个垂直于两个输入向量的向量
```

1.2.1.7.1.7.3.6.23 4、法向量 v2 的模平方 double lw = v2[0] * v2[0] + v2[1] * v2[1] + v2[2] * v2[2];

其长度等于 $|a \times b| = |a| * |b| * \sin(\theta)$ 物理意义是组成的 ABC 组成的平行四边形面积的平方 $(|AC| * h)^2$, h 是 B 点到 AC 的距离

1.2.1.7.1.7.3.6.24 5、更新 模的平方到 leng_wid

值越大, 说明组内点的分布越倾向于形成一个平面 (因为叉乘结果的模长与平行四边形的面积成正比)

1.2.1.7.1.7.3.6.25 7、最远点向量模平方比法向量模平方后在乘以最远点向量模平方的值 与 p2l_ratio 比较 if ((two_dis * two_dis / leng_wid) < p2l_ratio) 。返回 0。

```
double p2l_ratio; // preprocess.h, 在 preprocess.cpp 的构造函数内定义为 225. //  $|AC| * |AC| / (h * h) < 225$ , 也就是  $h > |AC| / 15$ , B 点到 AC 的距离大于 AC 边长的 1/15, 这种情况下当前点非平面点。  
|curr_direct.setZero(); return 0;
```

1.2.1.7.1.7.3.6.26 8、把两点之间的距离 从大到小排序 uint disarrsize = disarr.size();

```
uint disarrsize = disarr.size();
for (uint j = 0; j < disarrsize - 1; j++)
{
    for (uint k = j + 1; k < disarrsize; k++)
    {
        if (disarr[j] < disarr[k])
        {
            leng_wid = disarr[j];
            disarr[j] = disarr[k];
            disarr[k] = leng_wid;
        }
    }
}
```

1.2.1.7.1.7.3.6.27 9、看看排到最后的两点距离是否够大，否则构不成平面点 返回 0

```
if (disarr[disarr.size() - 2] < 1e-16) {curr_direct.setZero();return 0; }
```

1.2.1.7.1.7.3.6.28 10、如果激光雷达是 avia 的话 if (lidar_type == AVIA)

点间距离变化剧烈则表示 可能与激光束是平行的？

1.2.1.7.1.7.3.6.29 1、计算距离数组头部到中间的比，中间到尾部的比值

```
double dismax_mid = disarr[0] / disarr[disarrsize / 2];double dismid_min =
disarr[disarrsize / 2] / disarr[disarrsize - 2];
```

1.2.1.7.1.7.3.6.30 2、判断比值是否满足比值阈值 if (dismax_mid >= limit_maxmid || dismid_min >= limit_midmin)。点与点之间距离变化太大的时候 可能与激光束是平行的 就也舍弃了

limit_maxmid limit_midmin 定义于 preprocess.cpp 中的构造函数中 值为 6.25

1.2.1.7.1.7.3.6.31 11、else 否则其他雷达则判断头部与尾部的比 limit_maxmin

```
double dismax_min = disarr[0] / disarr[disarrsize - 2];if (dismax_min >=
limit_maxmin) {curr_direct.setZero();return 0; }limit_maxmin 定义于
preprocess.cpp 中的构造函数中 值为 3.24
```

1.2.1.7.1.7.3.6.32 12、获得当前点的方向向量（当前点到最远点构成的向量）返回 1

```
curr_direct << vx, vy, vz;curr_direct.normalize();return 1;
```

1.2.1.7.1.7.3.6.33 4、判断是否平面点 if (plane_type == 1) 判断标准：上个点的方向向量与当前点的方向向量点乘，值越大越真平面，越小越边缘平面。

除去当前点和最远点的组内点都是真平面。当前点需要再次判断是真平面还是边缘平面。

1.2.1.7.1.7.3.6.34 1、从当前点开始循环至组内最远点 for (uint j = i; j <= i_nex; j++), 修改特征类型

只要是当前点和最远点之间的点都视为真平面。当前点和最远点视为可能的平面。**（需要进一步判断）** if (j != i && j != i_nex) { types[j].ftype = Real_Plane; }else { types[j].ftype = Poss_Plane; }

1.2.1.7.1.7.3.6.35 2、判断是否上个点的状态（是否平面）为 1 且 上个点的方向向量模值>0.1m

if (last_state == 1 && last_direct.norm() > 0.1)当前点的方向向量与上一个点的方向向量越接近（越共线）则越真平面。反之，越小（越垂直）则为边缘平面。

1.2.1.7.1.7.3.6.36 1、上个点方向向量与当前点方向向量的点积。越小越接近垂直 double mod =

last_direct.transpose() * curr_direct;

1.2.1.7.1.7.3.6.37 2、平面边缘点判断 if (mod > -0.707 && mod < 0.707)

types[i].ftype = Edge_Plane;

1.2.1.7.1.7.3.6.38 3、若点积值较大（越重合）则为 当前点真平面

types[i].ftype = Real_Plane;

1.2.1.7.1.7.3.6.39 3、更新当前点索引为组内最后一个点和上个点状态

i = i_nex - 1; last_state = 1;

1.2.1.7.1.7.3.6.40 5、否则 else, 更新当前点索引为最远点以及上个点的状态

这一组内的点就都不再进行特征点的考虑了。 i = i_nex; last_state = 0;

1.2.1.7.1.7.3.6.41 6、保留上一个点的索引, 上个点对应最远点索引以及上个点的方向向量

last_i = i2; last_i_nex = i_nex; last_direct = curr_direct;

1.2.1.7.1.7.3.6.42 7、设置 plsize2 的数量为 plsize1 减去 3 的或 0 plsize2 = plsize > 3 ? plsize - 3 : 0;
类似第 3 项

用于判断边缘跳变点中获得上上个点留出冗余。

1.2.1.7.1.7.3.6.43 8、从第一个非盲区点+3 循环至 plsize2 for (uint i = head + 3; i < plsize2; i++)

目的：根据激光射线方向与邻点向量夹角，邻点向量间夹角以及当前点与邻点距离关系，判断边缘跳变点 Edge_Jump 还是导线 Wire 特征

1.2.1.7.1.7.3.6.44 1、若是盲区点或真平面 跳过，循环下个点 if (types[i].range < blind_sqr || types[i].ftype >= Real_Plane) { continue; }

1.2.1.7.1.7.3.6.45 2、上个点的点间距离和当前点的点间距离太小 则跳过 执行下个点 if (types[i - 1].dista < 1e-16 || types[i].dista < 1e-16) { continue; }

1.2.1.7.1.7.3.6.46 3、该线束点云的当前点位置 及向量数组初始化

```
Eigen::Vector3d vec_a(pl[i].x, pl[i].y, pl[i].z); Eigen::Vector3d vecs[2]; // 与邻点构成的向量
```

1.2.1.7.1.7.3.6.47 4、循环两个点 for (int j = 0; j < 2; j++)

判断当前点激光射线方向与上个点/下个点向量的边缘关系 edj[]: 盲区、边界、夹角等。

1.2.1.7.1.7.3.6.48 1、以 j 为条件，设置 m 的值。m 为 -1 或 1

1.2.1.7.1.7.3.6.49 2、判断当前点的前一个点和下一个点是否为盲区点 if (types[i + m].range < blind_sqr)，是则再判断当前点是否超出边界 inf_bound=10m，是则设置 edj 值为 Nr_inf 和 Nr_blind。

```
if (types[i].range > inf_bound) { types[i].edj[j] = Nr_inf; } else { types[i].edj[j] = Nr_blind; } continue; 其中 inf_bound 定义于 preprocess.cpp 构造函数，edj 则为 enum E_jump 的数组。
```

1.2.1.7.1.7.3.6.50 3、计算上一个点和下个点与当前点的向量并存入 vecs

```
vecs[j] = Eigen::Vector3d(pl[i + m].x, pl[i + m].y, pl[i + m].z); vecs[j] = vecs[j] - vec_a;
```

1.2.1.7.1.7.3.6.51 4、当前点激光线方向与邻点向量的夹角 cos 值，判断是两者夹角靠近 180 度还是 0 度 types[i].angle[j] = vec_a.dot(vecs[j]) / vec_a.norm() / vecs[j].norm();

```
if (types[i].angle[j] < jump_up_limit) { types[i].edj[j] = Nr_180; } else if (types[i].angle[j] > jump_down_limit) { types[i].edj[j] = Nr_zero; }
```

1.2.1.7.1.7.3.6.52 5、计算 当前点的两个邻点向量之间的夹角 cos 值，types[i].intersect = vecs[Prev].dot(vecs[Next]) / vecs[Prev].norm() / vecs[Next].norm();

1.2.1.7.1.7.3.6.53 6、判断 射线方向与上一个点向量为接近 90 度（8 度 ~ 170 度）且与下个点向量接近 0 度（小于 8 度）且与下个点距离超过 2.25 厘米 且 大于 4 倍的与上个点距离

```
if (types[i].edj[Prev] == Nr_nor && types[i].edj[Next] == Nr_zero && types[i].dista > 0.0225 && types[i].dista > 4 * types[i - 1].dista)
```

1.2.1.7.1.7.3.6.54 判断 当前点的两个邻点向量之间的夹角 cos 值 是否大与 cos160 度。if (types[i].intersect > cos160) Preprocess::edge_jump_judge

是，则进行边缘跳跃点评判 if (edge_jump_judge(pl, types, i, Prev))

```

{ types[i].ftype = Edge_Jump; } // bool
Preprocess::edge_jump_judge(const PointCloudXYZI &pl,
vector<orgtype> &types, uint i, Surround nor_dir)定义于
preprocess.cpp

```

1.2.1.7.1.7.3.6.55 1、判断如果是与上一个点的边缘属性 if (nor_dir == 0)，则判断上个点或者上上个点的测距是否盲区，满足则返回 **false**

```

if (types[i - 1].range < blind_sqr || types[i - 2].range < blind_sqr) { return
false; }

```

1.2.1.7.1.7.3.6.56 2、否则再判断 是否与下一个点的边缘属性 else if (nor_dir == 1)，则判断下个点或者下下个点的测距是否盲区，满足则返回 **false**

```

if (types[i + 1].range < blind_sqr || types[i + 2].range < blind_sqr) { return
false; }

```

1.2.1.7.1.7.3.6.57 3、获得当前点与相邻点的距离 double d1 = types[i + nor_dir - 1].dista;

1.2.1.7.1.7.3.6.58 4、上个点与上上个点距离 | 下个点与下下个点距离 double d2 = types[i + 3 * nor_dir - 2].dista;

1.2.1.7.1.7.3.6.59 5、判断，是否 当前点与邻点距离值小于 上个点与上上个点距离 | 下个点与下下个点距离 if (d1 < d2)， 调换距离值

```

d = d1; d1 = d2; d2 = d;

```

1.2.1.7.1.7.3.6.60 6、对距离值开根号

```

d1 = sqrt(d1); d2 = sqrt(d2);

```

1.2.1.7.1.7.3.6.61 7、判断，大的距离值 大于 两倍的小距离值 或者 两个距离值差距大于 10 厘米 则认为非边缘跳变点 if (d1 > edgea * d2 || (d1 - d2) > edgeb) { return false; }， 否则为边缘跳变点。

edg ea=2 edg eb = 0.1 定义于 preprocess.cpp 的构造函数。

1.2.1.7.1.7.3.6.62 7、否则再判断 射线方向与上一个点向量为接近 0 度（小于 8 度） 且 与下个点向量接近 90 度（8 度 ~ 170 度） 且 与上个点距离超过 2.25 厘米 且 大于 4 倍的 与下个点距离

1.2.1.7.1.7.3.6.63 判断 当前点的两个邻点向量之间的夹角 cos 值 是否大与 cos160 度。 if (types[i].intersect > cos160)

```

是， 则进行边缘跳跃点评判 if (edge_jump_judge(pl, types, i, Next))
{ types[i].ftype = Edge_Jump; } // bool
Preprocess::edge_jump_judge(const PointCloudXYZI &pl,
vector<orgtype> &types, uint i, Surround nor_dir)定义于
preprocess.cpp

```


1.2.1.7.1.7.3.6.64 8、否则再判断 射线方向与上一个点向量为接近 90 度 (8 度 ~ 170 度小于 8 度)
且 当前点与下个点距离过远 (一个位于盲区, 一个位于界外)

1.2.1.7.1.7.3.6.65 进行边缘跳跃评判 if (edge_jump_judge(pl, types, i, Prev)) { types[i].ftype =
Edge_Jump; }

1.2.1.7.1.7.3.6.66 9、否则再判断 当前点与上个点距离过远 (一个位于盲区, 一个位于界外) 且
射线方向与下一个点向量为接近 90 度 (8 度 ~ 170 度)

1.2.1.7.1.7.3.6.67 进行边缘跳跃评判 if (edge_jump_judge(pl, types, i, Next)) { types[i].ftype =
Edge_Jump; }

1.2.1.7.1.7.3.6.68 10、否则再判断 与上/下一个点的边缘属性都不是接近 90 度 (8 度 ~ 170 度)
else if (types[i].edj[Prev] > Nr_nor && types[i].edj[Next] > Nr_nor)

1.2.1.7.1.7.3.6.69 如果 激光点特征属性为 无, 则设该点为导线特征。if (types[i].ftype == Nor)
{ types[i].ftype = Wire; }

1.2.1.7.1.7.3.6.70 9、设置 plsize2 的数量为 plsize1 减去 1 并设置一个比率 plsize2 = plsize - 1;
double ratio;

为下方循环中的上个点留出冗余。

1.2.1.7.1.7.3.6.71 10、从第一个非盲区点+1 开始循环至 plsize2 for (uint i = head + 1; i < plsize2;
i++)

目的: 若当前点特征标志为无, 则根据邻点向量夹角大于 172.5 度, 且
与邻点距离的大/小 比值 小于 1.2。设当前点特征就算是 真平面点。如
若 邻点特征也都是 无, 则 将邻点也都设为 真平面。

1.2.1.7.1.7.3.6.72 1、判断当前点和邻点是否为盲区点 if (types[i].range < blind_sqr || types[i - 1].range < blind_sqr || types[i + 1].range < blind_sqr) { continue; }

1.2.1.7.1.7.3.6.73 2、判断 与上个点距离，与下个点距离 是否足够大 if (types[i - 1].dista < 1e-8 || types[i].dista < 1e-8) { continue; }

1.2.1.7.1.7.3.6.74 3、判断 当前激光点是否无特征 if (types[i].ftype == Nor)

1.2.1.7.1.7.3.6.75 1、判断 与上个点的距离 大于 与下个点的距离 ，比率则为 与上个点距离/与下个点距离 if (types[i - 1].dista > types[i].dista) { ratio = types[i - 1].dista / types[i].dista; }

1.2.1.7.1.7.3.6.76 2、否则 比率则为 与下个点距离/与上个点距离 else { ratio = types[i].dista / types[i - 1].dista; }

1.2.1.7.1.7.3.6.77 3、判断 两个邻点向量之间的 cos 值 是否小与 172.5 度对应的 cos--smallp_intersect，比率是否 小与 smallp_ratio=1.2 。 if (types[i].intersect < smallp_intersect && ratio < smallp_ratio)。 邻点向量夹角要大于 172.5 度。

其中 smallp_intersect smallp_ratio 定义于 preprocess.cpp

1.2.1.7.1.7.3.6.78 1、判断 上个点 的激光特征是否无，若是 则上个点设为真平面。 if (types[i - 1].ftype == Nor) { types[i - 1].ftype = Real_Plane; }

1.2.1.7.1.7.3.6.79 2、判断下个点的激光特征是否无，若是 则下个点设为真平面。 if (types[i + 1].ftype == Nor) { types[i + 1].ftype = Real_Plane; }

1.2.1.7.1.7.3.6.80 3、当前点也设为真平面 types[i].ftype = Real_Plane;

1.2.1.7.1.7.3.6.81 11、定义一个变量，上一个平面 int **last_surface** = -1;

用于跳过 point_filter_num 个点的标志。

1.2.1.7.1.7.3.6.82 12、从非盲区点开始循环至该线束整个点云 for (uint j = head; j < plsize; j++)

目的：将判断为真平面和可能平面的点都推入 pl_surf 中，将边缘跳跃点和边缘平面点都推入 pl_corn 中，将剩下的不够 point_filter_num 的点作归一化处理作为 1 个点推入 pl_surf 中。

1.2.1.7.1.7.3.6.83 1、判断 若当前点的特征类型为可能平面或者真平面 if (types[j].ftype == Poss_Plane || types[j].ftype == Real_Plane)

1.2.1.7.1.7.3.6.84 1、判断 若上个平面变量值为-1 则将当前点索引给它 if (last_surface == -1)
{ last_surface = j; }

1.2.1.7.1.7.3.6.85 2、判断 当前点索引是否== 上个平面索引+点滤波数量-1。意思是每次跳过 point_filter_num 个点进行判断体内的操作-推入当前点到 pl_surf 容器。

point_filter_num 来自 yaml 文件，点滤波数量 在 void Preprocess::set 函数中被设置为 point_filter_num = pfilt_num; 目前并未涉及该函数，故 point_filter_num 仍为 yaml 文件中的值。

1.2.1.7.1.7.3.6.86 1、定义一个点 PointType ap;

1.2.1.7.1.7.3.6.87 2、将当前点的属性赋予该定义点，并推入平面点云容器 pl_surf 中。重置 last_surface

```
PointType ap; ap.x = pl[j].x; ap.y = pl[j].y; ap.z = pl[j].z; ap.curvature =  
pl[j].curvature; pl_surf.push_back(ap); last_surface = -1;
```

1.2.1.7.1.7.3.6.88 2、否则 esle 进行另外特征判断

1.2.1.7.1.7.3.6.89 1、判断 当前点特征标志 为边缘跳跃 或者 边缘平面 是则推入角点点云容器中。
if (types[j].ftype == Edge_Jump || types[j].ftype == Edge_Plane)
{ pl_corn.push_back(pl[j]); }

1.2.1.7.1.7.3.6.90 2、判断 上个平面标志 是否不为-1 （有索引赋值过） if (last_surface != -1)

这是接着上一级的 1 进行的，若上一级中的 j 不满足跳过 point_filter_num 个点就会退出判断，那么剩下的不够 point_filter_num 的点也需要被处理，这里就是处理这些点的。

1.2.1.7.1.7.3.6.91 1、定义一个点 PointType ap;

1.2.1.7.1.7.3.6.92 2、从上个索引开始循环至当前索引前个点 for (uint k = last_surface; k < j; k++),
将点云中点的属性都给这个新定义点

```
ap.x += pl[k].x; ap.y += pl[k].y; ap.z += pl[k].z; ap.curvature +=  
pl[k].curvature;
```

1.2.1.7.1.7.3.6.93 3、将累加点进行归一化并推入 平面点云容器

```
ap.x /= (j - last_surface); ap.y /= (j - last_surface); ap.z /= (j -  
last_surface); ap.curvature /= (j - last_surface); pl_surf.push_back(ap);
```

1.2.1.7.1.7.3.6.94 3、重置上个平面索引标志 last_surface = -1;

1.2.1.7.1.7.4 4、打印特征提取耗时

time += omp_get_wtime() - t0; printf("Feature extraction time: %lf\n", time / count);
1.2.1.7.1.8 8、不进行特征提取 else, 则循环当前帧点云所有点, 并判断点对应的线束是否小于总线束。

for (uint i = 0; i < plsize; i++) { if ((msg->points[i].line < N_SCANS)) { 判断点间时间间隔, 重整时间戳为 ms, 并记录每个点相对于帧头时间的偏移, 并按 point_filter_num 来过滤点, 然后存入 pl_surf。与 fast-lid2 不同的是, 它没有判断与上个点的距离间隔是否足够。

1.2.1.7.1.8.1 1、有效点总数累加。

valid_num++;

1.2.1.7.1.8.2 2、将 ros 消息中的点属性都给点云容器 pl_full, 包括每个点的偏移时间 ms

pl_full[i].x = msg->points[i].x; pl_full[i].y = msg->points[i].y; pl_full[i].z = msg->points[i].z; pl_full[i].intensity = msg->points[i].reflectivity; pl_full[i].curvature = msg->points[i].offset_time / float(1000000); // use curvature as time of each laser points

1.2.1.7.1.8.3 3、判断 若是第一个点 需考虑该点的时间戳是否超过 1ms, 超过则置 0

pl_full[i].curvature = fabs(pl_full[i].curvature) < 1.0 ? pl_full[i].curvature : 0.0;

1.2.1.7.1.8.4 4、否则 else 判断与上个点的间隔是否小于 1ms, 小于则不用变, 大于则上一点时间 + 0.00416667f 作为该点时间。0.00416667f = 100ms/24000 点, avia 每帧产生 24000 个点。

pl_full[i].curvature = fabs(pl_full[i].curvature - pl_full[i - 1].curvature) < 1.0 ? pl_full[i].curvature : pl_full[i - 1].curvature + 0.004166667f; // float(100/24000)

1.2.1.7.1.8.5 5、判断此时点数 valid_num 是否为 point_filter_num 整数倍, 并将非盲区的整数倍的点存入 pl_surf。 if (valid_num % point_filter_num == 0)

if (pl_full[i].x * pl_full[i].x + pl_full[i].y * pl_full[i].y + pl_full[i].z * pl_full[i].z >= blind_sqr) { pl_surf.push_back(pl_full[i]); }

1.2.1.7.2 2、将存入平面点云容器的数据给 pcl_out, *pcl_out = pl_surf;

实际上, 如果不提取点云特征的话, pl_surf 存的就是跳过 point_filter_num 的所有点。

1.2.1.8 8、如果点云没有则解开互斥锁, 之间返回 空

if (!ptr || ptr->empty()) { ROS_ERROR("Received an empty point cloud"); mtx_buffer.unlock(); return; }

1.2.1.9 9、将当前帧处理后的点云 推入 雷达原始数据缓存器

将每帧点云都保存着 lid_raw_data_buffer.push_back(ptr);

1.2.1.10 10、将每帧点云的帧头时间都推入雷达帧头时间缓存器

lid_header_time_buffer.push_back(cur_head_time);

1.2.1.11 11、更新当前 lidar 帧头时间到最近的 lidar 时间戳

last_timestamp_lidar = cur_head_time;

1.2.1.12 12、解开互斥锁并通知所有线程

```
mtx_buffer.unlock();sig_buffer.notify_all();
```

1.2.2 2、Imu 消息订阅及回调函数

sub_imu = nh.subscribe(imu_topic, 200000, &LIVMapper::imu_cbk, this);回调函数对应的函数为：
void LIVMapper::imu_cbk(const sensor_msgs::Imu::ConstPtr &msg_in)声明于 LIVMapper.h，定义于 LIVMapper.cpp 将对齐到 lidar 帧头时间的 imu 推入其缓存器 imu_buffer，最近的 imu 时间戳更新 last_timestamp_imu。

1.2.2.1 1、若未使用 IMU 则直接返回 if (!imu_en) return;

1.2.2.2 2、最近的雷达时间戳小于 0 直接返回 if (last_timestamp_lidar < 0.0) return;

1.2.2.3 3、将 Imu 时间戳对齐到 lidar 时间戳

```
msg->header.stamp = ros::Time().fromSec(msg->header.stamp.toSec() - imu_time_offset);
```

imu_time_offset 声明于 LIVMapper.h，来自 yaml 文件。

1.2.2.4 记录时间戳 double timestamp = msg->header.stamp.toSec();

1.2.2.5 4、判断是否于激光雷达 同步 不能超过 0.5 秒

```
if (fabs(last_timestamp_lidar - timestamp) > 0.5 && (!ros_driver_fix_en)) {ROS_WARN("IMU and LiDAR not synced! delta time: %lf \n", last_timestamp_lidar - timestamp); }ros_driver_bug_fix (ros_driver_fix_en) 来自 yaml 文件，声明于 LIVMapper.h 具体解释可以参考 yaml 文件。
```

1.2.2.6 5、若使用 ros 驱动修复使能 ros_driver_fix_en，将多出来的时间差四舍五入给时间戳。

```
if (ros_driver_fix_en) timestamp += std::round(last_timestamp_lidar - timestamp) ; msg->header.stamp = ros::Time().fromSec(timestamp);
```

1.2.2.7 6、时间没用问题后 进行互斥锁 mtx_buffer.lock();

1.2.2.8 7、imu 时间回滚。当前时间戳小于上一个时间戳

```
if (last_timestamp_imu > 0.0 && timestamp < last_timestamp_imu) {mtx_buffer.unlock();sig_buffer.notify_all();ROS_ERROR("imu loop back, offset: %lf \n", last_timestamp_imu - timestamp);return; }笔者在无人机飞行时常出现该问题，具体原因为供电不稳，导致 stm32 多次重启。
```

1.2.2.9 8、更新上个 IMU 时间戳，并推入 imu 消息到 imu 缓存器中。解开互斥锁

```
last_timestamp_imu = timestamp;imu_buffer.push_back(msg); // cout<<"got imu: "<<timestamp<<" imu size "<<imu_buffer.size()<<endl;mtx_buffer.unlock();
```

1.2.2.10 9、判断 imu 频率里程计 imu_rate_odom (imu_prop_enable) 使能 if (imu_prop_enable)。默认是关着的--可略过

用于给 UAV 提供高频定位数据时才设为真。来自 yaml 文件

1.2.2.10.1 1、上 mtx_buffer_imu_prop 的互斥锁，mtx_buffer_imu_prop.lock();

1.2.2.10.2 2、判断 imu 需初始化否（不使用 IMU 和迭代次数够时则置为 false）（将 imu 消息推入 prop_imu_buffer） if (imu_prop_enable && !p_imu->imu_need_init) { prop_imu_buffer.push_back(*msg); }

1.2.2.10.3 3、获得最新的 IMU 数据，标签置真：newest_imu = *msg; new_imu = true;

1.2.2.10.4 4、解锁 mtx_buffer_imu_prop.unlock();

1.2.2.11 10、通知所有线程 sig_buffer.notify_all();

1.2.3 3、image 消息订阅及回调函数

sub_img = nh.subscribe(img_topic, 200000, &LIVMapper::img_cbk, this);回调函数对应的函数为：
void LIVMapper::img_cbk(const sensor_msgs::ImageConstPtr &msg_in)声明于 LIVMapper.h，定义于 LIVMapper.cpp 修正图像实际的触发时间，并将其转为 cv 格式分布存入对应的缓存器 img_buffer，img_time_buffer。

1.2.3.1 1、若不使用相机直接返回 if (!img_en) return;

1.2.3.2 2、声明一个新的图像消息指针 sensor_msgs::Image::Ptr msg(new sensor_msgs::Image(*msg_in));

1.2.3.3 3、将补偿 img 时间戳 double msg_header_time = msg->header.stamp.toSec() + img_time_offset; 为什么明明时间同步了却还是加 0.1s？因为这里的相机时间戳读取的是时间共享文件里的激光雷达帧头时间戳，而激光雷达需要 0.1s 才能完成采集，也就是说这个时间戳是 0.1s 之前的，需要补偿 0.1s 才是实际的图像采集触发时间。论文解析中也有相应的解释。

1.2.3.4 4、判断时间回滚

```

        if(msg_header_time < last_timestamp_img) {ROS_ERROR("image loop back. \n");return; }
1.2.3.5 5、加互斥锁 mtx_buffer.lock();
1.2.3.6 6、记录当前相机时间戳 double img_time_correct = msg_header_time;
1.2.3.7 7、如果与上帧时间过近则 直接返回
        if (img_time_correct - last_timestamp_img < 0.02) {ROS_WARN("Image need Jumps: %.6f",
            img_time_correct);mtx_buffer.unlock();sig_buffer.notify_all();return; }
1.2.3.8 8、从消息中获取 cv 格式的图片 并 推入缓存器
        cv::Mat img_cur =
            getImageFromMsg(msg);img_buffer.push_back(img_cur);img_time_buffer.push_back(img_time_
            correct);
1.2.3.9 9、更新上个图像时间戳 并解锁
        last_timestamp_img = img_time_correct;mtx_buffer.unlock();sig_buffer.notify_all();
1.2.4 4、发布者设置，有※号表示在系统中会执行的，其余的表示未被调用。
pubLaserCloudFullRes = nh.advertise<sensor_msgs::PointCloud2>("/cloud_registered", 100); ※ //
VIO 更新后，发布世界系的当前点云（LVIO 模式为 RGB 点云）pubNormal =
nh.advertise<visualization_msgs::MarkerArray>("visualization_marker", 100); //没用到
pubSubVisualMap = nh.advertise<sensor_msgs::PointCloud2>("/cloud_visual_sub_map_before", 100);
//没用到，由函数 publish_visual_sub_map 调用。pubLaserCloudEffect =
nh.advertise<sensor_msgs::PointCloud2>("/cloud_effected", 100); // 没用到，由函数
publish_effect_world 调用，标志位 pub_effect_point_en 控制。pubLaserCloudMap =
nh.advertise<sensor_msgs::PointCloud2>("/Laser_map", 100);// 没用到 pubOdomAftMapped =
nh.advertise<nav_msgs::Odometry>("/aft_mapped_to_init", 10); ※ //发布 LIO 更新后的位姿，由函
数 publish_odometry 调用 pubPath = nh.advertise<nav_msgs::Path>("/path", 10);※ //发布 LIO 更新
后的路径。plane_pub = nh.advertise<visualization_msgs::Marker>("/planner_normal", 1); // 没用到，
由函数 pubSinglePlane 调用，标志位 pub_plane_en 控制。voxel_pub =
nh.advertise<visualization_msgs::MarkerArray>("/voxels", 1); // 没用到 pubLaserCloudDyn =
nh.advertise<sensor_msgs::PointCloud2>("/dyn_obj", 100);// 没用到 pubLaserCloudDynRmed =
nh.advertise<sensor_msgs::PointCloud2>("/dyn_obj_removed", 100); // 没用到
pubLaserCloudDynDbg = nh.advertise<sensor_msgs::PointCloud2>("/dyn_obj_dbg_hist", 100);// 没用
到 mavros_pose_publisher = nh.advertise<geometry_msgs::PoseStamped>("/mavros/vision_pose/pose",
10); ※ //发布 LIO 更新后的位姿，用于无人机的。pubImage = it.advertise("/rgb_img", 1); ※ //发
布 VIO 更新后的图片 pubImuPropOdom =
nh.advertise<nav_msgs::Odometry>("/LIVO2/imu_propagate", 10000); //默认不会用，发布 Imu 频
率的里程计，仅是简单的运动学方程推导出来的位姿。由标志位 imu_prop_enable 控制。
imu_prop_timer = nh.createTimer(ros::Duration(0.004), &LIVMapper::imu_prop_callback, this); ※ //
一个周期为 0.004 秒（即 4 毫秒，250Hz）的定时器。每当定时器触发时，会自动调用
LIVMapper::imu_prop_callback 成员函数。voxelmap_manager->voxel_map_pub =
nh.advertise<visualization_msgs::MarkerArray>("/planes", 10000); //没用到

```

1.3 3、`mapper.run()`;

对应函数为：void LIVMapper::run()声明于 LIVMapper.h，定义于 LIVMapper.cpp.此函数负责整个系统的运行。

1.3.1 1、设置循环体的运行频率

`ros::Rate rate(5000)`;通常会在循环体内调用 `rate.sleep()`，让循环以设定的频率执行，每秒运行 5000 次（即每次循环间隔 0.2 毫秒），而不会因为处理速度过快而占满 CPU。

1.3.2 2、按设置频率进行循环

while (ros::ok()) 主要做了五件事：1、检查和执行回调队列中所有回调函数。`ros::spinOnce()`;2、将数据缓存器中同一时间区间的 lidar、Imu 以及 image 都取出，用于后续状态估计更新。`sync_packages(LidarMeasures)` 3、记录第一帧 lidar 点云的帧头时间。`handleFirstFrame()`;4、进行 ESIKF 的预测（前向传播）和点云运动去畸变（后向传播）。`processImu()`;5、顺序执行 LIO 和 VIO 状态更新。`stateEstimationAndMapping()`;

1.3.2.1 1、`ros::spinOnce()`;

回调函数队列触发，检查是否有新的消息到达，并触发订阅器相应的回调函数。运行至此

处，激光雷达 IMU，相机的数据均保存至，`lid_raw_data_buffer`，`imu_buffer`，`img_buffer` 需要注意的是，`spinOnce()`在执行的那一刻，并不是只处理队列中的一个回调函数就返回了，而是会处理当时队列中存在的所有回调函数。

1.3.2.2 2、`if(!sync_packages(LidarMeasures))`。

把各缓存区 对应时间内的激光点云/IMU 队列/图像，都以 ESIKF 更新顺序存到测量组中 `meas.measures` 和 `meas.pcl_proc_cur`。对应函数为：`bool`

`LIVMapper::sync_packages(LidarMeasureGroup &meas)`声明于 `LIVMapper.h`，定义于 `LIVMapper.cpp`。其中：`LidarMeasureGroup LidarMeasures` 声明于"`LIVMapper.h`"。若未同步则根据设定的频率（例如 5000 Hz）暂停一段时间，以避免循环过快运行。`rate.sleep()`;
`continue`; 进行下一次迭代。

1.3.2.2.1 1、判断使用的传感器缓存区是否有数据，否则直接返回 `false`

```
if (lid_raw_data_buffer.empty() && lidar_en) return false;if (img_buffer.empty() && img_en)
return false;if (imu_buffer.empty() && imu_en) return false;
```

1.3.2.2.2 2、使用 `switch` 来按照此时的 SLAM 模式进行同步 `switch (slam_mode_)`

1.3.2.2.2.1 1、`case ONLY_LIO`: 激光惯性里程计模式---可以略过

把缓存器中的激光雷达和 imu 数据都推入到激光雷达测量组 `meas` 中

1.3.2.2.2.1.1 1、若上一次 lio 更新时间戳为负（初始化时为负），将雷达缓存器的第一帧雷达帧头时间给它代表上一次更新结束时间戳 `if`
`(meas.last_lio_update_time < 0.0) meas.last_lio_update_time =`
`lid_header_time_buffer.front();`

1.3.2.2.2.1.2 2、判断一帧点云是否被推入到了激光雷达测量组 `meas` 中 `if`
`(!lidar_pushed)`

1.3.2.2.2.1.2.1 1、将激光雷达点云指针缓存器的第一帧点云给 给雷达测量组 `meas.lidar`。
`meas.lidar = lid_raw_data_buffer.front();`

1.3.2.2.2.1.2.2 2、判断该帧点云的数量是否足够 `if (meas.lidar->points.size() <= 1) return`
`false;`

有些判断冗余。

1.3.2.2.2.1.2.3 3、将该帧激光雷达的帧头时间戳给测量组 `meas.lidar_frame_beg_time`
`meas.lidar_frame_beg_time = lid_header_time_buffer.front();`

1.3.2.2.2.1.2.4 4、将该帧激光雷达的帧尾时间戳给测量组 `meas.lidar_frame_end_time`
`meas.lidar_frame_end_time = meas.lidar_frame_beg_time + meas.lidar->points.back().curvature / double(1000);`

1.3.2.2.2.1.2.5 5、将组雷达点云给组当前处理点云 `meas.pcl_proc_cur = meas.lidar;`

1.3.2.2.2.1.2.6 设置 已推入为真 `lidar_pushed = true;`

1.3.2.2.2.1.3 3、判断是否 使用 IMU 并且最近的 imu 时间戳小于激光雷达帧尾时间

`if (imu_en && last_timestamp_imu < meas.lidar_frame_end_time)`

`last_timestamp_imu` 在 `imu` 回调函数中被赋值 最近的 `imu` 时间应该大于最后一个点时间戳。`return false;` 等待直至 `imu` 的时间戳大于最后一个点时间戳 能够涵盖住整个激光雷达扫描。

1.3.2.2.2.1.4 4、测量组对象 `struct MeasureGroup m`

`double vio_time; double lio_time; deque<sensor_msgs::Imu::ConstPtr>
imu; cv::Mat img; MeasureGroup() {vio_time = 0.0; lio_time = 0.0; };`

1.3.2.2.2.1.5 5、清空测量组的 imu 队列，获得测量组的 lio 开始时间戳为最后一个激光点时间，上锁。`m.imu.clear(); m.lio_time = meas.lidar_frame_end_time;`

`mtx_buffer.lock();`

1.3.2.2.2.1.6 6、只要 imu 缓存器不空就循环 `while (!imu_buffer.empty())`

获得涵盖该帧激光雷达测量组的 `imu` 队列。同时不停弹出 `imu` 缓存器 `imu` 数据

1.3.2.2.2.1.6.1 1、判断缓存器头部 `imu` 消息时间戳是否大于最后一个点（说明不对应该帧点云） 是则退出循环 继续后面的缓存器数据弹出 `if`

`(imu_buffer.front()->header.stamp.toSec() > meas.lidar_frame_end_time)`
`break;`

1.3.2.2.2.1.6.2 2、将 `Imu` 数据逐个推入到测量组的 `imu` 队列中，直至出现时间戳大于最后一个点的（上面那个判断）。`Imu` 时间范围涵盖了激光雷达了。

1.3.2.2.2.1.7 7、将激光雷达数据及时间缓存器进行弹出，并解锁

`lid_raw_data_buffer.pop_front(); lid_header_time_buffer.pop_front(); mtx_buffer.unlock(); sig_buffer.notify_all();`

1.3.2.2.2.1.8 8、将 EKF 状态标志 设为 LIO 模型，将测量组推入到激光雷达测量组的 `measures` 中。

`meas.lio_vio_flg = LIO; // process lidar topic, so timestamp should be lidar scan
end.meas.measures.push_back(m);`

1.3.2.2.2.1.9 9、改变推入标志位，完成激光雷达帧与 `Imu` 队列的同步。

`lidar_pushed = false;`

1.3.2.2.2.1.10 10、退出 case

```
break;
```

1.3.2.2.2.2 2、case LIVO: 激光雷达，惯性，相机里程计模式

1.3.2.2.2.2.1 1、设置 EKF 状态标志位 第一帧为等待标志:

EKF_STATE last_lio_vio_flg = meas.lio_vio_flg; 其中: lio_vio_flg 在声明时被构造函数初始化为 WAIT。不是每一个 case 都包含 break。如果 case 语句不包含 break, 控制流将会继续后续的 case, 直到遇到 break 为止。

1.3.2.2.2.2.2 2、根据 EKF 状态标志位选择下一步应顺序执行模块

switch (last_lio_vio_flg)每次循环只执行 LIO 或 VIO。

1.3.2.2.2.2.2.1 1、case WAIT

当第一次运行时, lio_vio_flg 为 WAIT。但是这里什么也没有, 因此会继续执行 case VIO。需要注意的是, case VIO 并不是这次要执行 VIO 更新, 而是指刚完成 VIO 更新, 在这里准备 LIO 更新的数据。case LIO 同理。

1.3.2.2.2.2.2.2 2、case VIO

获得两种数据: 1、最近 LIO 更新时间到图像捕获时间之间的 imu 数据 m.imu, 存入 meas.measures 2、lidar 帧头时间到相机捕获时间之间的点作为当前帧处理点 meas.pcl_proc_cur 存入: meas.pcl_proc_cur 相机捕获时间之后的点作为下一帧处理点云 meas.pcl_proc_next 获得后就直接返回 true。case LIO 将在下个循环中进行。

1.3.2.2.2.2.2.2.1 1、设置 img 获取时间=img 缓存器头部 Img 触发时间+曝光时间

```
double img_capture_time = img_time_buffer.front() + exposure_time_init;
```

1.3.2.2.2.2.2.2.2 2、最近的 LIO 更新时间=lidar 缓存区头部 lidar 的帧头时间。last_lio_update_time 可以理解为上一次 LIO 更新结束时的时间。

```
if (meas.last_lio_update_time < 0.0) meas.last_lio_update_time =  
lid_header_time_buffer.front(); (第一帧 (第一次运行时)  
last_lio_update_time 是个负数) 如果是第一次运行, 那就是用第一帧点  
云的帧头时间更新 last_lio_update_time
```

1.3.2.2.2.2.2.2.3 3、最近的 lidar 时间戳 lid_newest_time 为激光缓存器最后一个点云帧头时间+最后帧最后点的偏移时间, 单位为秒。

```
double lid_newest_time = lid_header_time_buffer.back() +  
lid_raw_data_buffer.back()->points.back().curvature / double(1000);
```

1.3.2.2.2.2.2.2.4 4、最近的 IMU 时间戳为 imu 缓存器最后一个 imu 的时间戳。

```
double imu_newest_time = imu_buffer.back()->header.stamp.toSec();
```

- 1.3.2.2.2.2.2.5 5、判断 如果图像的捕获时间 早于 最近的 LIO 更新时间（上一次顺序更新的图像获取时间。LIO+VIO 视为一次顺序更新。）： 则 弹出该帧图像及时间，并返回 false.

```
if (img_capture_time < meas.last_lio_update_time +  
0.00001){img_buffer.pop_front();img_time_buffer.pop_front();ROS_ERROR("[ Data Cut ] Throw one image frame! \n");return false;}
```

- 1.3.2.2.2.2.2.6 6、判断 当前图像捕获时间晚于出最新激光雷达时间 或者 晚于最新 IMU 时间。返回 false 。即当前（缓存器头部）图像捕获时间必须早于最新的激光雷达和 IMU，否则就是丢失了前面的图像（缓存区里没有跟当前点云帧对应的图像）。

```
if (img_capture_time > lid_newest_time || img_capture_time >  
imu_newest_time)
```

- 1.3.2.2.2.2.2.7 7、构建一个测量组 struct MeasureGroup m;

- 1.3.2.2.2.2.2.8 8、清空 测量组的 Imu 队列，测量组的 LIO 时间为图像捕获时间（获得图像后先进行 LIO 更新），上锁

```
m.imu.clear();m.lio_time = img_capture_time;mtx_buffer.lock();
```

- 1.3.2.2.2.2.2.9 9、只要 imu 缓存器不空就循环 while (!imu_buffer.empty())

将最近 LIO 更新时间和图像捕获时间之间的 imu 数据推入到 测量组 m.imu 中

- 1.3.2.2.2.2.2.10 1、判断 imu 的时间开始晚于相机捕获时间了（足够涵盖图像数据了），就退出循环

```
if (imu_buffer.front()->header.stamp.toSec() > m.lio_time) break;
```

- 1.3.2.2.2.2.2.11 2、判断 Imu 时间 晚于 上次 LIO 更新时间，就推入 imu 到测量组的 imu 队列中

```
if (imu_buffer.front()->header.stamp.toSec() > meas.last_lio_update_time)  
m.imu.push_back(imu_buffer.front());
```

- 1.3.2.2.2.2.2.12 3、imu 缓存器弹出 imu 数据

```
imu_buffer.pop_front();
```

- 1.3.2.2.2.2.2.13 10、解锁 并 通知

```
mtx_buffer.unlock();sig_buffer.notify_all();
```

- 1.3.2.2.2.2.2.14 11、将下一个要处理的点云 给 当前处理点云 并 清空 下个处理的点云指针

```
*(meas.pcl_proc_cur) = *(meas.pcl_proc_next); //这里不明白没有关系，  
结合第 14 项能够读懂。（上一帧处理时有些点被认为是下一帧的，现在就把上一帧认为是下一帧的点拿过来，作为当前帧处理。）  
PointCloudXYZI().swap(*meas.pcl_proc_next);
```

- 1.3.2.2.2.2.2.15 12、计算激光雷达缓存器中所有点云的总点数：

```
int lid_frame_num = lid_raw_data_buffer.size();int max_size =
```

```
meas.pcl_proc_cur->size() + 24000 * lid_frame_num;
```

1.3.2.2.2.2.2.16 13、将当前帧点云指针和下一帧点云指针的容量都扩大到所有点云总点数。

```
meas.pcl_proc_cur->reserve(max_size);meas.pcl_proc_next->reserve(max_size);
```

1.3.2.2.2.2.2.17 14、只要 激光雷达缓存器不空就循环 while (!lid_raw_data_buffer.empty())

在相机捕获时间之前的点作为激光雷达测量组的当前处理点云
meas.pcl_proc_cur，相机捕获时间之后的点作为激光雷达测量组的下一个处理点云 meas.pcl_proc_next

1.3.2.2.2.2.2.18 1、判断 如果出现激光雷达的帧头时间 晚于 图像捕获时间就退出循环：

```
if (lid_header_time_buffer.front() > img_capture_time) break;
```

1.3.2.2.2.2.2.19 2、智能变量，获得激光雷达缓存器中头部点云，pcl。以及这个点云的帧头时间戳

```
auto pcl(lid_raw_data_buffer.front()->points);double  
frame_header_time(lid_header_time_buffer.front());
```

1.3.2.2.2.2.2.20 3、计算最大时间差上限 帧头时间 到 相机捕获时间的时间差 单位 ms

```
float max_offs_time_ms = (m.lio_time - frame_header_time) * 1000.0f;
```

1.3.2.2.2.2.2.21 4、循环这一帧点云 for (int i = 0; i < pcl.size(); i++)

1.3.2.2.2.2.2.22 1、智能变量 一个点 auto pt = pcl[i];

1.3.2.2.2.2.2.23 2、判断该点时间偏差是不是在最大时间差上限内：if (pcl[i].curvature < max_offs_time_ms)

更新该点时间偏差为 加上 帧头时间与最近 LIO 更新时间的时间差。
(第一帧时这个偏差为 0) pt.curvature += (frame_header_time -
meas.last_lio_update_time) * 1000.0f;将该点推入激光雷达测量组的当前
处理点云中**meas.pcl_proc_cur->points.push_back(pt)**;

1.3.2.2.2.2.2.24 3、否则 将该点时间偏差 加上 帧头时间与图像捕获时间的差 并存入激光雷达测量组 meas 的下个处理点云中。

```
pt.curvature += (frame_header_time - m.lio_time) *  
1000.0f;meas.pcl_proc_next->points.push_back(pt);
```

1.3.2.2.2.2.2.25 5、弹出激光雷达缓存器和时间缓存器头部

```
lid_raw_data_buffer.pop_front();lid_header_time_buffer.pop_front();
```

1.3.2.2.2.2.2.26 15、将测量组推入激光雷达测量组的 measures 并将 EKF 状态标志 更新为 LIO

```
meas.measures.push_back(m);meas.lio_vio_flg = LIO;
```

1.3.2.2.2.2.2.27 16、返回真给 LIVMapper::sync_packages

```
return true;
```

1.3.2.2.2.2.3 3、case LIO

获得一种数据：img 缓存器头部的图像数据 存入 **meas.measures**

1.3.2.2.2.2.3.1 1、img 获取时间 = img 时间缓存器头部时间+曝光时间

```
double img_capture_time = img_time_buffer.front() + exposure_time_init;  
如果是第二次运行，此时应该还是第一帧图像。
```

1.3.2.2.2.2.3.2 2、将 EKF 状态标志位设为 VIO

```
meas.lio_vio_flg = VIO;
```

1.3.2.2.2.2.3.3 3、清空 激光雷达测量组的测量（涉及 IMU 的那部分）

```
meas.measures.clear(); 在 VIO 更新中并不需要 IMU 数据
```

1.3.2.2.2.2.3.4 4、获得 IMU 时间

```
double imu_time = imu_buffer.front()->header.stamp.toSec();
```

1.3.2.2.2.2.3.5 5、声明一个 测量组 struct MeasureGroup m;

1.3.2.2.2.2.3.6 6、设置测量组的 VIO 时间为 图像的捕获时间

```
m.vio_time = img_capture_time;
```

1.3.2.2.2.2.3.7 7、设置测量组的 LIO 时间为 最近的 LIO 更新时刻

```
m.lio_time = meas.last_lio_update_time;
```

1.3.2.2.2.2.3.8 8、将 img 缓存器的头部图像给测量组并上锁

```
m.img = img_buffer.front();mtx_buffer.lock();
```

1.3.2.2.2.2.3.9 *8.5 将 IMU 数据存入测量组的部分注释了

没有 IMU 数据可以用了

1.3.2.2.2.2.3.10 9、弹出图像缓存器和图像时间缓存器

```
img_buffer.pop_front();img_time_buffer.pop_front();
```

1.3.2.2.2.2.3.11 10、解锁并通知

```
mtx_buffer.unlock();sig_buffer.notify_all();
```

1.3.2.2.2.2.3.12 11、将测量组推入激光雷达测量组的测量量中：meas.measures.push_back(m);

1.3.2.2.2.2.3.13 12、将雷达推入设置为 false,表示可以进行为下一次 ESIKF 顺序更新推入新的雷达帧了。 **lidar_pushed = false;----**在 LVIO 中 这个标志位似乎没有用。

1.3.2.2.2.2.3.14 13、返回真给 LIVMapper::sync_packages

1.3.2.2.3 3、case ONLY_LO:{xxx}----可以略过

1.3.2.2.3.1 1、是否雷达推入为假 if (!lidar_pushed)

1.3.2.2.3.1.1 1、判断 雷达缓存区为空 则返回 false

```
if (lid_raw_data_buffer.empty()) return false;
```

1.3.2.2.3.1.2 2、将缓存器中的头部点云以及始末时间存入激光雷达测量组中

```
meas.lidar = lid_raw_data_buffer.front(); // push the first lidar  
topicmeas.lidar_frame_beg_time = lid_header_time_buffer.front(); // generate  
lidar_beg_time  
meas.lidar_frame_end_time = meas.lidar_frame_beg_time +  
meas.lidar->points.back().curvature / double(1000); // calc lidar scan end  
time  
lidar_pushed = true;
```

1.3.2.2.3.2 2、声明一个测量组 struct MeasureGroup m;

1.3.2.2.3.3 3、将 LIO 开始时间设为帧末时间戳并上锁

```
m.lio_time = meas.lidar_frame_end_time;mtx_buffer.lock();
```

1.3.2.2.3.4 4、将激光雷达缓存器的头部点云和时间弹出并解锁

```
lid_raw_data_buffer.pop_front();lid_header_time_buffer.pop_front();mtx_buffer.unlo  
ck();sig_buffer.notify_all();
```

1.3.2.2.3.5 5、将激光雷达推入标志位 设为 false

1.3.2.2.3.6 6、将 EKF 状态标志位设为 LO 并推入测量组到激光雷达测量组的测量

```
meas.measures.push_back(m);
```

1.3.2.2.3.7 7、返回真到 LIVMapper::sync_packages 并退出

```
return true;break;
```

1.3.2.3 3、handleFirstFrame();

主要将第一帧激光雷达帧头时间存下来，并赋值给 imu 处理器 p_imu->first_lidar_time 对应函数：void LIVMapper::handleFirstFrame()声明于 LIVMapper.h，定义于 LIVMapper.cpp。

1.3.2.3.1 判断 是否第一帧 if (!is_first_frame)。获得第一帧激光雷达的帧头时间，将这个时间给 imu 处理器中的第一帧激光雷达时间 p_imu->first_lidar_time = _first_lidar_time; 修改第一帧标志位。

```
_first_lidar_time = LidarMeasures.last_lio_update_time;p_imu->first_lidar_time =  
_first_lidar_time; // Only for IMU data logis_first_frame = true;
```

1.3.2.4 4、processImu();

进行 IMU 前向传播（获得先验状态）和反向传播（获得去畸变点云）对应函数 void LIVMapper::processImu()声明于 LIVMapper.h，定义于 LIVMapper.cpp。

1.3.2.4.1 1、p_imu->Process2(LidarMeasures, _state, feats_undistort);

调用 Imu 处理器的 Process2 函数处理激光雷达测量组并获得先验状态和去畸变点云对应函数：void ImuProcess::Process2(LidarMeasureGroup &lidar_meas, StatesGroup &stat, PointCloudXYZI::Ptr cur_pcl_un)声明于 IMU_Processing.h，定义于 IMU_Processing.cpp 其中 state, feats_undistort 声明于 LIVMapper.hStatesGroup _state; PointCloudXYZI::Ptr feats_undistort;

1.3.2.4.1.1 1、三个时间记录量 double t1, t2, t3;

```
t1 = omp_get_wtime();记录开始时间到 t1
```

1.3.2.4.1.2 2、检查点云空不空

ROS_ASSERT(lidar_meas.lidar != nullptr);

1.3.2.4.1.3 3、判断是否使用了 IMU。如果不使用 IMU 则进行无 imu 的前向传播后返回。-----可略过

$$\mathbf{x} = \begin{bmatrix} {}^G\mathbf{R}_L \\ {}^G\mathbf{p}_L \\ {}^G\mathbf{v}_L \\ \boldsymbol{\omega}_L \end{bmatrix}, \mathbf{w} = \begin{bmatrix} \mathbf{n}_v \\ \mathbf{n}_\omega \end{bmatrix}, \mathbf{f}(\mathbf{x}, \mathbf{w}) = \begin{bmatrix} \boldsymbol{\omega}_L \\ {}^G\mathbf{v}_L \\ \mathbf{n}_v \\ \mathbf{n}_\omega \end{bmatrix}$$

if (!imu_en) {Forward_without_imu(lidar_meas, stat, *cur_pcl_un_); return;} 需要注意, 在 Fast-LIVO2 源码中, 如果不使用 IMU 的话, SLAM_MODEL 是 LO 模式, 此时状态向量就不能按照 Fast-LIVO2 论文中的理解。笔者推荐读者参考论文《Robust Real-time LiDAR-inertial Initialization》中的模型。与此处的代码实现是完全一致的。对应函数: void ImuProcess::Forward_without_imu(LidarMeasureGroup &meas, StatesGroup &state_inout, PointCloudXYZI &pcl_out)声明于 IMU_Processing.h, 定义于 IMU_Processing.cpp。

1.3.2.4.1.3.1 1、将激光雷达测量组中的 lidar 给点云变量: pcl_out = *(meas.lidar);

1.3.2.4.1.3.2 2、将点云按偏移时间从小到大进行排序 sort(pcl_out.points.begin(), pcl_out.points.end(), time_list);

1.3.2.4.1.3.3 3、获得帧头时间 const double &pcl_beg_time = meas.lidar_frame_beg_time;

1.3.2.4.1.3.4 4、获得帧尾时间: const double &pcl_end_time = pcl_beg_time + pcl_out.points.back().curvature / double(1000); 单位是秒

1.3.2.4.1.3.5 5、该帧点云最后点的偏移时间 const double &pcl_end_offset_time = pcl_out.points.back().curvature / double(1000);

1.3.2.4.1.3.6 6、设置 上次 LIO 更新时间为当前帧的帧尾时间 (给下一帧用的, 已经把当前帧视为上一帧了)

1.3.2.4.1.3.7 7、初始化关于状态系数矩阵和噪声协方差阵 MD(DIM_STATE, DIM_STATE) F_x, cov_w; 19x19

1.3.2.4.1.3.8 8、初始化传播时间间隔 double dt = 0;

1.3.2.4.1.3.9 9、判断是否为第一帧 if (b_first_frame) 设置间隔时间

dt = 0.1; 将时间间隔设为 0.1 b_first_frame = false;

1.3.2.4.1.3.10 10、否则 非第一帧 间隔时间为 点云帧头时间 - 上一帧点云帧头时间

else { dt = pcl_beg_time - time_last_scan; }

1.3.2.4.1.3.11 11、将当前帧头时间更新到上次扫描时间。time_last_scan =

pcl_beg_time;

1.3.2.4.1.3.12 12、协方差传播--获得先验状态和先验协方差

$$\begin{aligned} \mathbf{F}_{\bar{\mathbf{x}}} &= \frac{\partial((\bar{\mathbf{x}}_k \boxplus \delta \mathbf{x}_k) \boxplus (\Delta t \mathbf{f}(\bar{\mathbf{x}}_k \boxplus \delta \mathbf{x}_k, \mathbf{0})) \boxminus (\bar{\mathbf{x}}_k \boxplus \Delta t \mathbf{f}(\bar{\mathbf{x}}_k, \mathbf{0})))}{\partial \delta \mathbf{x}_k} \\ &= \begin{bmatrix} \text{Exp}(-\hat{\boldsymbol{\omega}}_{L_k} \Delta t) & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} \Delta t \\ \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} & \mathbf{I}_{3 \times 3} \Delta t & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} \end{bmatrix} \\ \mathbf{F}_{\mathbf{w}} &= \frac{\partial(\bar{\mathbf{x}}_k \boxplus (\Delta t \mathbf{f}(\bar{\mathbf{x}}_k, \mathbf{w}_k)) \boxminus (\bar{\mathbf{x}}_k \boxplus \Delta t \mathbf{f}(\bar{\mathbf{x}}_k, \mathbf{0})))}{\partial \mathbf{w}_k} \\ &= \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{I}_{3 \times 3} \Delta t & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} \Delta t \end{bmatrix} \end{aligned} \quad (5)$$

注意在 LO 模式下，采用的是恒速模型，这一点与 LiDAR_IMU_init 一样，因此会使用陀螺仪偏差量 bias_g 来代替角速度值，但是在 LIO 或 LIVO 中，bias_g 就只代表偏差量。对应的 F_x 会不一样。

1.3.2.4.1.3.12.1 1、角速度与时间间隔乘积并指数化 M3D Exp_f =

Exp(state_inout.bias_g, dt);

1.3.2.4.1.3.12.2 2、状态转移矩阵初始化(单位阵) F_x.setIdentity();

1.3.2.4.1.3.12.3 3、噪声协方差 (0 矩阵) cov_w.setZero();

1.3.2.4.1.3.12.4 4、误差状态转移函数中旋转更新函数对旋转误差状态的导数

F_x.block<3, 3>(0, 0) = Exp(state_inout.bias_g, -dt);

该矩阵还可以从[另一个角度](#)得到。

1.3.2.4.1.3.12.5 5、误差状态转移函数中旋转更新函数对陀螺仪偏差状态的导数：

$$F_{x,\text{block}<3, 3>(0, 10)} = \text{Eye3d} * dt;$$

1.3.2.4.1.3.12.6 6、误差状态转移函数中平移更新函数对速度误差状态的导数：

$$F_{x,\text{block}<3, 3>(3, 7)} = \text{Eye3d} * dt;$$

1.3.2.4.1.3.12.7 7、系统噪声协方差，只对陀螺仪和加表协方差部分初始化，对应状态向量中的第四个和第三个。

```
cov_w.block<3, 3>(10, 10).diagonal() = cov_gyr * dt * dt; // for omega in
constant model 连续两个 dt 说明已经做的 8 中的第二项。cov_w.block<3,
3>(7, 7).diagonal() = cov_acc * dt * dt; // for velocity in constant model
```

1.3.2.4.1.3.12.8 8、状态协方差 $\text{state_inout.cov} = F_x * \text{state_inout.cov} * F_x.\text{transpose}() + \text{cov_w};$

$$\hat{\mathbf{P}}_{k+1} = \mathbf{F}_{\tilde{\mathbf{x}}} \bar{\mathbf{P}}_k \mathbf{F}_{\tilde{\mathbf{x}}}^T + \mathbf{F}_w \mathbf{Q} \mathbf{F}_w^T$$

1.3.2.4.1.3.12.9 9、旋转更新 $\text{state_inout.rot_end} = \text{state_inout.rot_end} * \text{Exp}_f;$

$$\mathbf{R} = \mathbf{R} \text{Exp}((\boldsymbol{\omega}_m - \mathbf{b}_g) \Delta t)$$

1.3.2.4.1.3.12.10 10、平移更新 $\text{state_inout.pos_end} = \text{state_inout.pos_end} + \text{state_inout.vel_end} * dt;$

$$\mathbf{p} = \mathbf{p} + \mathbf{v} \Delta t$$

1.3.2.4.1.3.13 13、除了 L515 型号雷达，其他雷达还需运动补偿 if (lidar_type != L515)

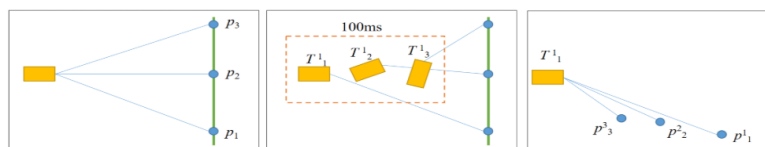


Figure 2. 点云坐标系发生变化

去畸变前所有点都是一个状态，需要基于最后一个点时刻的系统状态，获得其他点时刻的系统状态。然后获得其他点相对于最后一个的状态变换阵，其他点与自己状态变换阵的左乘，就获得以最后一个点为基准的其他点位置。将每一帧的各个点坐标转换到同一坐标系，就是去畸变的过程了。

1.3.2.4.1.3.13.1 1、获得点云的最后一个点 auto it_pcl = pcl_out.points.end() - 1;

1.3.2.4.1.3.13.2 2、声明 点间时间间隔 double dt_j = 0.0;

1.3.2.4.1.3.13.3 3、从点云末尾到点云头部 循环整个点云 for(it_pcl !=
pcl_out.points.begin(); it_pcl--)

1.3.2.4.1.3.13.3.1 1、最后点的偏移时间减去当前点的偏移时间: dt_j= pcl_end_offset_time - it_pcl->curvature/double(1000);

1.3.2.4.1.3.13.3.2 2、相对的旋转矩阵: M3D R_jk(Exp(state_inout.bias_g, - dt_j));

$${}^{L_{k+1}}\check{\mathbf{R}}_{L_j} = \text{Exp}(-\hat{\boldsymbol{\omega}}_{L_{k+1}} \Delta t_j)$$

这里省略了一个单位阵, 最后一个点相对于自己为不变

1.3.2.4.1.3.13.3.3 3、相对的平移: p_jk = - state_inout.rot_end.transpose() * state_inout.vel_end * dt_j;

$${}^{L_{k+1}}\check{\mathbf{p}}_{L_j} = -{}^G\hat{\mathbf{R}}_{L_{k+1}}^T {}^G\hat{\mathbf{v}}_{L_{k+1}} \Delta t_j$$

这里省略了一个 p_end = 0, 因为最后一个点相对于自己不变。

1.3.2.4.1.3.13.3.4 4、校正 V3D P_compensate = R_jk * P_j + p_jk;

$${}^{L_{k+1}}\mathbf{p}_j = {}^{L_{k+1}}\check{\mathbf{T}}_{L_j} {}^{L_j}\mathbf{p}_j$$

it_pcl->x = P_compensate(0);it_pcl->y = P_compensate(1);it_pcl->z =
P_compensate(2);

1.3.2.4.1.4 4、获得激光雷达测量组中最新的 IMU 队列

MeasureGroup meas = lidar_meas.measures.back();

1.3.2.4.1.5 5、判断是否 需要 imu 初始化 该函数仅执行一次。

if(imu_need_init) imu_need_init 来自 yaml 文件 设为真。主要做了: 1、记录 pcl 的帧尾时间 (相机获取时间) 2、IMU 相关量重置, 恢复重力向量单位, 设置初始状态的旋转和陀螺仪偏差 3、记录队列最后一个 imu 为最近 last_imu4、打印 imu 相关量, 打开 imu.txt 文件 5、直接返回函数, 不进行去畸变 (此时为初始化状态, 并不开展实际的系统流程, 系统实际将在 while (ros::ok()) 第三次循环开始)。由于未去畸变, feats_undistort 是空的, 因此会在后面 LIO 更新时进行返回, 进行下一次循环, 执行 VIO 更新, 但由于此时啥也没有, 也会返回并开始第三次循环, 这时候才开始走系统完整流程。

1.3.2.4.1.5.1 1、根据 EKF 状态标志位来确定 pcl 的帧尾时间 (这里应该都是图像获取时间吧)

double pcl_end_time = lidar_meas.lio_vio_flg == LIO ? meas.lio_time :
meas.vio_time;

1.3.2.4.1.5.2 2、判断 测量组中的 imu 队列是否空

if (meas.imu.empty()) { return; };

1.3.2.4.1.5.3 3、IMU_init(meas, stat, init_iter_num);

进行 IMU 初始化 对应函数：void ImuProcess::IMU_init(const MeasureGroup &meas, StatesGroup &state_inout, int &N)声明于 IMU_Processing.h, 定义于 IMU_Processing.cpp 目的：对 IMU 相关量进行重置，恢复重力向量为 m/s²，以及初始化状态旋转阵、陀螺仪偏差及最近 Imu 消息等。

1.3.2.4.1.5.3.1 1、初始化进度打印 百分制

```
ROS_INFO("IMU Initializing: %.1f%%", double(N) / MAX_INI_COUNT * 100);
```

1.3.2.4.1.5.3.2 2、当前角速度和加速度变量：V3D cur_acc, cur_gyr;

1.3.2.4.1.5.3.3 3、判断是否 如果是第一帧 if (b_first_frame)

重置 IMU 以及去畸变点云相关量，获得测量组第一个 Imu 数据

1.3.2.4.1.5.3.3.1 1、Reset()

对平均加速度/角速度，迭代次数，IMU 位姿容器等进行重置

1.3.2.4.1.5.3.3.2 1、平均角速度/加速度重置

```
mean_acc = V3D(0, 0, -1.0); mean_gyr = V3D(0, 0, 0);
```

1.3.2.4.1.5.3.3.3 2、上个角速度量重置 angvel_last = Zero3d;

1.3.2.4.1.5.3.3.4 3、imu 初始化标志位 设为真

```
imu_need_init = true;
```

1.3.2.4.1.5.3.3.5 4、设置初始迭代次数

```
init_iter_num = 1;
```

1.3.2.4.1.5.3.3.6 5、IMU 位姿容器清空

```
IMUpose.clear();
```

1.3.2.4.1.5.3.3.7 6、上个 IMU 消息指针重置

```
last_imu.reset(new sensor_msgs::Imu());实际上在构造 p_imu 就已进行了
```

1.3.2.4.1.5.3.3.8 7、当前去畸变点云指针重置

```
cur_pcl_un_.reset(new PointCloudXYZI());实际上在构造 p_imu 就已进行了
```

1.3.2.4.1.5.3.3.9 2、赋值初始迭代次数为 1（实际在 Reset 中已经实现了） $N = 1$;

1.3.2.4.1.5.3.3.10 3、失能 第一帧标志位 `b_first_frame = false`;

1.3.2.4.1.5.3.3.11 4、获得测量组内第一帧的 imu 数据给 `imu_acc`, `gyr_acc`, `mean_acc` 以及 `mean_gyr`

```
const auto &imu_acc = meas.imu.front()->linear_acceleration;const auto
&gyr_acc = meas.imu.front()->angular_velocity; mean_acc << imu_acc.x,
imu_acc.y, imu_acc.z; mean_gyr << gyr_acc.x, gyr_acc.y, gyr_acc.z;V3D
mean_acc; // 声明于 IMU_Processing.h V3D mean_gyr;
```

1.3.2.4.1.5.3.4 4、遍历测量组的 imu 队列中每个 imu 消息

```
for (const auto &imu : meas.imu)获得 imu 队列的平均加速度和角速度，并
累计次数 N
```

1.3.2.4.1.5.3.4.1 1、获得当前角速度加速度并求平均值--这里使用了均值增量更新的方法

```
const auto &imu_acc = imu->linear_acceleration;const auto &gyr_acc = imu-
>angular_velocity;cur_acc << imu_acc.x, imu_acc.y, imu_acc.z;cur_gyr <<
gyr_acc.x, gyr_acc.y, gyr_acc.z;mean_acc += (cur_acc - mean_acc) /
N;mean_gyr += (cur_gyr - mean_gyr) / N;
```

1.3.2.4.1.5.3.4.2 2、次数更新: $N++$;

1.3.2.4.1.5.3.5 5、对加速度均值计算模长

```
IMU_mean_acc_norm = mean_acc.norm();
```

1.3.2.4.1.5.3.6 6、获得重力向量，加速度均值归一化取反*重力值。恢复为 m/s^2 单位。

```
state_inout.gravity = -mean_acc / mean_acc.norm() * G_m_s2;
```

1.3.2.4.1.5.3.7 7、初始化 imu 系相对世界系原点的旋转为单位阵，陀螺仪偏差为 0。

```
state_inout.rot_end = Eye3d; // Exp(mean_acc.cross(V3D(0, 0, -1 /
scale_gravity)));state_inout.bias_g = Zero3d; // mean_gyr;
```

1.3.2.4.1.5.3.8 8、将测量组的最后一个 Imu 数据视为最近的（上次）Imu 值。

```
last_imu = meas.imu.back();
```

1.3.2.4.1.5.4 4、将 imu 初始化标志置为真

```
imu_need_init = true; 这句似乎不影响系统进程
```

1.3.2.4.1.5.5 5、获得 测量组 Imu 队列尾部数据为最近 IMU 消息。似乎与 3 中重复了。

```
last_imu = meas.imu.back();
```

1.3.2.4.1.5.6 6、判断 迭代次数是否超过限制次数 30

```
if (init_iter_num > MAX_INI_COUNT)MAX_INI_COUNT 来自 yaml 中的
imu_int_frame 打印 imu 初始化后的各个状态
```

1.3.2.4.1.5.6.1 1、imu 初始化标志设为 false

```
imu_need_init = false;
```

1.3.2.4.1.5.6.2 2、打印 IMU 初始化后的重力 加速度协方差，陀螺仪协方差

```
ROS_INFO("IMU Initials: Gravity: %.4f %.4f %.4f %.4f; acc covariance:
""%.8f %.8f %.8f; gry covariance: %.8f %.8f %.8f\n",stat.gravity[0],
stat.gravity[1], stat.gravity[2], mean_acc.norm(), cov_acc[0], cov_acc[1],
cov_acc[2], cov_gyr[0], cov_gyr[1],cov_gyr[2]);
```

1.3.2.4.1.5.6.3 3、打印 IMU 初始化后的 ba 协方差，bg 协方差

```
ROS_INFO("IMU Initials: ba covariance: %.8f %.8f %.8f; bg covariance:
""%.8f %.8f %.8f",cov_bias_acc[0], cov_bias_acc[1], cov_bias_acc[2],
cov_bias_gyr[0], cov_bias_gyr[1], cov_bias_gyr[2]);
```

1.3.2.4.1.5.6.4 4、打开 imu.txt 文件，用于后续数据保存

```
fout_imu.open(DEBUG_FILE_DIR("imu.txt"), ios::out);
```

1.3.2.4.1.5.7 7、直接返回 return;

1.3.2.4.1.6 6、UndistortPcl(lidar_meas, stat, *cur_pcl_un_);

进行点云去畸变 对应函数：void ImuProcess::UndistortPcl(LidarMeasureGroup &lidar_meas, StatesGroup &state_inout, PointCloudXYZI &pcl_out)声明于 IMU_Processing.h. 定义于 IMU_Processing.cpp

1.3.2.4.1.6.1 记录时间 double t0 = omp_get_wtime();

1.3.2.4.1.6.2 1、清空之前存的去畸变点云：pcl_out.clear();

1.3.2.4.1.6.3 2、取出测量组中最新的 imu 队列

```
MeasureGroup &meas = lidar_meas.measures.back(); auto v_imu = meas.imu;
```

1.3.2.4.1.6.4 4、将上次的最后一个 Imu 数据推到队列的最前面

```
v_imu.push_front(last_imu);
```

1.3.2.4.1.6.5 5、获得 IMU 队列的前后时间 imu_beg_time, imu_end_time

```
const double &imu_beg_time = v_imu.front()->header.stamp.toSec();const double
&imu_end_time = v_imu.back()->header.stamp.toSec();
```

1.3.2.4.1.6.6 6、将上次传播的结束时间戳作为本次传播的开始时间戳

```
const double prop_beg_time = last_prop_end_time;last_prop_end_time 声明于
IMU_Processing.h 奇怪的是 并没有定义初值。第一次执行的话会是一个随机数。
```

1.3.2.4.1.6.7 7、本次传播结束时间为图像获取时间

```
const double prop_end_time = lidar_meas.lio_vio_flg == LIO ? meas.lio_time :
meas.vio_time;如果此时 SLAM 是 LIO 模式，则将结束时刻设为 lio_time，否则
为 vio_time，有意思的是 lio_time 和 vio_time 都是图像获取时间。
```

1.3.2.4.1.6.8 8、判断是不是 LIO 模式

if (lidar_meas.lio_vio_flg == LIO) 获得待去畸变点云和设置此次传播的第一个 IMU 的 6 自由度位姿。

1.3.2.4.1.6.8.1 1、获得待传播的点云：pcl_wait_proc

```
pcl_wait_proc.resize(lidar_meas.pcl_proc_cur->points.size()); pcl_wait_proc =  
*(lidar_meas.pcl_proc_cur); 将测量组中的当前帧点云作为待传播（去畸变）  
点云。
```

1.3.2.4.1.6.8.2 2、设激光雷达测量组的当前扫描索引为 0---似乎没什么用

```
lidar_meas.lidar_scan_index_now = 0;
```

1.3.2.4.1.6.8.3 3、设置此次传播的第一个 IMU 位姿 set_pose6d，并推入到 IMU 位姿队列中 IMUpose

```
IMUpose.push_back(set_pose6d(0.0, acc_s_last, angvel_last, state_inout.vel_end,  
state_inout.pos_end, state_inout.rot_end)); 其中 acc_s_last 等量是上次前向传播  
后得到的 imu 最后时刻状态。set_pose6d 对应函数：auto set_pose6d(const  
double t, const Matrix<T, 3, 1> &a, const Matrix<T, 3, 1> &g, const Matrix<T, 3,  
1> &v, const Matrix<T, 3, 1> &p, const Matrix<T, 3, 3> &R) 声明并定义于  
common_lib.h
```

1.3.2.4.1.6.8.3.1 1、设置 6 自由度结构体的对象：Pose6D rot_kp;

1.3.2.4.1.6.8.3.2 2、设置该对象的偏移时间 t 设为 0

1.3.2.4.1.6.8.3.3 3、循环将加速度/角速度以及状态：旋转、平移和速度存入该对象

```
rot_kp.acc[i] = a(i); rot_kp.gyr[i] = g(i); rot_kp.vel[i] = v(i); rot_kp.pos[i] =  
p(i); for (int j = 0; j < 3; j++) rot_kp.rot[i * 3 + j] = R(i, j);
```

1.3.2.4.1.6.8.3.4 4、非复制返回该对象：return move(rot_kp);

1.3.2.4.1.6.9 9、前向传播

每个 imu 的状态及协方差

1.3.2.4.1.6.9.1 1、获得上一次前向传播获得的最后时刻 imu 状态以及最新的系统状态。

```
V3D acc_imu(acc_s_last), angvel_avr(angvel_last), acc_avr,  
vel_imu(state_inout.vel_end), pos_imu(state_inout.pos_end);
```

1.3.2.4.1.6.9.2 2、获得 IMU 的旋转状态：

```
M3D R_imu(state_inout.rot_end);
```

1.3.2.4.1.6.9.3 3、声明状态系数矩阵和过程噪声协方差阵：

$$\hat{\mathbf{P}}_{i+1} = \mathbf{F}_{\tilde{\mathbf{x}}} \hat{\mathbf{P}}_i \mathbf{F}_{\tilde{\mathbf{x}}}^T + \mathbf{F}_{\mathbf{w}} \mathbf{Q} \mathbf{F}_{\mathbf{w}}^T$$

MD(DIM_STATE, DIM_STATE) F_x, cov_w; 实际上，代码实现部分会将

$F_w * Q * F_w^T$ 作为整体用 cov_w 统一代表。

1.3.2.4.1.6.9.4 4、声明传播时间间隔和整个传播的时间间隔

`double dt, dt_all = 0.0;` imu 的采用时间。

1.3.2.4.1.6.9.5 5、声明 偏移时间

`double offs_t;` imu 测量相对传播开始（上次传播结束—图像捕获时间）的偏移时间。

1.3.2.4.1.6.9.6 6、逆曝光时间。并未参与到 IMU 前向传播中

`double tau;`

1.3.2.4.1.6.9.7 7、（第一次运行）判断 IMU 时间初始化标志位是否为假。将 tau 设置为 1.0，初始化时间标志为真。

```
if (!imu_time_init) { // imu_time = v_imu.front()->header.stamp.toSec() -
first_lidar_time; // tau = 1.0 / (0.25 * sin(2 * CV_PI * 0.5 * imu_time) + 0.75);
tau = 1.0; imu_time_init = true; }
```

1.3.2.4.1.6.9.8 8、非第一帧，否则，将状态的逆曝光时间给 tau：

`tau = state_inout.inv_expo_time;`

1.3.2.4.1.6.9.9 9、判断系统状态标志位（这里开始前向传播）

`switch (lidar_meas.lio_vio_flg) case VIO:lio_vio_flg 是啥都一样，因为 case LIO 是空的。`

1.3.2.4.1.6.9.9.1 1、时间间隔设置为 0： `dt = 0;`

1.3.2.4.1.6.9.9.2 2、遍历 imu 队列

`for (int i = 0; i < v_imu.size() - 1; i++)`获得这个 IMU 队列的下一个 Imu 到开始传播时刻的偏移时间，每个 imu 的先验协方差，先验姿态，先验加速度，先验位置，先验速度，

1.3.2.4.1.6.9.9.3 1、将加速度恢复到 m/s2 并计算当前 imu 与下个 imu 的时间间隔 dt

1.3.2.4.1.6.9.9.4 1、获得当前 imu 和下一个 Imu 数据：

`auto head = v_imu[i]; auto tail = v_imu[i + 1];`

1.3.2.4.1.6.9.9.5 2、如果下一个 IMU 的时间 早于 传播开始时间（上次传播的结束时间）--属于上次传播的 IMU 数据。进行下一次循环：

`if (tail->header.stamp.toSec() < prop_beg_time) continue;`

1.3.2.4.1.6.9.9.6 3、将这个和下个 IMU 数据求平均给 `angvel_avr`，`acc_avr`

```
angvel_avr << 0.5 * (head->angular_velocity.x + tail->angular_velocity.x), 0.5 * (head->angular_velocity.y + tail->angular_velocity.y), 0.5 * (head->angular_velocity.z + tail->angular_velocity.z);
```

```
>angular_velocity.z); acc_avr << 0.5 * (head->linear_acceleration.x +
tail->linear_acceleration.x), 0.5 * (head->linear_acceleration.y + tail-
>linear_acceleration.y), 0.5 * (head->linear_acceleration.z + tail-
>linear_acceleration.z);
```

- 1.3.2.4.1.6.9.9.7 4、将相邻两点的 IMU 到第一帧雷达帧头时间偏移和测量数据均值--->输出到文件 fout_imu 中

```
fout_imu << setw(10) << head->header.stamp.toSec() - first_lidar_time << "
" << angvel_avr.transpose() << " " << acc_avr.transpose() << endl;
```

- 1.3.2.4.1.6.9.9.8 5、将平均角速度减去陀螺仪偏差

- 1.3.2.4.1.6.9.9.9 6、加速度均值乘以归一化重力值减去加速度偏差。

`acc_avr = acc_avr * G_m_s2 / mean_acc.norm() - state_inout.bias_a;` [这行代码后面有来头的](#)。首先 Fast-LIVO 需要静止进行初始化（该行代码不适用于运动初始化的恢复），此时获得了初始化的均值加速度测量量 `mean_acc`，强假设一下：IMU 水平放置，且不考虑噪声，此时 `mean_acc` 的值应当是 `[0,0,1.0]`。若此时 `acc_avr` 为 `[0,0,1.0]`，通过将 `G_m_s2/mean_acc.norm()` 相当于将 `acc_avr` 单位恢复（缩放）到了 `m/s2`，但是考虑到偏差和噪声影响，这种恢复其实并不准确，因此在后续状态更新中，会对偏差进行优化，并消除偏差，尽可能获得准确；恢复量。

- 1.3.2.4.1.6.9.9.10 7、判断 当前 IMU 时间是否 早于 上次传播结束时间（上次传播结束时刻卡在当前 imu 和下一个 Imu 之间）

```
if (head->header.stamp.toSec() < prop_beg_time)将时间间隔设为下个 imu
时刻与上次传播结束时刻的差 dt = tail->header.stamp.toSec() -
last_prop_end_time;将偏移时间设为下个 imu 时刻与本次传播开始时刻
的差 offs_t = tail->header.stamp.toSec() - prop_beg_time;
```

- 1.3.2.4.1.6.9.9.11 8、否则 判断 当前 Imu 不是 imu 队列的倒数第二个

```
else if (i != v_imu.size() - 2)时间间隔就是两个 IMU 之间的间隔，偏移时
间就是下个 Imu 时刻与本次传播开始时刻的差 dt = tail-
>header.stamp.toSec() - head->header.stamp.toSec();offs_t = tail-
>header.stamp.toSec() - prop_beg_time;
```

- 1.3.2.4.1.6.9.9.12 9、否则 当前点是倒数第二个 imu

时间间隔为 图像捕获时刻-当前 Imu 时刻，偏移时间为 图像捕获时刻-传播开始时刻）“将最后一个 IMU 的状态设置为图像捕获时刻的状态”
`dt = prop_end_time - head->header.stamp.toSec();offs_t = prop_end_time - prop_beg_time;`

- 1.3.2.4.1.6.9.9.13 10、将时间总间隔加上本次间隔 `dt_all += dt;`

- 1.3.2.4.1.6.9.9.14 2、协方差传播

$$\hat{\mathbf{P}}_{i+1} = \mathbf{F}_{\tilde{\mathbf{x}}} \hat{\mathbf{P}}_i \mathbf{F}_{\tilde{\mathbf{x}}}^T + \mathbf{F}_{\mathbf{w}} \mathbf{Q} \mathbf{F}_{\mathbf{w}}^T; \hat{\mathbf{P}}_0 = \bar{\mathbf{P}}_{k-1}.$$

1.3.2.4.1.6.9.9.15 1、获得恢复到 m/s² 单位的加速度反对称阵 用于后续计算。

$$[\hat{\mathbf{a}}_i]^\wedge$$

```
M3D acc_avr_skew;acc_avr_skew << SKEW_SYM_MATRX(acc_avr);
```

1.3.2.4.1.6.9.9.16 2、根据角速度获得角度增量的指数形式 (angvel_avr 包含了偏差)

$$\text{Exp}((\boldsymbol{\omega}_m - \mathbf{b}_g) \Delta t)$$

```
M3D Exp_f = Exp(angvel_avr, dt);
```

1.3.2.4.1.6.9.9.17 * 噪声协方差 Q 阵 13*13 (这里漏了 曝光噪声项)

$$\begin{bmatrix} \sigma_{\eta_g}^2 & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \sigma_{\eta_a}^2 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \sigma_{\eta_{bg}}^2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \sigma_{\eta_{ba}}^2 \end{bmatrix}$$

1.3.2.4.1.6.9.9.18 3、对状态转移矩阵 `F_x.setIdentity();` 和系统噪声项 `cov_w.setZero();` 。维度 19*19, 初始化, 用于计算 先验协方差。

$$\mathbf{F}_{\bar{\mathbf{x}}} = \begin{bmatrix} \text{Exp}(-\hat{\boldsymbol{\omega}}_i \Delta t) & \mathbf{0} & \mathbf{0} & -\mathbf{A}(\hat{\boldsymbol{\omega}}_i \Delta t)^T \Delta t & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & \mathbf{I} \Delta t & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ -{}^G \hat{\mathbf{R}}_{t_i} [\hat{\mathbf{a}}_i]^\wedge \Delta t & \mathbf{0} & \mathbf{I} & \mathbf{0} & -{}^G \hat{\mathbf{R}}_{t_i} \Delta t & \mathbf{I} \Delta t \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} \end{bmatrix}, \mathbf{F}_{\mathbf{w}} = \begin{bmatrix} -\mathbf{A}(\hat{\boldsymbol{\omega}}_i \Delta t)^T \Delta t & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -{}^G \hat{\mathbf{R}}_{t_i} \Delta t & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} \Delta t & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} \Delta t \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}$$

`F_x.setIdentity(); cov_w.setZero();` (笔者拿 Fast-lio 的矩阵举例, 因此图中两个矩阵少了个曝光时间项) $\text{cov_w} = \mathbf{F_w} \mathbf{Q} \mathbf{F_w}^T$

1.3.2.4.1.6.9.9.19 1.1、状态向量

$$\mathbf{x} \triangleq [\mathbf{p}_I^T \quad \mathbf{v}_I^T \quad \mathbf{b}_g^T \quad \mathbf{b}_a^T \quad \mathbf{g}^T \quad \tau]^T$$

请读者注意, 在 Fast-LIVO2 实际代码中, 最后一项曝光量被放置在了 平移 $\mathbf{p}$ 和速度 $\mathbf{v}$ 之间。因此关于速度的系数块, 在代码中的矩阵索引是从 7 开始的, 6 是逆曝光项的导数系数。

1.3.2.4.1.6.9.9.20 1.2、噪声向量----非代码内容, 用于解析时参考的

$$\mathbf{w} \triangleq [\mathbf{n}_g^T \quad \mathbf{n}_a^T \quad \mathbf{n}_{b_g}^T \quad \mathbf{n}_{b_a}^T \quad n_\tau]^T$$

1.3.2.4.1.6.9.9.21 1.3、误差状态向量 (以 Fast-lio 举例--需加上逆曝光时间误差项) ----非代码内容, 用于解析时参考的

$$[\delta \boldsymbol{\theta}^T \quad \tilde{\mathbf{p}}_I^T \quad \tilde{\mathbf{v}}_I^T \quad \tilde{\mathbf{b}}_g^T \quad \tilde{\mathbf{b}}_a^T \quad \tilde{\mathbf{g}}^T]^T$$

1.3.2.4.1.6.9.9.22 2、F_x 阵设置 19*19

1.3.2.4.1.6.9.9.23 1、旋转误差项关于状态向量的方程

$$\approx \exp(\hat{w}_i \Delta t)^\top \cdot \delta \hat{r}_i - \Delta t \cdot \delta \hat{b}_{wi} - \Delta t \cdot n_{wi}$$

1.3.2.4.1.6.9.9.24 1、设置对旋转误差状态导数的系数： 矩阵的左上角 3*3 块

$$F_x.\text{block}<3, 3>(0, 0) = \text{Exp}(\text{angvel_avr}, -dt);$$

1.3.2.4.1.6.9.9.25 2、（可选 if(ba_bg_est_en)），设置对角速度偏差的导数系数：

$$F_x.\text{block}<3, 3>(0, 10) = -\text{Eye3d} * dt;$$

1.3.2.4.1.6.9.9.26 2、平移误差项关于状态向量的方程

$$= \delta p_i + \delta v_i \cdot \Delta t$$

1.3.2.4.1.6.9.9.27 1、对平移误差状态的导数系数

$$F_x.\text{block}<3, 3>(3, 3) = I$$

1.3.2.4.1.6.9.9.28 2、对速度误差状态的导数系数

$$F_x.\text{block}<3, 3>(3, 7) = \text{Eye3d} * dt;$$

1.3.2.4.1.6.9.9.29 3、速度误差项关于状态向量的方程（这里没写重力误差项，若是需要估计该项则在后面加上 $\delta g \cdot dt$ ，注意最后一项是加速度的反对称阵，不是加速度误差的反对称阵）

$$= \delta \hat{v}_i - \hat{R}_i \Delta t \cdot \delta \hat{b}_{ai} - \hat{R}_i \Delta t \cdot n_{ai} - \hat{R}_i [\delta \hat{a}_i]_\times \Delta t \cdot \delta \hat{r}_i$$

1.3.2.4.1.6.9.9.30 1、对旋转误差状态导数的系数

$$F_x.\text{block}<3, 3>(7, 0) = -R_imu * \text{acc_avr_skew} * dt;$$

1.3.2.4.1.6.9.9.31 2、对速度误差状态导数的系数

$$F_x.\text{block}<3, 3>(7, 7) = I$$

1.3.2.4.1.6.9.9.32 3、（可选 if(ba_bg_est_en)）对加速度偏差误差导数的系数

$$F_x.\text{block}<3, 3>(7, 13) = -R_imu * dt;$$

1.3.2.4.1.6.9.9.33 4、（可选 if(gravity_est_en)）对重力误差导数的系数

$$F_x.\text{block}<3, 3>(7, 16) = \text{Eye3d} * dt;$$

1.3.2.4.1.6.9.9.34 4、其他项

1.3.2.4.1.6.9.9.35 1、角速度偏差误差项关于状态向量的方程

$$\delta b_{w_{i+1}} = \delta b_{wi} + \Delta t \cdot n_{b_{wi}}$$

1.3.2.4.1.6.9.9.36 2、加速度偏差误差项关于状态向量的方程

$$\delta b_{a_{i+1}} = \delta b_{ai} + \Delta t \cdot n_{b_{ai}}$$

1.3.2.4.1.6.9.9.37 3、重力偏差项类似与上面

1.3.2.4.1.6.9.9.38 4、逆曝光时间偏差向量类似

1.3.2.4.1.6.9.9.39 3、cov_w 设置 19*19, F_w 类似于下图

$$\frac{\partial f(x,u,w)}{\partial w} = \begin{bmatrix} \frac{\partial \dot{R}}{\partial \eta_g} & \frac{\partial \dot{R}}{\partial \eta_a} & \frac{\partial \dot{R}}{\partial \eta_{bg}} & \frac{\partial \dot{R}}{\partial \eta_{ag}} \\ \frac{\partial \dot{p}}{\partial \eta_g} & \frac{\partial \dot{p}}{\partial \eta_a} & \frac{\partial \dot{p}}{\partial \eta_{bg}} & \frac{\partial \dot{p}}{\partial \eta_{ag}} \\ \frac{\partial \dot{v}}{\partial \eta_g} & \frac{\partial \dot{v}}{\partial \eta_a} & \frac{\partial \dot{v}}{\partial \eta_{bg}} & \frac{\partial \dot{v}}{\partial \eta_{ag}} \\ \frac{\partial \dot{b}_g}{\partial \eta_g} & \frac{\partial \dot{b}_g}{\partial \eta_a} & \frac{\partial \dot{b}_g}{\partial \eta_{bg}} & \frac{\partial \dot{b}_g}{\partial \eta_{ag}} \\ \frac{\partial \dot{b}_a}{\partial \eta_g} & \frac{\partial \dot{b}_a}{\partial \eta_a} & \frac{\partial \dot{b}_a}{\partial \eta_{bg}} & \frac{\partial \dot{b}_a}{\partial \eta_{ag}} \\ \frac{\partial \dot{g}}{\partial \eta_g} & \frac{\partial \dot{g}}{\partial \eta_a} & \frac{\partial \dot{g}}{\partial \eta_{bg}} & \frac{\partial \dot{g}}{\partial \eta_{ag}} \end{bmatrix}$$

注意 F_w 为 19*13 其与 F_x 同理，为状态误差项对噪声各项的求导系数。

1.3.2.4.1.6.9.9.40 1、（可选 if (exposure_estimate_en)）设置逆曝光协方差项

$$\text{cov_w}(6, 6) = \text{cov_inv_expo} * dt * dt;$$

1.3.2.4.1.6.9.9.41 2、旋转误差项对陀螺仪噪声的导数系数与噪声协方差乘。F_w 的左上角（对微小量做了近似约等于单位阵）。

$$\approx \exp(\hat{w}_i \Delta t)^\top \cdot \delta \hat{r}_i - \Delta t \cdot \delta \hat{b}_{wi} - \Delta t \cdot n_{wi}$$

$$\text{cov_w.block}(3, 3)(0, 0).diagonal() = \text{cov_gyr} * dt * dt;$$

1.3.2.4.1.6.9.9.42 3、关于加速度噪声的导数系数与噪声协方差乘

$$\text{cov_w.block}(3, 3)(7, 7) = R_imu * \text{cov_acc.asDiagonal()} * R_imu.transpose() * dt * dt;$$

1.3.2.4.1.6.9.9.43 4、对陀螺仪偏差噪声的导数系数与噪声协方差乘

```
cov_w.block<3, 3>(10, 10).diagonal() = cov_bias_gyr * dt * dt;
```

1.3.2.4.1.6.9.9.44 5、对加速度偏差噪声的导数系数与噪声协方差乘

```
cov_w.block<3, 3>(13, 13).diagonal() = cov_bias_acc * dt * dt
```

1.3.2.4.1.6.9.9.45 4、状态的先验协方差

$$\hat{\mathbf{P}}_{i+1} = \mathbf{F}_{\tilde{\mathbf{x}}} \hat{\mathbf{P}}_i \mathbf{F}_{\tilde{\mathbf{x}}}^T + \mathbf{F}_{\mathbf{w}} \mathbf{Q} \mathbf{F}_{\mathbf{w}}^T; \hat{\mathbf{P}}_0 = \bar{\mathbf{P}}_{k-1}.$$

```
state_inout.cov = F_x * state_inout.cov * F_x.transpose() + cov_w;
```

1.3.2.4.1.6.9.9.46 3、IMU 的姿态传播

$$\mathbf{R} = \mathbf{R} \text{Exp}((\boldsymbol{\omega}_m - \mathbf{b}_g) \Delta t)$$

```
R_imu = R_imu * Exp_f;
```

1.3.2.4.1.6.9.9.47 4、IMU 的全局坐标系下加速度（忽略图中噪声）

$${}^G \mathbf{R}_I (\mathbf{a}_m - \mathbf{b}_a - \mathbf{n}_a) + {}^G \mathbf{g}$$

```
acc_imu = R_imu * acc_avr + state_inout.gravity;
```

1.3.2.4.1.6.9.9.48 5、IMU 的位置传播

$$\mathbf{p} = \mathbf{p} + \mathbf{v} \Delta t + \frac{1}{2} (\mathbf{R} (\mathbf{a}_m - \mathbf{b}_a)) \Delta t^2 + \frac{1}{2} \mathbf{g} \Delta t^2$$

```
pos_imu = pos_imu + vel_imu * dt + 0.5 * acc_imu * dt * dt;
```

1.3.2.4.1.6.9.9.49 6、IMU 的速度传播

$$\mathbf{v} = \mathbf{v} + \mathbf{R} (\mathbf{a}_m - \mathbf{b}_a) \Delta t + \mathbf{g} \Delta t$$

```
vel_imu = vel_imu + acc_imu * dt;
```

1.3.2.4.1.6.9.9.50 7、保存这两个 IMU 数据获得的平均角速度和全局系下加速度

```
angvel_last = angvel_avr; acc_s_last = acc_imu;
```

1.3.2.4.1.6.9.9.51 8、推入 IMU 位姿（包括 下一个 Imu 到开始传播时刻的偏移时间，全局系的 imu 的均值加速度，均值角速度，先验速度，先验位置，先验姿态）

```
IMUpose.push_back(set_pose6d(off_s_t, acc_imu, angvel_avr, vel_imu, pos_imu, R_imu));
```

1.3.2.4.1.6.9.9.52 3、将最近的 LIO 更新时刻 更新为 本次 传播结束时刻（图像捕获时刻）

```
lidar_meas.last_lio_update_time = prop_end_time;
```

1.3.2.4.1.6.9.9.53 4、退出 break

1.3.2.4.1.6.9.10 10、将传播结束时刻的 IMU 速度，IMU 姿态，IMU 位置以及曝光时间更新到状态中。

```
state_inout.vel_end = vel_imu;state_inout.rot_end = R_imu;state_inout.pos_end  
= pos_imu;state_inout.inv_expo_time = tau;
```

1.3.2.4.1.6.9.11 11、获得队列的最后一个 IMU 消息 last_imu = v_imu.back();

1.3.2.4.1.6.9.12 12、更新上次传播结束时刻（本次图像捕获时刻）

```
last_prop_end_time = prop_end_time;
```

1.3.2.4.1.6.10 10、记录时间 t1: double t1 = omp_get_wtime();

1.3.2.4.1.6.11 11、开始进行点云去畸变（VIO 模式下不需要）。

```
if (lidar_meas.lio_vio_flg == LIO)本质上也是一种恒速模型
```

1.3.2.4.1.6.11.1 1、获得待去畸变点云的最后一个点。

```
auto it_pcl = pcl_wait_proc.points.end() - 1;
```

1.3.2.4.1.6.11.2 2、初始化一个旋转 extR_Ri（外参旋转的逆与 tk 时刻全局旋转的逆）和偏移 exrR_extT。

```
M3D extR_Ri(Lid_rot_to_IMU.transpose() * state_inout.rot_end.transpose());  
//世界系相对雷达系的旋转 V3D exrR_extT(Lid_rot_to_IMU.transpose() *  
Lid_offset_to_IMU); //Imu 系到雷达系的外参平移 Lid_rot_to_IMU 代表激光  
雷达到 imu 的旋转外参，Lid_offset_to_IMU 代表激光雷达到 Imu 的平移外  
参。
```

1.3.2.4.1.6.11.3 3、从最后一个 IMU 开始遍历直至队列第一个（倒着）：

```
这里就是取两个 Imu 之间的所有点，对这些点去畸变。for (auto it_kp =  
IMUpose.end() - 1; it_kp != IMUpose.begin(); it_kp--)
```

1.3.2.4.1.6.11.3.1 1、上一个 imu 和当前 Imu, 并取出上一个 imu 属性（每个 imu 间隔将从左侧 imu 开始传播）

```
auto head = it_kp - 1;auto tail = it_kp; R_imu <<  
MAT_FROM_ARRAY(head->rot); acc_imu <<  
VEC_FROM_ARRAY(head->acc); // cout<<"head imu acc:  
"<<acc_imu.transpose()<<endl; vel_imu << VEC_FROM_ARRAY(head->  
>vel); pos_imu << VEC_FROM_ARRAY(head->pos); angvel_avr <<  
VEC_FROM_ARRAY(head->gyr);
```

1.3.2.4.1.6.11.3.2 2、递减遍历：只要这个点的偏移时间 晚于 在左侧 imu 的时间。

```
for (; it_pcl->curvature / double(1000) > head->offset_time; it_pcl--)
```

1.3.2.4.1.6.11.3.3 1、获得当前点相对左侧 imu 的时间间隔：

```
dt = it_pcl->curvature / double(1000) - head->offset_time;
```

1.3.2.4.1.6.11.3.4 2、获得该点的状态（全局），因为 R_imu 就是全局的。

```
M3D R_i(R_imu * Exp(angvel_avr, dt));
```

1.3.2.4.1.6.11.3.5 3、获得该点到 tk 时刻的平移向量（全局）

```
: V3D T_ei(pos_imu + vel_imu * dt + 0.5 * acc_imu * dt * dt -  
state_inout.pos_end);
```

1.3.2.4.1.6.11.3.6 4、获得该点（相对采样时刻激光雷达系下的点坐标）

```
V3D P_i(it_pcl->x, it_pcl->y, it_pcl->z);
```

1.3.2.4.1.6.11.3.7 5、获得补偿后的点

```
V3D P_compensate = (extR_Ri * (R_i * (Lid_rot_to_IMU * P_i +  
Lid_offset_to_IMU) + T_ei) - exrR_extT);
```

1.3.2.4.1.6.11.3.8 6、校正该点--直至到第一个点

```
it_pcl->x = P_compensate(0); it_pcl->y = P_compensate(1); it_pcl->z =  
P_compensate(2); if (it_pcl == pcl_wait_proc.points.begin()) break;
```

1.3.2.4.1.6.11.4 4、将去完畸变的点云复制给 pcl_out，并清空 pcl_wait_proc 和 IMUPose

```
pcl_out = pcl_wait_proc; pcl_wait_proc.clear(); IMUPose.clear();
```

1.3.2.4.2 2、if (gravity_align_en) gravityAlignment(); ---可略过

判断是否进行重力对齐 ** 仅进行一次** 来自 yaml 文件 bool gravity_align_en //

LIVMapper.hvoid LIVMapper::gravityAlignment() 定义于 IMU_Processing.cpp

1.3.2.4.2.1 1、如果 imu 不需要初始化了且重量对齐未完成 if (!p_imu->imu_need_init

&& !gravity_align_finished)，获得状态重力与地球重力的旋转阵，将状态都旋转到对齐了地球重力的方向上。完成对齐。

1.3.2.4.3 3、记录状态，并将先验状态_state 和去畸变点云 feats_undistort 给体素地图管理器 voxelmap_manager

```
state_propagat = _state; // 声明于 LIVMapper.h StatesGroup
```

```
state_propagat; voxelmap_manager->state_ = _state; // 声明于 LIVMapper.h
```

```
VoxelMapManagerPtr voxelmap_manager; voxelmap_manager->feats_undistort_ =  
feats_undistort;
```

1.3.2.5 5、stateEstimationAndMapping();

```
void LIVMapper::stateEstimationAndMapping() 定义于 LIVMapper.cppvoid
```

```
LIVMapper::stateEstimationAndMapping() { switch (LidarMeasures.lio_vio_flg) { case VIO:  
handleVIO(); // 进行 VIO 状态更新 break; case LIO: case LO: handleLIO(); // 进行  
LIO 状态更新 break; }}
```

1.3.2.5.1 1、void LIVMapper::handleLIO()

定义于 LIVMapper.cpp 首先对点云进行体素滤波，并投到体素管理器中进行状态估计（EKF 的更新部分），优化后根据需要进行保存状态到评估文件，更新体素管理器的点不确定性、平面属性，将点转到世界系后进行发布和打印设置。

1.3.2.5.1.1 1、数据有效判断

1.3.2.5.1.1.1 1、将先验旋转状态转为欧拉角：

```
euler_cur = RotMtoEuler(_state.rot_end);V3D euler_cur; 声明于 LIVMapper.h
```

1.3.2.5.1.1.2 2、将先验状态输出 fout_pre 到文件 mat_pre.txt 中

```
fout_pre << setw(20) << LidarMeasures.last_lio_update_time (此时图像捕获时间)
- _first_lidar_time << " " << euler_cur.transpose() * 57.3 << " " <<
_state.pos_end.transpose() << " " << _state.vel_end.transpose() << " " <<
_state.bias_g.transpose() << " " << _state.bias_a.transpose() << " " <<
V3D(_state.inv_expo_time, 0, 0).transpose() << endl;
```

1.3.2.5.1.1.3 3、检查去畸变特征点云是否有点，没有点直接退出

```
if (feats_undistort->empty() || (feats_undistort == nullptr)) {std::cout << "[ LIO ]: No
point!!!" << std::endl;return; }
```

1.3.2.5.1.2 记录时间 double t0 = omp_get_wtime();

1.3.2.5.1.3 2、点云降采样

1.3.2.5.1.3.1 4、对去畸变点云进行体素滤波，

```
downSizeFilterSurf.setInputCloud(feats_undistort);downSizeFilterSurf.filter(*feats_d
own_body); feats_down_body 体素滤波后的去畸变点云
```

1.3.2.5.1.4 记录时间 double t_down = omp_get_wtime();

1.3.2.5.1.5 3、获得各系下的体素点云并构建体素地图

获得体素点云，计算体素坐标，体素中心坐标，生成八叉树节点，获得节点的平面属性，子叶分割（递归初始化节点），节点状态更新。读者可参考论文：

《Efficient and Probabilistic Adaptive Voxel Mapping for Accurate Online LiDAR Odometry》，不看也行，笔者已将关键公式附在代码上

1.3.2.5.1.5.1 5、体素图管理器 voxelmap_manager 获得 lidar 系和世界系下的降采样点云

```
feats_down_size = feats_down_body->points.size();voxelmap_manager-
>feats_down_body_ = feats_down_body;transformLidar(_state.rot_end,
_state.pos_end, feats_down_body, feats_down_world); ※ // 将 lidar 系转到世界系
voxelmap_manager->feats_down_world_ = feats_down_world;voxelmap_manager-
>feats_down_size_ = feats_down_size;其中 transformLidar 对应函数： void
LIVMapper::transformLidar(const Eigen::Matrix3d rot, const Eigen::Vector3d t,
const PointCloudXYZI::Ptr &input_cloud, PointCloudXYZI::Ptr &trans_cloud)
```

1.3.2.5.1.5.1.1 1、首先将世界系点云置空并重置大小与载体系点云一样：

```
PointCloudXYZI().swap(*trans_cloud);trans_cloud->reserve(input_cloud-
>size());
```

1.3.2.5.1.5.1.2 2、遍历每个载体系体素点：for (size_t i = 0; i < input_cloud->size(); i++), 将体素点乘以外参和状态转到世界系。只包括了位置和反射强度信息。

```
pcl::PointXYZINormal p_c = input_cloud->points[i];Eigen::Vector3d p(p_c.x,
p_c.y, p_c.z); p = (rot * (extR * p + extT) + t); PointType pi;pi.x = p(0);pi.y
= p(1);pi.z = p(2);pi.intensity = p_c.intensity;trans_cloud-
>points.push_back(pi);
```

1.3.2.5.1.5.2 6、判断地图是否初始化，第一次运行时执行 if (!lidar_map_inited)（仅执行一次，用于初始化根节点）

```
if (!lidar_map_inited) { lidar_map_inited = true; voxelmap_manager->BuildVoxelMap(); ※ //构建体素地图。 } 对应函数 void VoxelMapManager::BuildVoxelMap() 定义于 voxel_map.cpp 获得每个节点的平面属性，体素坐标，体素中心坐标，子叶分割（递归初始化节点），节点状态更新。
```

1.3.2.5.1.5.2.1 1、对一些参数进行初始化

```
float voxel_size = config_setting_.max_voxel_size_; // 最大体素尺寸  
==0.5m float planer_threshold = config_setting_.planner_threshold_; // 平面阈值  
==0.01 int max_layer = config_setting_.max_layer_; // 2, 从 0 计数，因此总层数为 3 层  
int max_points_num = config_setting_.max_points_num_; // 最大点数 50（每个节点）  
std::vector<int> layer_init_num = config_setting_.layer_init_num_; // 层的初始化点数量阈值
```

1.3.2.5.1.5.2.2 2、声明一个类型为 不确定性点的向量

```
std::vector<pointWithVar> input_points;
```

1.3.2.5.1.5.2.3 3、遍历世界系下所有的点

```
for (size_t i = 0; i < feats_down_world_->size(); i++) 根据激光雷达深度误差和轴承误差，计算载体系和世界系点的不确定性，将带不确定性 var 的世界点 point_w 赋予 pointWithVar 对象，并推入到 input_points 中。
```

1.3.2.5.1.5.2.3.1 1、声明一个 不确定性点 对象

```
pointWithVar pv;
```

1.3.2.5.1.5.2.3.2 2、将世界系点坐标给不确定性点的世界系坐标

```
pv.point_w << feats_down_world_->points[i].x, feats_down_world_->points[i].y, feats_down_world_->points[i].z;
```

1.3.2.5.1.5.2.3.3 3、将载体系的点赋予一个三维向量中

```
V3D point_this(feats_down_body_->points[i].x, feats_down_body_->points[i].y, feats_down_body_->points[i].z);
```

1.3.2.5.1.5.2.3.4 4、M3D var; 计算载体系点的不确定性（协方差）（根据深度误差和方向误差）

$$\delta_{L_{\mathbf{P}_i}} = \underbrace{\begin{bmatrix} \omega_i & -d_i[\omega_i]_{\wedge} \mathbf{N}(\omega_i) \end{bmatrix}}_{\mathbf{A}_i} \begin{bmatrix} \delta_{d_i} \\ \delta_{\omega_i} \end{bmatrix} \sim \mathcal{N}(\mathbf{0}, \Sigma_{L_{\mathbf{P}_i}}),$$

$$\Sigma_{L_{\mathbf{P}_i}} = \mathbf{A}_i \begin{bmatrix} \Sigma_{d_i} & \mathbf{0}_{1 \times 2} \\ \mathbf{0}_{2 \times 1} & \Sigma_{\omega_i} \end{bmatrix} \mathbf{A}_i^T.$$

```
calcBodyCov(point_this, config_setting_.dept_err_, config_setting_.beam_err_, var); 对应函数: void calcBodyCov(Eigen::Vector3d &pb, const float range_inc, const float degree_inc, Eigen::Matrix3d &cov) 声明于 voxel_map.h, 定义于
```

voxel_map.cpp 可参考论文《Efficient and Probabilistic Adaptive Voxel Mapping for Accurate Online LiDAR Odometry》和《Pixel-level Extrinsic Self Calibration of High Resolution LiDAR and Camera in Targetless Environments》

1.3.2.5.1.5.2.3.5 1、如果载体系点的 z 轴值为 0，则将其置为 0.0001. 避免将 0 作为除数

```
if (pb[2] == 0) pb[2] = 0.0001;
```

1.3.2.5.1.5.2.3.6 2、计算该点的距离

```
float range = sqrt(pb[0] * pb[0] + pb[1] * pb[1] + pb[2] * pb[2]);
```

1.3.2.5.1.5.2.3.7 3、计算测距的方差

```
float range_var = range_inc * range_inc; 0.02*0.02。基于深度误差得到  
dept_err
```

1.3.2.5.1.5.2.3.8 4、关于方向方差的矩阵

```
Eigen::Matrix2d direction_var;
```

1.3.2.5.1.5.2.3.9 5、获得方向方差

```
direction_var << pow(sin(DEG2RAD(degree_inc)), 2), 0, 0,  
pow(sin(DEG2RAD(degree_inc)), 2); degree_inc = cam_err == 0.05
```

1.3.2.5.1.5.2.3.10 6、计算该点的方向

```
Eigen::Vector3d direction(pb);
```

1.3.2.5.1.5.2.3.11 7、对方向归一化（长度变为 1 但方向不变）

```
direction.normalize();
```

1.3.2.5.1.5.2.3.12 8、计算方向的反对称矩阵

```
Eigen::Matrix3d direction_hat << 0, -direction(2), direction(1), direction(2),  
0, -direction(0), -direction(1), direction(0), 0;
```

1.3.2.5.1.5.2.3.13 9、计算 N 的 N1 和 N2 以及 N

```
Eigen::Vector3d base_vector1(1, 1, -(direction(0) + direction(1)) /  
direction(2)); base_vector1.normalize(); Eigen::Vector3d base_vector2 =  
base_vector1.cross(direction); base_vector2.normalize(); Eigen::Matrix<double,  
3, 2> N; N << base_vector1(0), base_vector2(0), base_vector1(1),  
base_vector2(1), base_vector1(2), base_vector2(2);
```

1.3.2.5.1.5.2.3.14 10、计算 A

```
Eigen::Matrix<double, 3, 2> A = range * direction_hat * N;
```

1.3.2.5.1.5.2.3.15 11、计算不确定性（协方差）

```
cov = direction * range_var * direction.transpose() + A * direction_var *  
A.transpose();
```


1.3.2.5.1.5.2.3.16 5、进一步计算世界系点的不确定性（协方差）到 pv，并将 pv 推入 input_points

$$\Sigma_{w_{p_i}} = {}^W_L R \Sigma_{L_{p_i}} {}^W_L R^T + {}^W_L R [{}^L_{p_i}]_{\wedge} \Sigma_R [{}^L_{p_i}]^T {}^W_L R^T + \Sigma_t \quad (3)$$

感谢热心读者的提醒，笔者这里标志一下：这里的协方差计算与论文实现是不一致的，具体而言就是少了旋转，也少了 lidar2imu 的外参。这确实是一个代码 bug，这与 lidar 噪声的设置有关，具体怎么相关的请移步[这里](#)和[这里](#)了解详情！

```
M3D point_crossmat; point_crossmat <<
SKEW_SYM_MATRX(point_this); var = (state_rot_end * extR_) * var *
(state_rot_end * extR_).transpose() + (-point_crossmat) *
state_cov.block<3, 3>(0, 0) * (-point_crossmat).transpose() +
state_cov.block<3, 3>(3, 3);pv.var = var;input_points.push_back(pv);
```

1.3.2.5.1.5.2.4 4、获取 input_points 中的数：uint plsize = input_points.size();

1.3.2.5.1.5.2.5 5、遍历所有不确定性点

for (uint i = 0; i < plsize; i++)计算该帧世界系点云所有点对应的体素坐标并将该点推入到对应的树节点临时点容器中，或者生成一个新的节点。注意体素中心到体素各面的距离与层数和体素边长的关系如下：D = V_size * (1/2)^n，第一层也就是根节点为边长的二分之一，第二场为边长的四分之一，第三层为边长的八分之一。

1.3.2.5.1.5.2.5.1 1、获取当前结构体（点）

```
const pointWithVar p_v = input_points[i]
```

1.3.2.5.1.5.2.5.2 2、各轴除以体素尺寸获取当前点所在体素的空间位置 float loc_xyz[3];

```
for (int j = 0; j < 3; j++) {loc_xyz[j] = p_v.point_w[j] / voxel_size;if
(loc_xyz[j] < 0) { loc_xyz[j] -= 1.0; } }比如点[0.8,3.6,2.4]，除以体素尺寸
0.5，可知该点位于：x 轴方向 1.6 个体素，y 轴方向 7.2 个体素，z 轴
方向 4.8 个体素处 loc_xyz=[1.6,7.2,4.8] 整个大体素（方块）坐标：
[1,7,4]。至于为什么小于 0 要-1，（这涉及体素滤波原理）可以这么理解，
激光点位置在[0~0.5,0~0.5,0~0.5]范围内的整个大体素（方块）坐标
为[0,0,0]，那么激光点位置在[0~0.5,0~0.5,0~0.5]范围内的整个大体素
（方块）坐标为[-1,0,0]，在[0~0.5,0~0.5,0~0.5]范围内的整个大体素
（方块）坐标为[-1,-1,-1]。以此类推点[-0.8,-3.5,-2.4]的 loc_xyz=[-2.6,-
8.2,-5.8]，整个大体素（方块）坐标[-2, -8, -5]
```

1.3.2.5.1.5.2.5.3 3、将 loc_xyz 化为整型后 赋予 体素位置类对象

```
VOXEL_LOCATION position((int64_t)loc_xyz[0], (int64_t)loc_xyz[1],
(int64_t)loc_xyz[2]);
```

1.3.2.5.1.5.2.5.4 4、利用哈希表返回该体素位置的八叉树索引

```
auto iter = voxel_map_.find(position);
```

1.3.2.5.1.5.2.5.5 5、判断 如果不等于哈希表末尾 if (iter != voxel_map_.end())，则说明该体素方块有八叉树节点，将点推入到该节点中，更新点数。

```
voxel_map_[position]-
>temp_points_.push_back(p_v);voxel_map_[position]->new_points_++;
```

1.3.2.5.1.5.2.5.6 6、否则 说明该体素没有八叉树节点，生成一个根节点。

1.3.2.5.1.5.2.5.7 1、生成一个新的八叉树：

```
VoxelOctoTree *octo_tree = new VoxelOctoTree(max_layer, 0,  
layer_init_num[0], max_points_num, planer_threshold); // 设置最大层索引  
2. 当前层数为 0，点的数量阈值 5，点数上限 50，
```

1.3.2.5.1.5.2.5.8 2、将该树节点赋值给哈希表该位置处

```
voxel_map_[position] = octo_tree;
```

1.3.2.5.1.5.2.5.9 3、设置四分之一的体素尺寸

```
voxel_map_[position]->quater_length_ = voxel_size / 4; 为了计算子体素中  
心点在世界系的坐标，所以需要子体素的半边长，即 1/4 父体素边长。
```

1.3.2.5.1.5.2.5.10 4、该位置体素的中心点坐标

```
voxel_map_[position]->voxel_center_ = (0.5+position) * voxel_size; 可  
以这么理解 展开该式： position*voxel_size+0.5*voxel_size，第一项是先  
将体素坐标根据体素尺寸恢复到绝对尺度，把体素坐标看做体素顶点中  
离原点最近的那个顶点的坐标，那么中心点的坐标都比这个顶点多一半  
的体素尺寸，加上这个距离，就是体素中心了。
```

1.3.2.5.1.5.2.5.11 5、将当前点推入哈希表该体素位置的八叉树中

```
voxel_map_[position]->temp_points_.push_back(p_v);
```

1.3.2.5.1.5.2.5.12 6、更新该树的点数

```
voxel_map_[position]->new_points_++;
```

1.3.2.5.1.5.2.5.13 7、设置层的初始数量（用于规定该层点的阈值）

```
voxel_map_[position]->layer_init_num_ = layer_init_num; [5, 5, 5, 5, 5]
```

1.3.2.5.1.5.2.6 6、遍历哈希表中的索引对每个根节点进行初始化：

```
for (auto iter = voxel_map_.begin(); iter != voxel_map_.end(); ++iter) iter-  
>second->init_octo_tree();对应函数： void VoxelOctoTree::init_octo_tree()声  
明于 voxel_map.h, 定义于 voxel_map.cpp 获得节点的平面属性，体素坐标，  
体素中心坐标，子叶分割，节点状态更新。
```

1.3.2.5.1.5.2.6.1 1、判断该树节点的点数是否超过了点数最小阈值

```
if (temp_points_.size() > points_size_threshold )获得节点的平面属性，体  
素坐标，体素中心坐标，子叶分割，节点状态更新。
```

1.3.2.5.1.5.2.6.2 1、 `init_plane`(temp_points_, plane_ptr_);

```
调用初始化平面函数： 根据该节点已有点判断平面情况并将结果给该  
节点的平面标志指针。（世界系下）对应函数： void  
VoxelOctoTree::init_plane(const std::vector<pointWithVar> &points,  
VoxelPlane *plane) 声明于 voxel_map.h, 定义于 voxel_map.cpp 获得平  
面法向量，（基于点不确定性）平面不确定性，平面方程各参数，平面
```

为真标志位，平面更新标志位，平面的 ID 号等。

1.3.2.5.1.5.2.6.3 1、对该节的平面属性进行初始化

```
plane->plane_var_ = Eigen::Matrix<double, 6, 6>::Zero(); // 平面不确定性
plane->covariance_ = Eigen::Matrix3d::Zero(); // 协方差 plane->center_ =
Eigen::Vector3d::Zero(); // 中心点 plane->normal_ =
Eigen::Vector3d::Zero(); // 平面法向量 plane->points_size_ = points.size();
//平面的点数 plane->radius_ = 0; // 最大特征值的开方
```

1.3.2.5.1.5.2.6.4 2、遍历该节点的每个不确定性点

$$\bar{\mathbf{p}} = \frac{1}{N} \sum_{i=1}^N w_i \mathbf{p}_i; \quad \mathbf{A} = \frac{1}{N} \sum_{i=1}^N (w_i \mathbf{p}_i - \bar{\mathbf{p}}) (w_i \mathbf{p}_i - \bar{\mathbf{p}})^T;$$

for (auto pv : points){plane->covariance_ += pv.point_w *
pv.point_w.transpose();plane->center_ += pv.point_w;} 这里计算的是该节点所有点的累加和，以及世界点与自己本身的转置。这里不明白也不要紧，读者要结合下面的第三项进行理解。笔者注：在编写代码时，往往实现的是公式的展开化简式，而非死板地硬套公式。

1.3.2.5.1.5.2.6.5 3、计算平面协方差 $E((P-\bar{P})(P-\bar{P})^T) = \text{COV}$ $\bar{P}$ 代表均值（期望）

$$\begin{aligned} & \frac{1}{N} \sum_{i=1}^N (w_i \mathbf{p}_i - \bar{\mathbf{p}}) (w_i \mathbf{p}_i - \bar{\mathbf{p}})^T \\ &= \frac{1}{N} \sum_{i=1}^N (w_i \mathbf{p}_i \mathbf{p}_i^T - \bar{\mathbf{p}} w_i \mathbf{p}_i^T - w_i \bar{\mathbf{p}}^T \mathbf{p}_i + \bar{\mathbf{p}} \bar{\mathbf{p}}^T) \\ &= \frac{1}{N} \left(\sum_{i=1}^N w_i \mathbf{p}_i \mathbf{p}_i^T - \bar{\mathbf{p}} \sum_{i=1}^N w_i \mathbf{p}_i^T - \sum_{i=1}^N w_i \bar{\mathbf{p}}^T \mathbf{p}_i + N \bar{\mathbf{p}} \bar{\mathbf{p}}^T \right) \\ &= \frac{1}{N} \sum_{i=1}^N w_i \mathbf{p}_i \mathbf{p}_i^T - \frac{1}{N} \bar{\mathbf{p}} \sum_{i=1}^N w_i \mathbf{p}_i^T - \frac{1}{N} \sum_{i=1}^N w_i \bar{\mathbf{p}}^T \mathbf{p}_i + N \bar{\mathbf{p}} \bar{\mathbf{p}}^T \end{aligned}$$

plane->center_ = plane->center_ / plane->points_size; plane->covariance_ =
plane->covariance_ / plane->points_size - plane->center_ * plane->
center_.transpose(); 可以看到左侧均值的累加工作在第二项已经完成了，
下面读者将介绍右侧跟公式的对应关系，如图所示进行展开，笔者将最后一步留给读者，聪明的你一定能明白。

1.3.2.5.1.5.2.6.6 4、获得协方差的特征向量和特征值

```
Eigen::EigenSolver<Eigen::Matrix3d> es(plane-
>covariance_); Eigen::Matrix3cd evects = es.eigenvectors(); Eigen::Vector3cd
evals = es.eigenvalues();
```

1.3.2.5.1.5.2.6.7 5、特征值的实数部分

```
Eigen::Vector3d evalsReal; evalsReal = evals.real();
```

1.3.2.5.1.5.2.6.8 6、找到 evals 向量的最小值，中间值以及最大值的索引

```
Eigen::Matrix3f::Index evalsMin,
evalsMax; evalsReal.rowwise().sum().minCoeff(&evalsMin); evalsReal.roww
ise().sum().maxCoeff(&evalsMax); int evalsMid = 3 - evalsMin - evalsMax;
```

// 因为特征值向量容器的大小是 3，通过此法可以获得特征值中间值的索引。

1.3.2.5.1.5.2.6.9 7、获得每个特征值对应的特征向量

```
Eigen::Vector3d evecMin = evecs.real().col(evalsMin);Eigen::Vector3d
evecMid = evecs.real().col(evalsMid);Eigen::Vector3d evecMax =
evecs.real().col(evalsMax);
```

1.3.2.5.1.5.2.6.10 8、生成一个对角阵，元素为点数的倒数

$$\frac{\partial \mathbf{q}}{\partial W_{\mathbf{p}_i}} = \mathbf{diag} \left(\frac{1}{N}, \frac{1}{N}, \frac{1}{N} \right)$$

```
Eigen::Matrix3d J_Q; J_Q << 1.0 / plane->points_size_, 0, 0, 0, 1.0 / plane-
>points_size_, 0, 0, 0, 1.0 / plane->points_size_;
```

1.3.2.5.1.5.2.6.11 9、判断最小的特征是否小于平面阈值

$$[\mathbf{n}, \mathbf{q}]^T = \mathbf{f}(^W\mathbf{p}_1, ^W\mathbf{p}_2, \dots, ^W\mathbf{p}_N).$$

if (evalsReal(evalsMin) < planer_threshold_)获得平面法向量，（基于点不确定性）平面不确定性，平面方程各参数，平面为真标志位，平面更新标志位，平面的 ID 号。

1.3.2.5.1.5.2.6.12 1、遍历此节点所有的点获得平面的不确定性（基于点的不确定性得到）

$$\Sigma_{\mathbf{n}, \mathbf{q}} = \sum_{i=1}^N \frac{\partial \mathbf{f}}{\partial W_{\mathbf{p}_i}} \Sigma_{^W\mathbf{p}_i} \frac{\partial \mathbf{f}}{\partial W_{\mathbf{p}_i}}^T$$

for (int i = 0; i < points.size(); i++)分别求法向量对世界点的导数和中心点对世界点的导数。又已知了世界点的不确定性，因此可以获得平面的不确定性。

1.3.2.5.1.5.2.6.13 1、生成一个 63 的矩阵 J Eigen::Matrix<double, 6, 3> J; 和 33 的矩阵 F:

```
Eigen::Matrix3d F;
```

1.3.2.5.1.5.2.6.14 2、遍历三个特征值索引获得 F

$$\frac{\partial \mathbf{n}}{\partial W_{\mathbf{p}_i}} = \mathbf{U} \begin{bmatrix} \mathbf{F}_{1,3}^{\mathbf{p}_i} \\ \mathbf{F}_{2,3}^{\mathbf{p}_i} \\ \mathbf{F}_{3,3}^{\mathbf{p}_i} \end{bmatrix}, \mathbf{F}_{m,3}^{\mathbf{p}_i} = \begin{cases} \frac{(^W\mathbf{p}_i - \mathbf{q})^T}{N(\lambda_3 - \lambda_m)} (\mathbf{u}_m \mathbf{n}^T + \mathbf{n} \mathbf{u}_m^T), m \neq 3, \\ \mathbf{0}_{1 \times 3}, m = 3. \end{cases}$$

$$\frac{\partial \mathbf{q}}{\partial W_{\mathbf{p}_i}} = \mathbf{diag} \left(\frac{1}{N}, \frac{1}{N}, \frac{1}{N} \right) \quad (7)$$

for (int m = 0; m < 3; m++),

1.3.2.5.1.5.2.6.15 1、判断 若不是最小值索引 if (m != (int)evalsMin) 则计算该索引处 F 的值。

```
(points[i].point_w - plane->center_).transpose() / ((plane->points_size_) *
```

```
(evalsReal[evalsMin] - evalsReal[m])) * (evecs.real().col(m) *
evecs.real().col(evalsMin).transpose() + evecs.real().col(evalsMin) *
evecs.real().col(m).transpose());
```

1.3.2.5.1.5.2.6.16 2、若是最小索引值 F_m 为 0 向量

```
Eigen::Matrix<double, 1, 3> F_m; F_m << 0, 0, 0; F.row(m) = F_m;
```

1.3.2.5.1.5.2.6.17 3、获得法向量对点的导数，以及中心点对点的导数

$$\frac{\partial \mathbf{n}}{\partial \mathbf{w}_{\mathbf{p}_i}} = \mathbf{U} \begin{bmatrix} \mathbf{F}_{1,3}^{\mathbf{p}_i} \\ \mathbf{F}_{2,3}^{\mathbf{p}_i} \\ \mathbf{F}_{3,3}^{\mathbf{p}_i} \end{bmatrix}, \mathbf{F}_{m,3}^{\mathbf{p}_i} = \begin{cases} \frac{(\mathbf{w}_{\mathbf{p}_i} - \mathbf{q})^T}{N(\lambda_3 - \lambda_m)} (\mathbf{u}_m \mathbf{n}^T + \mathbf{n} \mathbf{u}_m^T), m \neq 3, \\ \mathbf{0}_{1 \times 3}, m = 3. \end{cases}$$

$$\frac{\partial \mathbf{q}}{\partial \mathbf{w}_{\mathbf{p}_i}} = \mathbf{diag} \left(\frac{1}{N}, \frac{1}{N}, \frac{1}{N} \right) \quad (7)$$

```
J.block<3, 3>(0, 0) = evecs.real() * F; J.block<3, 3>(3, 0) = J_Q;
```

1.3.2.5.1.5.2.6.18 4、获得平面的不确定性

```
plane->plane_var_ += J * points[i].var * J.transpose();
```

1.3.2.5.1.5.2.6.19 2、更新平面属性

1.3.2.5.1.5.2.6.20 1、平面法向量

```
plane->normal_ << evecs.real()(0, evalsMin), evecs.real()(1, evalsMin),
evecs.real()(2, evalsMin);
```

1.3.2.5.1.5.2.6.21 2、平面 y 方向的向量（与法向量垂直）

```
plane->y_normal_ << evecs.real()(0, evalsMid), evecs.real()(1, evalsMid),
evecs.real()(2, evalsMid);
```

1.3.2.5.1.5.2.6.22 3、平面 x 方向的向量（与法向量垂直）

```
plane->x_normal_ << evecs.real()(0, evalsMax), evecs.real()(1, evalsMax),
evecs.real()(2, evalsMax);
```

1.3.2.5.1.5.2.6.23 4、平面特征值

```
plane->min_eigen_value_ = evalsReal(evalsMin); plane->mid_eigen_value_
= evalsReal(evalsMid); plane->max_eigen_value_ = evalsReal(evalsMax);
```

1.3.2.5.1.5.2.6.24 5、平面残差定义为最大特征值的开方

```
plane->radius_ = sqrt(evalsReal(evalsMax));
```

1.3.2.5.1.5.2.6.25 6、计算平面方程 Ax + By + Cz + d = 0 的 d

```
plane->d_ = -(plane->normal_(0) * plane->center_(0) + plane->normal_(1) *
plane->center_(1) + plane->normal_(2) * plane->center_(2));
```

1.3.2.5.1.5.2.6.26 7、设该节点的平面属性为真平面，平面更新标志位设为真。

```
plane->is_plane_ = true; plane->is_update_ = true;
```

1.3.2.5.1.5.2.6.27 3、平面是否未初始化 if (!plane->is_init_), 赋予当前平面一个 Id 序号, 平面 id 自加, 初始化为真。

```
plane->id_ = voxel_plane_id; voxel_plane_id++; plane->is_init_ = true;
```

1.3.2.5.1.5.2.6.28 10、否则 最小的特征值大于平面阈值: 平面更新设为真, 但不是平面

```
plane->is_update_ = true; plane->is_plane_ = false;
```

1.3.2.5.1.5.2.6.29 2、判断是否 为平面

```
if (plane_ptr->is_plane_ == true) 为真该节点就终止了, 并将节点中的临时点清空, 留出存储。
```

1.3.2.5.1.5.2.6.30 1、将该树节点设为终止节点

```
octo_state_ = 0;
```

1.3.2.5.1.5.2.6.31 2、判断, 若该节点点数是否超出了最大点数

```
if (temp_points_.size() > max_points_num_) { update_enable_ = false; std::vector<pointWithVar>().swap(temp_points_); new_points_ = 0; }  
移除当前节点临时点云, 并失能更新标志 (此节点不再更新)
```

1.3.2.5.1.5.2.6.32 3、否则 该节点状态设为非终止节点继续分割树节点。

```
else { octo_state_ = 1; cut_octo_tree(); } 对应函数: void VoxelOctoTree::cut_octo_tree() 声明于 voxel_map.h, 定义于 voxel_map.cpp
```

1.3.2.5.1.5.2.6.33 1、如果该节点的层数已经到了最大层, 将该节点设为终止点。返回

```
if (layer_ >= max_layer_) { octo_state_ = 0; return; }
```

1.3.2.5.1.5.2.6.34 2、遍历该节点的所有临时点

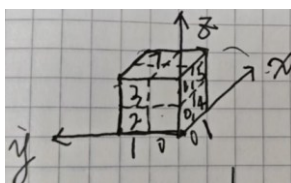
```
for (size_t i = 0; i < temp_points_.size(); i++) 划分 (生成) 子叶, 计算叶的坐标和中心点坐标, 将点推入该叶中。循环后 8 个子节点已全部划分完毕。
```

1.3.2.5.1.5.2.6.35 1、生成一个位置数组(最靠近原点坐标的) int xyz[3] = {0, 0, 0};

1.3.2.5.1.5.2.6.36 2、看似判断实则是计算子叶的坐标

```
if (temp_points_[i].point_w[0] > voxel_center_[0]) { xyz[0] = 1; }  
if (temp_points_[i].point_w[1] > voxel_center_[1]) { xyz[1] = 1; }  
if (temp_points_[i].point_w[2] > voxel_center_[2]) { xyz[2] = 1; }
```

1.3.2.5.1.5.2.6.37 3、初始化该点所在的叶数: int leafnum = 4 * xyz[0] + 2 * xyz[1] + xyz[2];



根据该式定义该点在八个子叶的哪个叶上

1.3.2.5.1.5.2.6.38 4、判断该点所在叶是否为空（是否有八叉树节点（对象））：if (leaves_[leafnum] == nullptr)。

生成新的八叉树并计算对应的子体素中心

1.3.2.5.1.5.2.6.39 1、生成一个八叉树节点给该叶，层数在原来父节点层数上加 1：leaves_[leafnum] = new VoxelOctoTree(max_layer_, layer_ + 1, layer_init_num_[layer_ + 1], max_points_num_, planer_threshold_);

1.3.2.5.1.5.2.6.40 2、将点数下限数组给该叶：leaves_[leafnum]->layer_init_num_ = layer_init_num_;

1.3.2.5.1.5.2.6.41 3、计算子叶的中心（在父节点的中心上计算得到）

```
leaves_[leafnum]->voxel_center_[0] = voxel_center_[0] + (2 * xyz[0] - 1) *
quater_length_;leaves_[leafnum]->voxel_center_[1] = voxel_center_[1] + (2
* xyz[1] - 1) * quater_length_;leaves_[leafnum]->voxel_center_[2] =
voxel_center_[2] + (2 * xyz[2] - 1) * quater_length_;
```

1.3.2.5.1.5.2.6.42 4、为计算下一层的各子叶中心做准备（计算体素边长的八分之一）

```
leaves_[leafnum]->quater_length_ = quater_length_ / 2;
```

1.3.2.5.1.5.2.6.43 5、将这个点推到该子叶的临时点中，子叶的点数更新

```
leaves_[leafnum]-
>temp_points_.push_back(temp_points_[i]);leaves_[leafnum]-
>new_points_++;
```

1.3.2.5.1.5.2.6.44 3、遍历这八个子叶并只对有效的子叶做处理，一种递归实现。

```
for (uint i = 0; i < 8; i++){ if (leaves_[i] != nullptr){ 判断该节点的点数是
否满足最小点数阈值； 重复实现上一级的第 1 项到第 4 项}}1、判断
该树节点的点数是否超过了点数阈值 if (temp_points_.size() >
points_size_threshold_); 进行平面初始化; 判断平面; 判断是否继续分割
体素, 更新节点状态。
```

1.3.2.5.1.5.2.6.45 4、设置初始化节点为真 init_octo_ = true; , 点数置为 0. new_points_ = 0;

1.3.2.5.1.5.2.6.46 2、没有超过点数最小阈值（可以默认该节点不做任何处理），该节点初始化状态为 false . 点数太少。

1.3.2.5.1.6 记录时间 double t1 = omp_get_wtime();

1.3.2.5.1.7 4、用体素管理器进行状态估计

状态估计中还会再计算一次点的不确定性更新体素管理器的状态和协方差（后验）_state , 以及基于后验状态获得的不确定性点的列表**_pv_list **=
voxelmap_manager->pv_list_;

1.3.2.5.1.7.1 7、voxelmap_manager->StateEstimation(state_propagat);

基于先验状态 进行状态更新对应函数：void

VoxelMapManager::StateEstimation(StatesGroup &state_propagat) 声明于
voxel_map.h 定义于 voxel_map.cpp

1.3.2.5.1.7.1.1 1、根据当前帧点云数量初始化反对称阵 `cross_mat_list_` 和载体协方差阵 `body_cov_list_` 的列表长度。（用于后面的观测方程雅可比阵）

```
cross_mat_list_.clear();cross_mat_list_.reserve(feats_down_size_);body_cov_list_.clear();body_cov_list_.reserve(feats_down_size_);
```

1.3.2.5.1.7.1.2 2、遍历当前帧的点，进行不确定性计算和点反对称阵计算。

```
for (size_t i = 0; i < feats_down_body_->size(); i++)。计算每个点的不确定性到 body_cov_list_，以及 IMU 系下点的反对称阵到 cross_mat_list_
```

1.3.2.5.1.7.1.2.1 1、获得当前循环点并避免高度为 0 的点

```
V3D point_this(feats_down_body_->points[i].x, feats_down_body_->points[i].y, feats_down_body_->points[i].z);if (point_this[2] == 0) { point_this[2] = 0.001; } 避免 0 高度点。
```

1.3.2.5.1.7.1.2.2 2、计算载体点的不确定性，并将不确定性推入载体系不确定性列表 `body_cov_list_` 中

```
calcBodyCov(point_this, config_setting_.dept_err_, config_setting_.beam_err_, var);body_cov_list_.push_back(var);
```

1.3.2.5.1.7.1.2.3 3、将 lidar 系的点转到 IMU 坐标系

```
point_this = extR_ * point_this + extT_;
```

1.3.2.5.1.7.1.2.4 4、计算 imu 系下点的反对称阵 `M3D point_crossmat`; 并推入列表 `cross_mat_list_`

```
point_crossmat << SKEW_SYM_MATRX(point_this);cross_mat_list_.push_back(point_crossmat);
```

1.3.2.5.1.7.1.3 3、重置 不确定性点列表 `pv_list_`

```
vector<pointWithVar>().swap(pv_list_);pv_list_.resize(feats_down_size_);
```

1.3.2.5.1.7.1.4 4、生成状态系数阵并初始化

```
MD(DIM_STATE, DIM_STATE) G, H_T_H, I_STATE; G.setZero();H_T_H.setZero();I_STATE.setIdentity();
```

1.3.2.5.1.7.1.5 5、三个关于 EKF 运行情况的标志位：EKF 初始化，EKF 收敛，EKF 停止

```
bool flg_EKF_initied, flg_EKF_converged, EKF_stop_flg = 0;
```

1.3.2.5.1.7.1.6 6、ESKF 迭代更新，次数不超过 5 次：

```
for (int iterCount = 0; iterCount < config_setting_.max_iterations_; iterCount++) 更新状态（后验状态）及协方差（后验协方差）。
```

1.3.2.5.1.7.1.6.1 1、记录每次迭代的总残差值： `double total_residual = 0.0;`

1.3.2.5.1.7.1.6.2 2、每次迭代得到的世界系雷达点： `world_lidar`

```
pcl::PointCloud<pcl::PointXYZI>::Ptr world_lidar(new
```



```
pcl::PointCloud<pcl::PointXYZI>);TransformLidar(state_.rot_end,
state_.pos_end, feats_down_body_, world_lidar);
```

1.3.2.5.1.7.1.6.3 3、旋转协方差 rot_var 和平移协方差 t_var 单拎出来：

```
M3D rot_var = state_.cov.block<3, 3>(0, 0); M3D t_var =
state_.cov.block<3, 3>(3, 3);
```

1.3.2.5.1.7.1.6.4 4、遍历 lidar 系当前帧点云的每个点：

```
for (size_t i = 0; i < feats_down_body_>size(); i++)每次状态迭代后，更新
不确定点列表 pv_list_ 的载体不确定性点和世界系不确定性点
```

1.3.2.5.1.7.1.6.5 2、获得载体系和世界系的位置：pv.point_b 和 pv.point_w

```
pv.point_b << feats_down_body_>points[i].x, feats_down_body_>
points[i].y, feats_down_body_>points[i].z;pv.point_w << world_lidar-
>points[i].x, world_lidar->points[i].y, world_lidar->points[i].z;
```

1.3.2.5.1.7.1.6.6 1、引用重置后的不确定性点（获得世界系和载体系的不确定性）

```
pointWithVar &pv = pv_list_[i];
```

1.3.2.5.1.7.1.6.7 3、计算世界系下点的不确定性：

```
cov = state_.rot_end * cov * state_.rot_end.transpose() + (-point_crossmat) *
rot_var * (-point_crossmat.transpose()) + t_var;
```

1.3.2.5.1.7.1.6.8 4、将不确定性返回给体素管理器的不确定性点列表 **pv**：

```
pv.var = cov;pv.body_var = body_cov_list_[i];
```

1.3.2.5.1.7.1.6.9 5、清空点到平面相关状态 列表

```
ptpl_list_ : ptpl_list_.clear();其中 std::vector<PointToPlane> ptpl_list_;声明
于 voxel_map.h
```

1.3.2.5.1.7.1.6.10 6、BuildResidualListOMP(**pv_list_**, **ptpl_list_**);

（基于 OpenMP）构建残差列表：点到平面状态。对应函数：void VoxelMapManager::BuildResidualListOMP(std::vector<pointWithVar> &pv_list, std::vector<PointToPlane> &ptpl_list) 声明于 voxel_map.h，定义于 voxel_map.cpp 将所有匹配到平面的不确定性点的属性及其平面属性 PointToPlane 都推入到 ptpl_list 中

1.3.2.5.1.7.1.6.11 1、获得关于最大层数 max_layer 2，体素尺寸 voxel_size 0.5 和 sigma_num 的配置

```
int max_layer = config_setting_.max_layer_;double voxel_size =
config_setting_.max_voxel_size_;double sigma_num =
config_setting_.sigma_num_;
```

1.3.2.5.1.7.1.6.12 2、点到平面状态列表清空

```
ptpl_list.clear();
```

1.3.2.5.1.7.1.6.13 3、声明三个容器：不确定点数来初始化 所有的点到平面状态列表 all_ptpl_list（成功的），有用的点到平面状态 useful_ptpl（点到平面状态是否成功标志容器），索引 index

```
std::vector<PointToPlane> all_ptpl_list(pv_list.size());std::vector<bool>
useful_ptpl(pv_list.size());std::vector<size_t> index(pv_list.size());
```

1.3.2.5.1.7.1.6.14 4、遍历点数给索引列表和有用点到平面列表初始化

```
for (size_t i = 0; i < index.size(); ++i)index[i] = i;useful_ptpl[i] = false;
```

1.3.2.5.1.7.1.6.15 5、多线程设置，根据 CPU 核数设置线程数：

```
omp_set_num_threads(MP_PROC_NUM);
```

在接下来的并行区域内，程序将使用 MP_PROC_NUM 个线程来执行任务

1.3.2.5.1.7.1.6.16 6、遍历索引向量

```
for (int i = 0; i < index.size(); i++)找到每个点能够匹配上的平面（此节点
或邻近节点都行），并将点及其对应平面的状态都保存到 all_ptpl_list
中
```

1.3.2.5.1.7.1.6.17 1、引用当前不确定性点

```
pointWithVar &pv = pv_list[i];
```

1.3.2.5.1.7.1.6.18 2、计算该点的（根节点）体素坐标：float loc_xyz[3]; position

```
float loc_xyz[3];for (int j = 0; j < 3; j++) {loc_xyz[j] = pv.point_w[j] /
voxel_size;if (loc_xyz[j] < 0) { loc_xyz[j] -= 1.0; } }VOXEL_LOCATION
position((int64_t)loc_xyz[0], (int64_t)loc_xyz[1], (int64_t)loc_xyz[2]);
```

1.3.2.5.1.7.1.6.19 3、根据该位置，利用哈希表找到其对应的八叉树根节点索引

```
auto iter = voxel_map_.find(position);
```

1.3.2.5.1.7.1.6.20 4、判断 是不是有效索引

```
if (iter != voxel_map_.end())获得这一帧所有不确定性点的点到平面状态
```

1.3.2.5.1.7.1.6.21 1、取出索引对应的八叉树节点

```
VoxelOctoTree *current_octo = iter->second;
```

1.3.2.5.1.7.1.6.22 2、声明一个单个的点到平面状态

```
PointToPlane single_ptpl;
```

1.3.2.5.1.7.1.6.23 3、声明两个量：点到平面匹配成功标志位 is_sucess; 概率 prob

```
bool is_sucess = false;double prob = 0;
```

1.3.2.5.1.7.1.6.24 4、build_single_residual(pv, current_octo, 0, is_sucess, prob, single_ptpl);

```
构建单个点到平面残差对应函数： void
VoxelMapManager::build_single_residual(pointWithVar &pv, const
```

VoxelOctoTree *current_octo, const int current_layer, bool
&is_sucess, double &prob, PointToPlane &single_ptpl) 计算点到平面的绝对距离，并计算该距离的期望和协方差，若点到平面距离够小则将匹配成功标志位设为 真 is_sucess = true;，并根据期望和协方差计算概率 prob，概率值满足后就将点和平面的属性都给 single_ptpl 可参考论文
《Efficient and Probabilistic Adaptive Voxel Mapping for Accurate Online LiDAR Odometry》

1.3.2.5.1.7.1.6.25 1、初始化最大层数 max_layer 2 和 sigma 数量 3

```
int max_layer = config_setting_.max_layer_; double sigma_num =  
config_setting_.sigma_num_;
```

1.3.2.5.1.7.1.6.26 2、设置一个残差倍数（用于后续判断）

```
double radius_k = 3;
```

1.3.2.5.1.7.1.6.27 3、获得不确定性点的世界坐标

```
Eigen::Vector3d p_w = pv.point_w;
```

1.3.2.5.1.7.1.6.28 4、判断当前节点的平面属性的平面标志位是否为真

```
if (current_octo->plane_ptr_->is_plane_) 计算点到平面的状态给  
single_ptpl
```

1.3.2.5.1.7.1.6.29 1、获得当前节点的平面属性

```
VoxelPlane &plane = *current_octo->plane_ptr_;
```

1.3.2.5.1.7.1.6.30 2、计算中心点到世界点的向量

```
Eigen::Vector3d p_world_to_center = p_w - plane.center_;
```

1.3.2.5.1.7.1.6.31 3、根据平面公式计算点到平面距离：

```
float dis_to_plane = fabs(plane.normal_(0) * p_w(0) + plane.normal_(1) *  
p_w(1) + plane.normal_(2) * p_w(2) + plane.d_);
```

1.3.2.5.1.7.1.6.32 4、求世界点到平面中心点向量的模长平方：dis_to_center

```
float dis_to_center = (plane.center_(0) - p_w(0)) * (plane.center_(0) - p_w(0))  
+ (plane.center_(1) - p_w(1)) * (plane.center_(1) - p_w(1)) +  
(plane.center_(2) - p_w(2)) * (plane.center_(2) - p_w(2));
```

1.3.2.5.1.7.1.6.33 5、世界点到平面中心点向量在平面上投影的模长：range_dis

```
float range_dis = sqrt(dis_to_center - dis_to_plane * dis_to_plane);
```

1.3.2.5.1.7.1.6.34 6、判断这个投影模长是否小于 3 倍的平面协方差的最大特征值：

$$d_i \sim \mathcal{N}(0, \Sigma_{\mathbf{w}_i}), \quad \Sigma_{\mathbf{w}_i} = \mathbf{J}_{\mathbf{w}_i} \Sigma_{\mathbf{n}_i, \mathbf{q}_i, \mathbf{w}_{\mathbf{p}_i}} \mathbf{J}_{\mathbf{w}_i}^T$$

if (range_dis <= radius_k * plane.radius_) 构建关于这个点到平面距离的期望和协方差，点到平面距离够小则将匹配成功标志位设为 真 is_sucess = true;，并根据期望和协方差计算概率，概率值满足后就将点和平面的属性都给 single_ptpl

1.3.2.5.1.7.1.6.35 1、将点到平面距离整理为期望加一阶项的形式：计算得到点到平面距离协方差（方差） σ_l 。

$$d_i = (\mathbf{n}_i^{gt} \text{ 田 } \delta_{\mathbf{n}_i})^T [({}^W\mathbf{p}_i^{gt} + \delta_{w_{\mathbf{p}_i}}) - \mathbf{q}_i^{gt} - \delta_{\mathbf{q}_i}]$$

$$\approx \underbrace{\mathbf{n}_i^{gtT} ({}^W\mathbf{p}_i^{gt} - \mathbf{q}_i^{gt})}_0 + \underbrace{\mathbf{J}_{\mathbf{n}_i} \delta_{\mathbf{n}_i} + \mathbf{J}_{\mathbf{q}_i} \delta_{\mathbf{q}_i} + \mathbf{J}_{w_{\mathbf{p}_i}} \delta_{w_{\mathbf{p}_i}}}_{\mathbf{w}_i}$$

1.3.2.5.1.7.1.6.36 1、声明一个 1*6 的雅可比矩阵：Eigen::Matrix<double, 1, 6> J_nq;

1.3.2.5.1.7.1.6.37 2、构建这个 J 阵（只构建了前两项）这里将平面不确定性与世界点不确定性分开实现 J。这里关于 J 的代码只将平面不确定性考虑在内。

$$\mathbf{J}_{\mathbf{w}_i} = [\mathbf{J}_{\mathbf{n}_i}, \mathbf{J}_{\mathbf{q}_i}, \mathbf{J}_{w_{\mathbf{p}_i}}] = [({}^W\mathbf{p}_i - \mathbf{q}_i)^T, -\mathbf{n}_i^T, \mathbf{n}_i^T]$$

J_nq.block<1, 3>(0, 0) = p_w - plane.center_; J_nq.block<1, 3>(0, 3) = -plane.normal_;

1.3.2.5.1.7.1.6.38 3、基于平面不确定性构建协方差第一项和第二项+点不确定性构建第三项=点到平面距离协方差

$$\Sigma_{\mathbf{w}_i} = \mathbf{J}_{\mathbf{w}_i} \Sigma_{\mathbf{n}_i, \mathbf{q}_i, w_{\mathbf{p}_i}} \mathbf{J}_{\mathbf{w}_i}^T$$

double sigma_l = J_nq * plane.plane_var_ * J_nq.transpose(); sigma_l += plane.normal_.transpose() * pv.var * plane.normal_;

1.3.2.5.1.7.1.6.39 2、判断 点到平面距离是否小于 3 倍 平方差：

if (dis_to_plane < sigma_num * sqrt(sigma_l)) 将匹配成功标志位设为 真 is_sucess = true;，并根据期望和协方差计算概率，概率值满足后就将点和平面的属性都给 single_ptpl

1.3.2.5.1.7.1.6.40 1、小于则说明 成功匹配上：is_sucess = true;

1.3.2.5.1.7.1.6.41 2、计算概率 this_prob（根据期望和协方差来计算概率）

double this_prob = 1.0 / (sqrt(sigma_l)) * exp(-0.5 * dis_to_plane * dis_to_plane / sigma_l);

1.3.2.5.1.7.1.6.42 3、判断这个节点的点平面匹配概率是否满足阈值：if (this_prob > prob)

将点和节点平面的相关属性都给 single_ptpl

1.3.2.5.1.7.1.6.43 1、平面与平面上的点共享法向量。

pv.normal = plane.normal_;

1.3.2.5.1.7.1.6.44 2、single_ptpl 相关属性赋值

1.3.2.5.1.7.1.6.45 1、将匹配点的载体不确定性 body_cov_、世界系协方差 point_b_、世界点 point_w_ 赋予给单个点到平面属性 single_ptpl。

single_ptpl.body_cov_ = pv.body_var; single_ptpl.point_b_ =

```
pv.point_b;single_ptpl.point_w_ = pv.point_w;
```

- 1.3.2.5.1.7.1.6.46 2、该节点平面不确定性 **plane_var_**、平面法向量 **normal_**、平面中心点 **center_**、平面方程参数 **d_**、当前层数 **current_layer**、点到平面距离 给单个点到平面属性 **single_ptpl** :

```
single_ptpl.plane_var_ = plane.plane_var_;single_ptpl.normal_ =  
plane.normal_;single_ptpl.center_ = plane.center_;single_ptpl.d_ =  
plane.d_;single_ptpl.layer_ = current_layer;single_ptpl.dis_to_plane_ =  
plane.normal_(0) * p_w(0) + plane.normal_(1) * p_w(1) + plane.normal_(2)  
* p_w(2) + plane.d_;
```

- 1.3.2.5.1.7.1.6.47 4、直接返回 **return**;

- 1.3.2.5.1.7.1.6.48 3、否则 直接返回 （匹配成功标志位默认为 False） **return**;

- 1.3.2.5.1.7.1.6.49 7、否则 直接返回 （匹配成功标志位默认为 False） **return**;

- 1.3.2.5.1.7.1.6.50 5、否则循环其子节点，并构建单个残差 （递归计算所有子节点的点到平面状态）

```
VoxelOctoTree *leaf_octo = current_octo-  
>leaves_[leafnum];build_single_residual(pv, leaf_octo, current_layer + 1,  
is_sucess, prob, single_ptpl);
```

- 1.3.2.5.1.7.1.6.51 5、判断 如果 该点到此节点及其子节点的平面都匹配不成功： **if (!is_sucess)**

该点在此节点及其子节点的平面都匹配不成功，则移动体素位置到附近体素再计算一次 **build_single_residual**。

- 1.3.2.5.1.7.1.6.52 1、计算该点的体素位置的附近体素位置

```
VOXEL_LOCATION near_position = position;
```

- 1.3.2.5.1.7.1.6.53 2、以四分之一体素边长为界限，将 **loc_xyz** 比较 体素中心加上或减去该界限的大小 来移动体素坐标，获得附件体素位置

```
if (loc_xyz[0] > (current_octo->voxel_center_[0] + current_octo-  
>quater_length_)) { near_position.x = near_position.x + 1; }else if  
(loc_xyz[0] < (current_octo->voxel_center_[0] - current_octo-  
>quater_length_)) { near_position.x = near_position.x - 1; }if (loc_xyz[1] >  
(current_octo->voxel_center_[1] + current_octo->quater_length_))  
{ near_position.y = near_position.y + 1; }else if (loc_xyz[1] < (current_octo-  
>voxel_center_[1] - current_octo->quater_length_)) { near_position.y =  
near_position.y - 1; }if (loc_xyz[2] > (current_octo->voxel_center_[2] +  
current_octo->quater_length_)) { near_position.z = near_position.z + 1; }else  
if (loc_xyz[2] < (current_octo->voxel_center_[2] - current_octo-  
>quater_length_)) { near_position.z = near_position.z - 1; }
```

- 1.3.2.5.1.7.1.6.54 3、只要该邻近体素位置有节点，就再计算一次（仅一次） **build_single_residual**

```
auto iter_near = voxel_map_.find(near_position);if (iter_near !=  
voxel_map_.end()) { build_single_residual(pv, iter_near->second, 0,  
is_sucess, prob, single_ptpl); }
```

1.3.2.5.1.7.1.6.55 6、判断，是否匹配到平面。if (is_sucsess)。将这一对匹配的点和平面属性存入到点平面列表中。并设该点为有效点到平面的标志为真。

```
mylock.lock();useful_ptpl[i] = true;all_ptpl_list[i] =
single_ptpl;mylock.unlock();
```

1.3.2.5.1.7.1.6.56 7、否则 该点为有效点到平面的标志 为假

```
mylock.lock();useful_ptpl[i] = false;mylock.unlock();
```

1.3.2.5.1.7.1.6.57 7、按照有效点平面容器记录的标志，将匹配到平面的不确定性点状态及其到该平面的状态都推入到 **ptpl_list** 中

```
for (size_t i = 0; i < useful_ptpl.size(); i++) {if (useful_ptpl[i])
{ ptpl_list.push_back(all_ptpl_list[i]); } }
```

1.3.2.5.1.7.1.6.58 7、累加每个点到平面的距离，做总距离 **total_residual**

```
for (int i = 0; i < ptpl_list.size(); i++){total_residual +=
fabs(ptpl_list[i].dis_to_plane_);}
```

1.3.2.5.1.7.1.6.59 8、获得有效点数量（匹配平面成功的 点数）

```
effct_feat_num_ = ptpl_list.size();
```

1.3.2.5.1.7.1.6.60 打印计算完残差后的信息

```
cout << "[ LIO ] Raw feature num: " << feats_undistort_->size() << ",
downsampled feature num:" << feats_down_size_ << " effective feature num:
" << effct_feat_num_ << " average residual: " << total_residual /
effct_feat_num_ << endl;
```

1.3.2.5.1.7.1.6.61 9、关于观测的雅可比阵 **H** 和测量协方差 **R**（已经有匹配的点和平面了）

$$\begin{aligned} \text{残差: } \mathbf{y} &= \mathbf{z} - h(\bar{\mathbf{x}}) \\ \text{测量矩阵: } \mathbf{H} &= \left. \frac{\partial h(\bar{\mathbf{x}}_t)}{\partial \bar{\mathbf{x}}} \right|_{\bar{\mathbf{x}}_t} \\ \text{增益: } \mathbf{K} &= \bar{\mathbf{P}} \mathbf{H}^T \left(\mathbf{H} \bar{\mathbf{P}} \mathbf{H}^T + \mathbf{R} \right)^{-1} \end{aligned}$$

$$\text{状态更新: } \mathbf{x} = \bar{\mathbf{x}} + \mathbf{K} \mathbf{y}$$

$$\text{协方差更新: } \mathbf{P} = (\mathbf{I} - \mathbf{K} \mathbf{H}) \bar{\mathbf{P}}$$

重点是前两行，求得残差，然后根据观测方程对状态误差求导得到 **H** 阵。获得每个点对应观测方程的协方差 **R**（实际上求得是 **R** 的逆 **R_inv**）、雅可比矩阵 **Hsub**，以及测量残差（ $z-h(x,v)=0$ - 点到平面距离）**meas_vec(i)**，便于 **K** 计算的 $\mathbf{H}^T * \mathbf{R}^{-1} \mathbf{Hsub_T_R_inv}$

1.3.2.5.1.7.1.6.62 1、初始化一些矩阵用于后续更新 **Hsub** 测量雅可比矩阵、**Hsub_T_R_inv**、**R_inv** 观测、**meas_vec**

$$\mathbf{K} = (\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} + \mathbf{P}^{-1})^{-1} \mathbf{H}^T \mathbf{R}^{-1}$$

```
MatrixXd Hsub(effct_feat_num_, 6); // 观测方程对误差状态的雅可比阵
MatrixXd Hsub_T_R_inv(6, effct_feat_num_); // ??? VectorXd
R_inv(effct_feat_num_); // ??? VectorXd meas_vec(effct_feat_num_);
```

// 测量向量 meas_vec.setZero();

1.3.2.5.1.7.1.6.63 观测方程雅可比（这里的 R 和 EKF 中的 R 不是一回事，这里是旋转矩阵，EKF 中的 R 是观测方程（求点到平面距离）协方差）

$$\begin{aligned}\mathbf{res}_i &= \mathbf{h}(\hat{\mathbf{x}})_i - \mathbf{z}_i \\ &= \mathbf{n}_i^T \cdot (\hat{\mathbf{R}}\mathbf{p}_i + \hat{\mathbf{p}} - \mathbf{q}_i) \\ \mathbf{H}(\tilde{\mathbf{R}})_i &= -\mathbf{n}_i^T \cdot \hat{\mathbf{R}} \cdot [\mathbf{p}_i] \\ &= ([\mathbf{p}_i] \cdot \hat{\mathbf{R}}^T \cdot \mathbf{n}_i)^T \\ \mathbf{H}(\tilde{\mathbf{p}})_i &= \mathbf{n}_i^T\end{aligned}$$

1.3.2.5.1.7.1.6.64 2、遍历所有有效点：for (int i = 0; i < effect_feat_num_; i++)

计算出每个点对应观测方程的协方差 **R**（实际上求得是 **R** 的逆 **R_inv**）、雅可比矩阵 **Hsub**，以及测量残差（ $z-h(x,v)=0$ - 点到平面距离）**meas_vec(i)**，便于 K 计算的 $H^T * R^{-1} Hsub_T_R_inv$

1.3.2.5.1.7.1.6.65 1、计算该有效点的载体点 **point_body** 和 IMU 系下点 **point_this**，并计算 IMU 系点的反对称阵 **point_crossmat**

```
auto &ptpl = ptpl_list[i]; V3D point_this(ptpl.point_b_); point_this =
extR_ * point_this + extT_; V3D point_body(ptpl.point_b_); M3D
point_crossmat; point_crossmat << SKEW_SYM_MATRX(point_this);
```

1.3.2.5.1.7.1.6.66 2、获得世界系点：V3D **point_world** = state_propagat.rot_end * point_this + state_propagat.pos_end;

1.3.2.5.1.7.1.6.67 点到平面距离的分布---供解析参考用

$$d_i \sim \mathcal{N}(0, \Sigma_{\mathbf{w}_i}), \quad \Sigma_{\mathbf{w}_i} = \mathbf{J}_{\mathbf{w}_i} \Sigma_{\mathbf{n}_i, \mathbf{q}_i, \mathbf{w}_{\mathbf{p}_i}} \mathbf{J}_{\mathbf{w}_i}^T$$

1.3.2.5.1.7.1.6.68 第一项是平面协方差，下面是世界点协方差。---供解析参考用

$$\Sigma_{\mathbf{n}_i, \mathbf{q}_i, \mathbf{w}_{\mathbf{p}_i}} = \begin{bmatrix} \Sigma_{\mathbf{n}_i, \mathbf{q}_i} & 0 \\ 0 & \Sigma_{\mathbf{w}_{\mathbf{p}_i}} \end{bmatrix}$$

1.3.2.5.1.7.1.6.69 3、获得点到平面距离关于平面的雅可比阵，**J_nq** 在计算残差 BuildResidualListOMP 中也会计算这个。

$$\mathbf{J}_{\mathbf{w}_i} = [\mathbf{J}_{\mathbf{n}_i}, \mathbf{J}_{\mathbf{q}_i}, \mathbf{J}_{\mathbf{w}_{\mathbf{p}_i}}] = [(\mathbf{w}_{\mathbf{p}_i} - \mathbf{q}_i)^T, -\mathbf{n}_i^T, \mathbf{n}_i^T]$$

Eigen::Matrix<double, 1, 6> J_nq; J_nq.block<1, 3>(0, 0) = point_world - ptpl_list[i].center_; J_nq.block<1, 3>(0, 3) = -ptpl_list[i].normal_;

1.3.2.5.1.7.1.6.70 4、声明一个不确定性（协方差）世界点的不确定性（但只有第一项，并未严格遵循论文）：
`var = state_propagat.rot_end * extR_ * ptpl_list[i].body_cov_ * (state_propagat.rot_end * extR_).transpose();`

$$\Sigma_{w_{p_i}} = \boxed{{}^W_L \mathbf{R} \Sigma_{L_{p_i}} {}^W_L \mathbf{R}^T} + {}^W_L \mathbf{R} [{}^L_{p_i} \mathbf{p}_i]_{\wedge} \Sigma_{\mathbf{R}} [{}^L_{p_i} \mathbf{p}_i]_{\wedge}^T {}^W_L \mathbf{R}^T + \Sigma_t \quad (3)$$

1.3.2.5.1.7.1.6.71 5、获得点到平面距离协方差（分开求得）就是观测方程的协方差 R

$$\Sigma_{w_i} = \mathbf{J}_{w_i} \Sigma_{n_i, q_i, w_{p_i}} \mathbf{J}_{w_i}^T$$

注意，这里的 R 与论文《Efficient and Probabilistic Adaptive Voxel Mapping for Accurate Online LiDAR Odometry》也不完全一致，论文从雷达系点考虑的，这里是从世界点考虑的，中间有一个外参

1.3.2.5.1.7.1.6.72 5、获得点到平面距离协方差中的平面项

`double sigma_l = J_nq * ptpl_list[i].plane_var_ * J_nq.transpose();`

1.3.2.5.1.7.1.6.73 6、获得点到平面距离协方差中的世界点项：

`sigma_l + ptpl_list[i].normal_.transpose() * var * ptpl_list[i].normal_`

1.3.2.5.1.7.1.6.74 6、获得 EKF 中观测方程协方差的逆 $\mathbf{R}_{inv(i)}$ 。与标准 EKF 不同，为了避免计算大矩阵的逆，火星实验室使用下式进行计算，与标准 EKF 中的增益是相等的。

`R_inv(i) = 1.0 / (0.001 + sigma_l + ptpl_list[i].normal_.transpose() * var * ptpl_list[i].normal_);`

1.3.2.5.1.7.1.6.75 标准 EKF 增益

$$\mathbf{K} = \mathbf{P} \mathbf{H}^T (\mathbf{H} \mathbf{P} \mathbf{H}^T + \mathbf{R})^{-1}$$

1.3.2.5.1.7.1.6.76 便于计算的增益

$$\mathbf{K} = (\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} + \mathbf{P}^{-1})^{-1} \mathbf{H}^T \mathbf{R}^{-1}$$

1.3.2.5.1.7.1.6.77 7、计算测量方程的雅可比阵 H `Hsub.row(i)`

`Hsub.row(i) << VEC_FROM_ARRAY(A), ptpl_list[i].normal_[0], ptpl_list[i].normal_[1], ptpl_list[i].normal_[2];`

1.3.2.5.1.7.1.6.78 1、计算关于旋转误差的雅可比阵

$$\begin{aligned} \mathbf{H}(\tilde{\mathbf{R}})_i &= -\mathbf{n}_i^T \cdot \hat{\mathbf{R}} \cdot [\mathbf{p}_i] \\ &= ([\mathbf{p}_i] \cdot \hat{\mathbf{R}}^T \cdot \mathbf{n}_i)^T \end{aligned}$$

`V3D A(point_crossmat * state_.rot_end.transpose() * ptpl_list[i].normal_);`

1.3.2.5.1.7.1.6.79 2、关于平移误差的雅可比阵

$$\mathbf{H}(\tilde{\mathbf{p}})_i = \mathbf{n}_i^T$$

ptpl_list[i].normal_[0], ptpl_list[i].normal_[1], ptpl_list[i].normal_[2]

1.3.2.5.1.7.1.6.80 8、计算计算增益中的 $\mathbf{H}^T \mathbf{R}^{-1}$ Hsub_T_R_inv

$$\mathbf{H}^T \mathbf{R}^{-1}$$

Hsub_T_R_inv.col(i) << A[0] * R_inv(i), A[1] * R_inv(i), A[2] * R_inv(i),
ptpl_list[i].normal_[0] * R_inv(i), ptpl_list[i].normal_[1] * R_inv(i),
ptpl_list[i].normal_[2] * R_inv(i);

1.3.2.5.1.7.1.6.81 9、求观测方程的残差 添加到测量向量中 meas_vec(i) (观测量 (0) 与估计量 (点到平面距离) 的差)

meas_vec(i) = -ptpl_list[i].dis_to_plane;

1.3.2.5.1.7.1.6.82 便于计算而重整的增益 K --- 供解析参考用

$$\mathbf{K} = (\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} + \mathbf{P}^{-1})^{-1} \mathbf{H}^T \mathbf{R}^{-1}$$

1.3.2.5.1.7.1.6.83 R3LIVE 代码中实际的增益 K --- 供解析参考用

$$\mathbf{K} = (\mathbf{H}^T \mathbf{H} + (\mathbf{P}/\mathbf{R})^{-1})^{-1} \mathbf{H}^T$$

1.3.2.5.1.7.1.6.84 10、EKF 停止标志位 EKF_stop_flg 和收敛标志位 flg_EKF_converged 设为假 False

EKF_stop_flg = false; flg_EKF_converged = false;

1.3.2.5.1.7.1.6.85 11、进行状态更新

$$\mathbf{K} = \left((\mathbf{H}^\kappa)^T \mathbf{R}^{-1} \mathbf{H}^\kappa + \hat{\mathbf{P}}^{-1} \right)^{-1} (\mathbf{H}^\kappa)^T \mathbf{R}^{-1}$$

$$\hat{\mathbf{x}}^{\kappa+1} = \hat{\mathbf{x}}^\kappa + (-\mathbf{K} \mathbf{z}^\kappa - (\mathbf{I} - \mathbf{K} \mathbf{H}^\kappa) (\hat{\mathbf{x}}^\kappa - \hat{\mathbf{x}})).$$

根据上面获得的 H、R、z 对状态 state_ 进行更新，并根据旋转和平移增量的大小判断是否收敛 flg_EKF_converged，记录当前状态欧拉角。

1.3.2.5.1.7.1.6.86 1、生成一个所有点的增益 K 矩阵 (每一个点的 K 构成一个大 K) : MatrixXd

K(DIM_STATE, effct_feat_num_);

1.3.2.5.1.7.1.6.87 2、将 Hsub_T_R_inv 乘以 测量 残差: auto &&HTz = Hsub_T_R_inv * meas_vec; 相当于下式乘以测量残差 z。

$$\mathbf{H}^T \mathbf{R}^{-1}$$

1.3.2.5.1.7.1.6.88 3、计算 $\mathbf{H}^T \mathbf{H} \cdot \text{block}(6, 6) > (0, 0) = \mathbf{H}_{\text{sub_T_R_inv}} * \mathbf{H}_{\text{sub}}$; (这里的形式明明是 $\mathbf{H}^T \mathbf{H}$)

$$\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H}$$

1.3.2.5.1.7.1.6.89 4、计算 $\mathbf{K}_1$

$$\mathbf{K} = (\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} + \mathbf{P}^{-1})^{-1} \mathbf{H}^T \mathbf{R}^{-1}$$

$(\mathbf{H_T_H} \cdot \text{block}(\text{DIM_STATE}, \text{DIM_STATE}) > (0, 0) + \text{state_cov} \cdot \text{block}(\text{DIM_STATE}, \text{DIM_STATE}) > (0, 0) \cdot \text{inverse}()) \cdot \text{inverse}();$

1.3.2.5.1.7.1.6.90 5、计算协方差更新项 $\mathbf{K}\mathbf{H}$: $\mathbf{G} \cdot \text{block}(\text{DIM_STATE}, 6) > (0, 0)$

$$\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H}) \bar{\mathbf{P}}$$

$\mathbf{G} \cdot \text{block}(\text{DIM_STATE}, 6) > (0, 0) = \mathbf{K_1} \cdot \text{block}(\text{DIM_STATE}, 6) > (0, 0) * \mathbf{H_T_H} \cdot \text{block}(6, 6) > (0, 0);$

1.3.2.5.1.7.1.6.91 6、计算传播的先验状态和当前迭代状态的差（盒式减）: `auto vec = state_propagat - state_;`

$$\hat{\mathbf{x}}^{\kappa+1} = \hat{\mathbf{x}}^{\kappa} \boxplus (-\mathbf{K}\mathbf{z}^{\kappa} - (\mathbf{I} - \mathbf{K}\mathbf{H}^{\kappa}) (\hat{\mathbf{x}}^{\kappa} \boxminus \hat{\mathbf{x}})).$$

1.3.2.5.1.7.1.6.92 7、计算增量: $\mathbf{K_1} \cdot \text{block}(\text{DIM_STATE}, 6) > (0, 0) * \mathbf{H}\mathbf{Tz}$ 对应红框,
`vec.block(DIM_STATE, 1) > (0, 0) - G.block(DIM_STATE, 6) > (0, 0) * vec.block(6, 1) > (0, 0);` 对应蓝框展开

$$\hat{\mathbf{x}}^{\kappa+1} = \hat{\mathbf{x}}^{\kappa} \boxplus (-\mathbf{K}\mathbf{z}^{\kappa} - (\mathbf{I} - \mathbf{K}\mathbf{H}^{\kappa}) (\hat{\mathbf{x}}^{\kappa} \boxminus \hat{\mathbf{x}})).$$

`solution = K_1.block(DIM_STATE, 6) > (0, 0) * HTz + vec.block(DIM_STATE, 1) > (0, 0) - G.block(DIM_STATE, 6) > (0, 0) * vec.block(6, 1) > (0, 0);`

1.3.2.5.1.7.1.6.93 8、状态更新: `state_ += solution;`

$$\hat{\mathbf{x}}^{\kappa+1}$$

1.3.2.5.1.7.1.6.94 9、旋转增量 `rot_add` 和平移增量 `t_add`

`auto rot_add = solution.block(3, 1) > (0, 0); auto t_add = solution.block(3, 1) > (3, 0);`

1.3.2.5.1.7.1.6.95 10、如果旋转增量的角度小于 0.01 度，或者平移增量的距离小于 0.015 厘米，则认为该 EKF 收敛了: `if ((rot_add.norm() * 57.3 < 0.01) && (t_add.norm() * 100 < 0.015))`

`flg_EKF_converged = true;`

1.3.2.5.1.7.1.6.96 11、记录下当前状态的欧拉角：V3D `euler_cur = state_rot_end.eulerAngles(2, 1, 0);`

1.3.2.5.1.7.1.6.97 12、重匹配判定：只要 EKF 收敛位为真 或 重匹配数为 0 且此时迭代计数为最大迭代数倒数第二了。重匹配数加 1。也就是说只有收敛了，且已经是迭代到最大迭代数的倒数第二了（没到达该数之前都得继续迭代），`rematch_num` 才会是 2，后验协方差更新才会满足条件。

```
if (flg_EKF_converged || ((rematch_num == 0) && (iterCount ==
(config_setting_.max_iterations_ - 2)))) { rematch_num++; }
```

1.3.2.5.1.7.1.6.98 13、收敛判定 以及协方差更新：若 EKF 停止位为假，且，达到最大迭代此时

```
if (!EKF_stop_flg && (rematch_num >= 2 || (iterCount ==
config_setting_.max_iterations_ - 1)))。
```

1.3.2.5.1.7.1.6.99 1、更新状态协方差 `state_cov`

$$\mathbf{P} = (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}$$

```
state_cov.block<DIM_STATE, DIM_STATE>(0, 0) =
(I_STATE.block<DIM_STATE, DIM_STATE>(0, 0) -
G.block<DIM_STATE, DIM_STATE>(0, 0)) *
state_cov.block<DIM_STATE, DIM_STATE>(0, 0);
```

1.3.2.5.1.7.1.6.100 2、最新的位置 `position_last_ = state_pos_end;`

1.3.2.5.1.7.1.6.101 3、旋转的四元数

```
geoQuat_ = tf::createQuaternionMsgFromRollPitchYaw(euler_cur(0),
euler_cur(1), euler_cur(2));
```

1.3.2.5.1.7.1.6.102 4、停止标志位设为真

```
EKF_stop_flg = true;
```

1.3.2.5.1.7.1.6.103 14、如果停止标志位设为真 则推出迭代循环 `if (EKF_stop_flg) break;`

1.3.2.5.1.7.2 8、获得估计后的状态 `voxelmap_manager->state_;` 以及 基于后验状态更新的不确定性点的列表 `voxelmap_manager->pv_list_;`

```
_state = voxelmap_manager->state_; _pv_list = voxelmap_manager->pv_list_;
```

1.3.2.5.1.8 记录时间 `double t2 = omp_get_wtime();`

1.3.2.5.1.9 5、根据估计的状态决定是否高频发布，评估，里程计发布

1.3.2.5.1.9.1 9、判断 以 imu 频率传播使能（一般 false）：`if (imu_prop_enable)`。涉及 `imu_prop_callback`。---可略过

```
ekf_finish_once = true; // ekf 完成一次 设为 真 latest_ekf_state = _state; // 获取
最新的 ekf 状态 latest_ekf_time = LidarMeasures.last_lidar_update_time; // 最新的
ekf 时间 state_update_flg = true; // 状态更新标志为真
```

1.3.2.5.1.9.2 10、判断 是否 输出位姿（evo 评估使用）if (pose_output_en)

输出时刻和状态到 evoFile 中

1.3.2.5.1.9.2.1 1、位姿文件打开标志位 静态 pos_ope

1.3.2.5.1.9.2.2 2*、静态，计数器（未使用） ocount

1.3.2.5.1.9.2.3 3、文件输出函数：std::ofstream outFile（未使用），evoFile;

1.3.2.5.1.9.2.4 4、判断 pos_ope 是否为真 if(!pos_ope)，不管是 if 或 else 都会打开对应文件，区别是会改变 pos_ope 的标志位。如果文件不存在，会自动创建；如果文件已存在，追加模式会将新内容写入文件末尾，而不会覆盖原有内容。

```
evoFile.open(std::string(ROOT_DIR) + "Log/result/" + seq_name + ".txt",
std::ios::out);。evoFile.open(std::string(ROOT_DIR) + "Log/result/" + seq_name
+ ".txt", std::ios::app);
```

1.3.2.5.1.9.2.5 5、变换矩阵（未使用）

```
Eigen::Matrix4d outT;
```

1.3.2.5.1.9.2.6 6、旋转状态的四元数：Eigen::Quaterniond q(_state.rot_end);

1.3.2.5.1.9.2.7 7、设置以固定格式输出：evoFile << std::fixed;

应用 std::fixed 后，浮点数将以固定小数点格式输出，而不是科学计数法（如 1.23e+03）

1.3.2.5.1.9.2.8 8、输出 上次更新时间（此次图像捕获时间）以及状态

```
evoFile << LidarMeasures.last_lid_update_time << " " << _state.pos_end[0]
<< " " << _state.pos_end[1] << " " << _state.pos_end[2] << " " << q.x() << " " <<
q.y() << " " << q.z() << " " << q.w() << std::endl;
```

1.3.2.5.1.9.3 11、将状态估计后的旋转转为欧拉角 euler_cur 再转为四元数 geoQuat

```
euler_cur = RotMtoEuler(_state.rot_end); geoQuat =
tf::createQuaternionMsgFromRollPitchYaw(euler_cur(0), euler_cur(1), euler_cur(2));
```

1.3.2.5.1.9.4 12、publish_odometry(pubOdomAftMapped); 发布后验状态的旋转和平移

调用函数发布里程计：void LIVMapper::publish_odometry(const ros::Publisher &pubOdomAftMapped)

1.3.2.5.1.10 记录时间 double t3 = omp_get_wtime();

1.3.2.5.1.11 6、根据后验状态对点的不确定性列表进行更新、体素地图（平面属性）更新、复制列表给_pv_list

1.3.2.5.1.11.1 13、声明一个世界激光雷达点云指针 并按 状态估计后的状态将去畸变的体素点云转到世界系下。

```
PointCloudXYZI::Ptr world_lidar(new
PointCloudXYZI());transformLidar(_state.rot_end, _state.pos_end, feats_down_body,
world_lidar);
```

1.3.2.5.1.11.2 14、遍历世界系下的当前帧点获得更新完不确定性点的列表

for (size_t i = 0; i < world_lidar->points.size(); i++)，注意，这里的代码实现与论文 Efficient and Probabilistic Adaptive Voxel Mapping for Accurate Online LiDAR Odometry 并不严格一致。

1.3.2.5.1.11.2.1 1、获得体素管理器中不确定性点列的世界坐标

```
point_w : voxelmap_manager->pv_list_[i].point_w << world_lidar->points[i].x,
world_lidar->points[i].y, world_lidar->points[i].z;列表里存的是一个结构体。
std::vector<pointWithVar> pv_list_; typedef struct pointWithVar
```

1.3.2.5.1.11.2.2 2、取出体素管理器中该点对应的反对称形式

```
M3D point_crossmat = voxelmap_manager->cross_mat_list_[i];
```

1.3.2.5.1.11.2.3 3、取出体素管理器中载体系下该点的协方差阵

```
M3D var = voxelmap_manager->body_cov_list_[i];
```

1.3.2.5.1.11.2.4 4、更新不确定性到世界系下

```
var = (_state.rot_end * extR) * var * (_state.rot_end * extR).transpose() + (-
point_crossmat) * _state.cov.block<3, 3>(0, 0) * (-point_crossmat).transpose() +
_state.cov.block<3, 3>(3, 3);这部分可以参考论文 Efficient and Probabilistic
Adaptive Voxel Mapping for Accurate Online LiDAR Odometry。但注意，与
论文并不严格一致。
```

1.3.2.5.1.11.2.5 5、更新到体素管理器的不确定点列表中

```
voxelmap_manager->pv_list_[i].var = var;
```

1.3.2.5.1.11.3 15、voxelmap_manager->UpdateVoxelMap(voxelmap_manager->pv_list_);

根据该列表更新体素地图：输入的是不确定性点列表 void VoxelMapManager::UpdateVoxelMap(const std::vector<pointWithVar> &input_points) 定义于 voxel_map.cpp 根据新来的点，对该点对应的节点平面属性进行更新)

1.3.2.5.1.11.3.1 1、获得该体素图的设置：体素大小，平面阈值，最大层数，最大点数，层初始化点数下限。

```
float voxel_size = config_setting_.max_voxel_size_;float planer_threshold =
config_setting_.planner_threshold_;int max_layer =
config_setting_.max_layer_;int max_points_num =
config_setting_.max_points_num_;std::vector<int> layer_init_num =
```

```
config_setting_layer_init_num_;
```

1.3.2.5.1.11.3.2 2、遍历每个不确定点

```
uint plsize = input_points.size();for (uint i = 0; i < plsize; i++) (根据当前帧更新的不确定性点，对该点对应的节点平面属性进行更新)
```

1.3.2.5.1.11.3.2.1 1、引用不确定性点：const pointWithVar p_v = input_points[i];

1.3.2.5.1.11.3.2.2 2、计算体素坐标

```
float loc_xyz[3];for (int j = 0; j < 3; j++) {loc_xyz[j] = p_v.point_w[j] / voxel_size;if (loc_xyz[j] < 0) { loc_xyz[j] -= 1.0; } }
```

1.3.2.5.1.11.3.2.3 3、查询该体素对应的根节点

```
VOXEL_LOCATION position((int64_t)loc_xyz[0], (int64_t)loc_xyz[1], (int64_t)loc_xyz[2]);auto iter = voxel_map_.find(position);
```

1.3.2.5.1.11.3.2.4 4、判断，有对应节点，更新该点到八叉树中（根据新来的点，对节点平面属性进行更新）

```
if (iter != voxel_map_.end()) {voxel_map_[position]->UpdateOctoTree(p_v)}
```

1.3.2.5.1.11.3.2.5 1、判断节点是否初始化了（一般是新的节点），将点推入到节点的临时点列表，当点足够时（>5）进行八叉树初始化

```
new_points_++;temp_points_.push_back(pv);if (temp_points_.size() > points_size_threshold_) { init_octo_tree(); }
```

1.3.2.5.1.11.3.2.6 2、否则就是已有节点

若平面属性为真，则根据更新使能与否对平面进行初始化。若为假，则去子叶查找（包括生成新叶等操作），再根据更新使能与否进行平面初始化。也有可简化的地方（作用有限）：在去子叶查找的那个地方，可以直接判断已有节点的子叶是否更新使能与否，然后在进行八叉树更新。

1.3.2.5.1.11.3.2.7 1、判断该节点的平面属性是否为平面后再判断是否更新使能（只要点不超过最大点）

```
if (plane_ptr_->is_plane_) {if (update_enable_)} 推入新点，用于对该节点进行平面初始化。此外，若点数过多，清空临时点，update_enable_为false。（该节点不再进行后续更新）
```

1.3.2.5.1.11.3.2.8 1、新点计数累加：new_points_++;

1.3.2.5.1.11.3.2.9 2、将不确定性点推入临时点列表：

```
temp_points_.push_back(pv);
```

1.3.2.5.1.11.3.2.103、判断 如果点数超过了更新阈值（5），则对该节点进行平面初始化，新点数设为零：

```
if(new_points_ > update_size_threshold_)init_plane(temp_points_,
plane_ptr_);    new_points_ = 0;
```

1.3.2.5.1.11.3.2.114、判断 临时点点数是否超过最大点数（50），则更新使能设为 false，清空临时点：. 不管这个节点有没有平面，只有点数过多就清空临时点。

```
if(temp_points_.size() >= max_points_num_)update_enable_ =
false;std::vector<pointWithVar>().swap(temp_points_);new_points_ = 0;
```

1.3.2.5.1.11.3.2.122、否则该节点平面属性不是平面，就去子叶查找：else{ }

对于非终止叶，可以节点判断后八叉树更新或再分子叶，判断节点继续八叉树更新。对于终止叶，直接推入该点，平面初始化，点数超了就清空临时点，关闭更新使能。

1.3.2.5.1.11.3.2.131、如果该节点不是最后一层

```
if(layer_ < max_layer_)，若是子叶有节点，进行八叉树更新。若是无节点，生成节点，再八叉树更新
```

1.3.2.5.1.11.3.2.141、计算所在的子叶位置

```
int xyz[3] = {0, 0, 0};if (pv.point_w[0] > voxel_center_[0]) { xyz[0] = 1; }if
(pv.point_w[1] > voxel_center_[1]) { xyz[1] = 1; }if (pv.point_w[2] >
voxel_center_[2]) { xyz[2] = 1; }int leafnum = 4 * xyz[0] + 2 * xyz[1] +
xyz[2];
```

1.3.2.5.1.11.3.2.152、判断 该子叶是否有对应节点，有则执行更新八叉树函数

```
if (leaves_[leafnum] != nullptr) { leaves_[leafnum]->UpdateOctoTree(pv); }
```

1.3.2.5.1.11.3.2.163、否则就生成新的节点到该子叶再进行八叉树更新。

```
leaves_[leafnum]->UpdateOctoTree(pv);
```

1.3.2.5.1.11.3.2.172、否则 就是最后一个节点，判断更新使能是否为真：else{if (update_enable_)}

推入新点，用于对该节点进行平面初始化。此外，若点数过多，清空临时点，update_enable_为 false.（该节点不再进行后续更新）

1.3.2.5.1.11.3.2.181、新点计数累加：new_points_++;

1.3.2.5.1.11.3.2.192、将不确定性点推入临时点列表：temp_points_.push_back(pv);

1.3.2.5.1.11.3.2.203、判断 如果点数超过了更新阈值（5），则对该节点进行平面初始化，新点数设为零

```
if(new_points_ > update_size_threshold_)init_plane(temp_points_,
plane_ptr_);    new_points_ = 0;
```

1.3.2.5.1.11.3.2.214、判断临时点点数是否超过最大点数（50），则更新使能设为 false，清空临时点：if (temp_points.size() >= max_points_num). 不管这个节点有没有平面，只有点数过多就清空临时点。

```
update_enable_ =  
false;std::vector<pointWithVar>().swap(temp_points_);new_points_ = 0;
```

1.3.2.5.1.11.3.2.225、否则，生成一个新的节点 然后再更新该点到八叉树中

```
VoxelOctoTree *octo_tree = new VoxelOctoTree(max_layer, 0,  
layer_init_num[0], max_points_num, planer_threshold);voxel_map_[position]  
= octo_tree;voxel_map_[position]->layer_init_num_ =  
layer_init_num;voxel_map_[position]->quater_length_ = voxel_size /  
4;voxel_map_[position]->voxel_center_[0] = (0.5 + position.x) *  
voxel_size;voxel_map_[position]->voxel_center_[1] = (0.5 + position.y) *  
voxel_size;voxel_map_[position]->voxel_center_[2] = (0.5 + position.z) *  
voxel_size;voxel_map_[position]->UpdateOctoTree(p_v);
```

1.3.2.5.1.11.4 16、将更新后的不确定性点与 vio 管理器进行共享。

```
_pv_list = voxelmap_manager->pv_list_;
```

1.3.2.5.1.12 记录时间：double t4 = omp_get_wtime();

1.3.2.5.1.13 7、当前帧转到世界系并准备发布：pcl_w_wait_pub

1.3.2.5.1.13.1 17、判断体素管理器的地图滑动使能（清除超过某一范围内的所有体素及表索引）

```
if(voxelmap_manager->config_setting_.map_sliding_en) voxelmap_manager->  
mapSliding();般不使用（作用是防止地图太大）
```

1.3.2.5.1.13.1.1 1、判断最近状态的位置与上次滑动位置的距离是否够大 8m：

```
if((position_last_ - last_slide_position).norm() <  
config_setting_.sliding_thresh), 不够大则直接返回
```

```
std::cout<<RED<<"[DEBUG]: Last sliding length "<<(position_last_ -  
last_slide_position).norm()<<RESET<<"\n";return;
```

1.3.2.5.1.13.1.2 2、若是超出了滑动阈值

1.3.2.5.1.13.1.2.1 1、将此时位置作为上次滑动位置：last_slide_position = position_last_;

1.3.2.5.1.13.1.2.2 记录时间：double t_sliding_start = omp_get_wtime();

1.3.2.5.1.13.1.2.3 2、计算此处位置的体素坐标：float loc_xyz[3];

```
for (int j = 0; j < 3; j++) {loc_xyz[j] = position_last_[j] /  
config_setting_.max_voxel_size_;if (loc_xyz[j] < 0) { loc_xyz[j] -= 1.0; } }
```

1.3.2.5.1.13.1.2.4 3、根据该体素坐标清空超出范围的存储：clearMemOutOfMap

```
clearMemOutOfMap((int64_t)loc_xyz[0] + config_setting_.half_map_size,  
(int64_t)loc_xyz[0] - config_setting_.half_map_size,  
(int64_t)loc_xyz[1] + config_setting_.half_map_size, (int64_t)loc_xyz[1] -
```



```

config_setting_half_map_size, (int64_t)loc_xyz[2] +
config_setting_half_map_size, (int64_t)loc_xyz[2] -
config_setting_half_map_size);void
VoxelMapManager::clearMemOutOfMap(const int& x_max,const int&
x_min,const int& y_max,const int& y_min,const int& z_max,const int&
z_min )

```

1.3.2.5.1.13.1.2.5 1、声明删除的体素计数: int delete_voxel_cout = 0;

1.3.2.5.1.13.1.2.6 2、循环整个体素哈希表 for (auto it = voxel_map_.begin(); it != voxel_map_.end();)

1.3.2.5.1.13.1.2.7 1、取出体素位置并判断其是否应该被移除 should_remove

```

const VOXEL_LOCATION& loc = it->first;bool should_remove = loc.x >
x_max || loc.x < x_min || loc.y > y_max || loc.y < y_min || loc.z > z_max ||
loc.z < z_min;

```

1.3.2.5.1.13.1.2.8 2、若应该被移除 if (should_remove)

1.3.2.5.1.13.1.2.9 1、删除迭代器对应的八叉树 delete it->second;

1.3.2.5.1.13.1.2.102、哈希表擦除该迭代器 it = voxel_map_.erase(it);

1.3.2.5.1.13.1.2.113、删除计数自加 delete_voxel_cout++;

1.3.2.5.1.13.1.2.123、否则迭代器自加 (继续循环)

```

else {++it; }

```

1.3.2.5.1.13.1.2.13计算 所耗时间, 返回

```

double t_sliding_end = omp_get_wtime();std::cout<<RED<<"[DEBUG]:
Map sliding using "<<t_sliding_end - t_sliding_start<<"
secs"<<RESET<<"\n";return;

```

1.3.2.5.1.13.2 18、根据稠密地图使能决定使用 未体素化的去畸变点 还是 体素化后的去畸变点。并获得点数。laserCloudFullRes, 一般是前者

```

PointCloudXYZI::Ptr laserCloudFullRes(dense_map_en ? feats_undistort :
feats_down_body);int size = laserCloudFullRes->points.size();

```

1.3.2.5.1.13.3 19、根据上面选择的点云的点数, 设置世界系下的点云指针

```

PointCloudXYZI::Ptr laserCloudWorld(new PointCloudXYZI(size, 1));

```

1.3.2.5.1.13.4 20、循环每个点, 其实就是将点云转到世界系下 (没有 RGB, 有反射强度)

```

RGBpointBodyToWorld(&laserCloudFullRes->points[i], &laserCloudWorld-
>points[i]);对应实现: for (int i = 0; i < size; i++)
{RGBpointBodyToWorld(&laserCloudFullRes->points[i], &laserCloudWorld-
>points[i]); }void LIVMapper::RGBpointBodyToWorld(PointType const *const pi,
PointType *const po)

```

1.3.2.5.1.13.5 21、将世界系下的点云设为 待发布的世界系的点云 `pcl_w_wait_pub`:

```
*pcl_w_wait_pub = *laserCloudWorld;
```

1.3.2.5.1.14 8、发布各类型点云、路径

1.3.2.5.1.14.1 22、判断 不使用图像则执行 `if(!img_en)`。涉及 VIO 管理器。（该发布者涉及色彩，，LIVO 模式的 LIO 时不会调用该函数）纯 LIO 或 LO 可以调用，但就是发布普通点云。

```
publish_frame_world(pubLaserCloudFullRes, vio_manager);
```

1.3.2.5.1.14.1.1 1、判断 `pcl_w_wait_pub` 是否有点;`if (pcl_w_wait_pub->empty()) return;`

1.3.2.5.1.14.1.2 2、这里简要介绍功能：将世界点转到相机系和像素系，只要该像素位于图像范围，对其进行色彩插值，并转为 ros 消息类型并发布

1.3.2.5.1.14.1.3 3、还有保存 PCD 功能：保存间隔大于 0 时可用

1.3.2.5.1.14.2 23、判断 是否发布有效点使能 `if (pub_effect_point_en)`。是则（发布能够匹配到平面的点，转到世界系发布）

```
publish_effect_world(pubLaserCloudEffect, voxelmap_manager->ptpl_list_);
```

1.3.2.5.1.14.3 24、判断是否发布体素平面地图：

```
if (voxelmap_manager->config_setting_.is_pub_plane_map_)，是则  
voxelmap_manager->pubVoxelMap();void VoxelMapManager::pubVoxelMap()简  
单介绍：将每个节点的平面属性同函数 GetUpdatePlane 给 pub_plane_list，根据  
平面的不确定性调用 mapJet 生成 r, g, b 值，再根据平面属性和 r,g,b 值生成一  
个 visualization_msgs::MarkerArray voxel_plane，再把这个 voxel_plane 发布出去
```

1.3.2.5.1.14.4 25、发布路径 `publish_path(pubPath)`; 和 `mavros` 格式的状态位姿话题

```
publish_mavros(mavros_pose_publisher);
```

1.3.2.5.1.15 9、打印 LIO 整体信息

1.3.2.5.1.15.1 26、总帧数累加 `frame_num++;`，计算平均耗时 `aver_time_consu = aver_time_consu * (frame_num - 1) / frame_num + (t4 - t0) / frame_num;;`

1.3.2.5.1.15.2 27、打印降采样耗时 `(t_down - t0)`，ICP 算法耗时 `(t2 - t1)`，体素 地图更新 `(t4 - t3)`，本帧总耗时 `(t4 - t0)`，平均耗时 `(aver_time_consu)`。

1.3.2.5.1.15.3 28、转为欧拉角，将更新后的状态保存到 `fout_out` 中，`mat_out.txt`

1.3.2.5.2 2、`void LIVMapper::handleVIO()`

定义于 LIVMapper.cpp 在执行完 LIO 后的下个循环会执行 VIO 更新

- 1.3.2.5.2.1 1、获得状态的旋转欧拉角 **euler_cur** 同时将上次 LIO 获得的状态输出到文件 **fout_pre**
- ```
euler_cur = RotMtoEuler(_state.rot_end);fout_pre << std::setw(20) <<
LidarMeasures.last_lio_update_time - _first_lidar_time << " " << euler_cur.transpose() *
57.3 << " " << _state.pos_end.transpose() << " " << _state.vel_end.transpose() << " " <<
_state.bias_g.transpose() << " " << _state.bias_a.transpose() << " " <<
V3D(_state.inv_expo_time, 0, 0).transpose() << std::endl;
```
- 1.3.2.5.2.2 2、输出待发布的世界点的点数:
- ```
std::cout << "[ VIO ] Raw feature num: " << pcl_w_wait_pub->points.size() << std::endl;
```
- 1.3.2.5.2.3 3、判断 这一帧图像捕获时间与第一帧点云帧头时间差再减去-10 的绝对值是否 小于 0.3 (帧计数 6/2 * 0.1) 。-----可略过
- ```
if (fabs((LidarMeasures.last_lio_update_time - _first_lidar_time) - plot_time) <
(frame_cnt / 2 * 0.1)) : **x+10 < 0.3 **是: vio_manager->plot_flag = true;否:
vio_manager->plot_flag = false;
```
- 1.3.2.5.2.4 4、vio\_manager->**processFrame**(LidarMeasures.measures.**back**().img, \_pv\_list, voxelmap\_manager->voxel\_map, LidarMeasures.last\_lio\_update\_time - \_first\_lidar\_time);
- 调用 vio 管理器出来这一帧图像，传入了这一帧的图像，不确定点列表，体素地图，与第一帧点云帧头的时间差。对应函数：void VIOManager::processFrame(cv::Mat &img, vector<pointWithVar> &pg, const unordered\_map<VOXEL\_LOCATION, VoxelOctoTree\*> &feat\_map, double img\_time)声明于 vio.h 定义于 vio.cpp

- 1.3.2.5.2.4.1 1、首先判断图像的宽高对不对，不对则发出警告并将图像尺寸进行插值重置。

```
if (width != img.cols || height != img.rows) {if (img.empty()) printf("[VIO] Empty Image!\n");cv::resize(img, img, cv::Size(img.cols * image_resize_factor, img.rows * image_resize_factor), 0, 0, CV_INTER_LINEAR); }实际图像与 yaml 中的是否一致。
```

- 1.3.2.5.2.4.2 2、将图像分别复制给两个图像变量 **img\_rgb** , **img\_cp** 做不同用途

```
img_rgb = img.clone();img_cp = img.clone();
```

- 1.3.2.5.2.4.3 3、将原彩色图像转换为灰度图像 **img**

```
if (img.channels() == 3) cv::cvtColor(img, img, CV_BGR2GRAY);
```

- 1.3.2.5.2.4.4 4、**new\_frame\_.reset(new Frame(cam, img));**

根据相机模型和灰度图像初始化一个新帧。就是将灰度图像给 new\_frame\_ 其中 FramePtr new\_frame\_ 声明于 vio.h。在初始化时会调用其对应的构造函数：

```
`Frame::Frame(vk::AbstractCamera *cam, const cv::Mat &img) :
id_(frame_counter_++), cam_(cam)`{ initFrame(img); ※ // 初始化}
```

- 1.3.2.5.2.4.4.1 1、构造函数被调用时，帧计数器会自加 1

```
frame_counter_++
```

- 1.3.2.5.2.4.4.2 2、获得相机模型

```
cam_(cam)
```

- 1.3.2.5.2.4.4.3 3、initFrame(**img**);

基于当前图像进行初始化对应函数：void Frame::initFrame(const cv::Mat

&img)声明于 frame.h, 定义于 frame.cpp

#### 1.3.2.5.2.4.4.3.1 1、判断图像是不是空的

```
: if (img.empty()) { throw std::runtime_error("Frame: provided image is empty"); }
```

#### 1.3.2.5.2.4.4.3.2 2、判断实际图像长宽与预定义相机模型是否匹配, 否则报错

```
if (img.cols != cam_ -> width() || img.rows != cam_ -> height()) {throw std::runtime_error("Frame: provided image has not the same size as the camera model"); }
```

#### 1.3.2.5.2.4.4.3.3 3、判断图像是否已转为灰度图, 否则报错

```
if (img.type() != CV_8UC1) { throw std::runtime_error("Frame: provided image is not grayscale"); }
```

#### 1.3.2.5.2.4.4.3.4 4、将图像给新帧的成员 img\_

```
img_ = img;
```

### 1.3.2.5.2.4.5 5、updateFrameState(\*state);

获得根据状态获得世界系相对相机系的位姿 (SE3) new\_frame\_ -> T\_f\_w\_ 对应函数: void VIOManager::updateFrameState(StatesGroup state)声明于 vio.h, 定义于 vio.cpp

#### 1.3.2.5.2.4.5.1 1、获得 IMU 系相对世界系得位姿 Rwi, Pwi

```
M3D Rwi(state.rot_end); V3D Pwi(state.pos_end);
```

#### 1.3.2.5.2.4.5.2 2、获得世界系相对相机系得位姿 Rcw Pcw

```
Rcw = Rci * Rwi.transpose(); Pcw = -Rci * Rwi.transpose() * Pwi + Pci;
```

#### 1.3.2.5.2.4.5.3 3、将位姿以李群的形式保存到 new\_frame\_ -> T\_f\_w\_

```
new_frame_ -> T_f_w_ = SE3(Rcw, Pcw);
```

### 1.3.2.5.2.4.6 6、resetGrid();

重置当前图像栅格 对应函数: void VIOManager::resetGrid()声明于 vio.h, 定义于 vio.cpp 对相关容器进行初始化并将总点数 total\_points 设为 0grid\_num, map\_index, map\_dist, update\_flag, scan\_value, retrieve\_voxel\_points, append\_voxel\_points。

#### 1.3.2.5.2.4.6.1 1、使用了<algorithm>头文件中的 fill 函数, 对多个容器进行初始化

```
fill(grid_num.begin(), grid_num.end(), TYPE_UNKNOWN); // 每个栅格的类型
fill(map_index.begin(), map_index.end(), 0); // 没用到---可略过
fill(map_dist.begin(), map_dist.end(), 10000.0f); // 视觉地图点到图像的距离
fill(update_flag.begin(), update_flag.end(), 0); // 表明该体素点是否为视觉地图点, 是则为 1.
fill(scan_value.begin(), scan_value.end(), 0.0f); // 该视觉地图点对应像素的角点分数
```

1.3.2.5.2.4.6.2 2、候选体素点容器 `retrieve_voxel_points` 清空并重置为栅格总数大小

```
retrieve_voxel_points.clear();retrieve_voxel_points.resize(length);每个栅格都留有一个候选体素点。
```

1.3.2.5.2.4.6.3 3、添加体素点容器 `append_voxel_points` 清空并重置为栅格总数大小

```
append_voxel_points.clear(); //可作为新视觉地图点的体素点
append_voxel_points.resize(length);在之后的计算中，会基于
append_voxel_points 中的点生成新视觉点 VisualPoint 以及新的特征 Feature。
```

1.3.2.5.2.4.6.4 3、总点数值为零; `total_points = 0;`

**1.3.2.5.2.4.7 记录时间: `double t1 = omp_get_wtime();`**

**1.3.2.5.2.4.8 7、`retrieveFromVisualSparseMap(img, pg, feat_map);` 这个函数真的很长.....**

传入灰度图，不确定性点列表，体素哈希图对应函数: void  
VIOManager::retrieveFromVisualSparseMap(cv::Mat img, vector<pointWithVar> &pg, const unordered\_map<VOXEL\_LOCATION, VoxelOctoTree \*> &plane\_map)  
请读者注意，进入函数 retrieveFromVisualSparseMap 后会在函数内部再看到一个变量 feat\_map (unordered\_map<VOXEL\_LOCATION, VOXEL\_POINTS \*> feat\_map;)，不要跟当前的 feat\_map (unordered\_map<VOXEL\_LOCATION, VoxelOctoTree \*>) 整混淆了。主要是从全局视觉地图点中，获得当前视野内的对应图像每个栅格的视觉地图点 visual\_submap。

1.3.2.5.2.4.8.1 1、判断 视觉管理器的全局视觉地图点是否有数

```
if (feat_map.size() <= 0) return;若是首次运行该函数，将直接返回。
```

1.3.2.5.2.4.8.2 记录时间: `double ts0 = omp_get_wtime();`

1.3.2.5.2.4.8.3 2、视觉子地图进行重置（当前帧视野内的体素点）

```
visual_submap->reset(); 其中: SubSparseMap *visual_submap; 定义于 vio.h 调用函数: void reset() {propa_errors.clear(); //未进行 VIO 更新前 视觉地图点仿射到参考图像上的图像块像素与当前图像上图像块的像素光度误差。errors.clear(); //进行 VIO 更新后 视觉地图点仿射到参考图像上的图像块像素与当前图像上图像块的像素光度误差。warp_patch.clear(); //视觉地图点仿射到参考图像上的图像块像素 search_levels.clear(); //基于仿射变换阵筛选的 search_level 要么是 0 要么是 1.voxel_points.clear(); //在当前视野内的体素点 inv_expo_list.clear(); //当前视野内点对应的参考图像块逆曝光时间，add_from_voxel_map.clear(); //候选视觉地图点的体素点，专门给体素射线上的点用的。 }
```

1.3.2.5.2.4.8.4 3、视觉子特征图（哈希表）清空:

```
sub_feat_map.clear(); //将当前帧视野下的体素点对应值为
0unordered_map<VOXEL_LOCATION, int> sub_feat_map;声明于 vio.h，定义于 vio.h。
```

#### 1.3.2.5.2.4.8.5 4、体素大小

```
float voxel_size = 0.5;
```

#### 1.3.2.5.2.4.8.6 5、判断 法向量使能与否 不能则清空 扭曲图（哈希表）

```
if (!normal_en) warp_map.clear(); unordered_map<int, Warp *> warp_map;
```

#### 1.3.2.5.2.4.8.7 6、声明一个 32 单通道的深度图像 **depth\_img** 并用指针 **it** 指向其数据

```
cv::Mat depth_img = cv::Mat::zeros(height, width, CV_32FC1); float *it = (float *)depth_img.data;
```

#### 1.3.2.5.2.4.8.8 7、声明一个浮点体素位置; int loc\_xyz[3];

#### 1.3.2.5.2.4.8.9 8、遍历不确定性点

```
for (int i = 0; i < pg.size(); i++) 1、对 sub_feat_map 进行初始化，将当前帧不确定性点对应体素位置的所有键值都置为 0； 2、并在距离原始图像边界为 border 的范围获得每个像素深度给 depth_img。计算不确定性点投影的图像像素点处的深度，并记录到 depth_img 中。
```

##### 1.3.2.5.2.4.8.9.1 1、取出不确定性点列表的世界点

```
V3D pt_w = pg[i].point_w;
```

##### 1.3.2.5.2.4.8.9.2 2、计算体素位置：VOXEL\_LOCATION **position**

```
for (int j = 0; j < 3; j++) { loc_xyz[j] = floor(pt_w[j] / voxel_size); if (loc_xyz[j] < 0) { loc_xyz[j] -= 1.0; } } VOXEL_LOCATION position(loc_xyz[0], loc_xyz[1], loc_xyz[2]);
```

##### 1.3.2.5.2.4.8.9.3 3、获得该位置的子特征图索引

```
auto iter = sub_feat_map.find(position);
```

##### 1.3.2.5.2.4.8.9.4 4、如果 **sub\_feat\_map** 索引没有对应的值，则用 0 生成一个值，如果有对应的值则重置该值为 0。不管怎么样都置为 0 为何还要判断？

```
if (iter == sub_feat_map.end()) { sub_feat_map[position] = 0; } else { iter->second = 0; }
```

##### 1.3.2.5.2.4.8.9.5 5、用新帧的成员将世界点转到相机系下

```
V3D pt_c(new_frame_ ->w2f(pt_w));
```

##### 1.3.2.5.2.4.8.9.6 6、判断 点是否在相机正前方：if (pt\_c[2] > 0)

转到像素坐标系，并在距离原始图像边界为 border 的范围获得该像素坐标下的深度值给 **depth\_img**

##### 1.3.2.5.2.4.8.9.7 1、生成一个像素坐标向量：V2D px;

##### 1.3.2.5.2.4.8.9.8 2、根据相机系的点投影到像素

```
px = new_frame_ ->cam_ ->world2cam(pt_c);
```

1.3.2.5.2.4.8.9.9 3、判断 该像素点是否位于 相机视图内：

```
if (new_frame_ -> cam_ -> isInFrame(px.cast<int>(), border))在距离原始图像
边界为 border 的范围内计算深度， depth_img 获得相机系下的深度值。
inline bool isInFrame(const Vector2i & obs, int boundary=0) const.
{if(obs[0]>=boundary && obs[0]<width()-boundary&& obs[1]>=boundary
&& obs[1]<height()-boundary)return true;return false;}
```

1.3.2.5.2.4.8.9.10 1、获得该相机系下点的深度： float depth = pt\_c[2];

1.3.2.5.2.4.8.9.11 2、获得该像素的列 col 和行 row：

```
int col = int(px[0]);int row = int(px[1]);
```

1.3.2.5.2.4.8.9.12 3、将该深度赋给 depth\_img

```
it[width * row + col] = depth;
```

1.3.2.5.2.4.8.10 9、生成一个容器，删除键列表，

```
vector<VOXEL_LOCATION> DeleteKeyList; （用于记录要删除体素的位置）
```

1.3.2.5.2.4.8.11 10、遍历整个子特征图（当前帧不确定性点）

```
for (auto &iter : sub_feat_map)请各位读者注意，这部分就是论文中提及的为
了快速获得视野内的视觉地图点，先用当前帧点云去确定体素坐标，再取
视觉地图点。之后计算视觉地图点对应的栅格单元坐标，更新栅格类型
grid_num，选出该栅格位置深度最小的作为候选点 retrieve_voxel_points，并
记录深度 map_dist。不在视野内则将视觉地图点的体素位置推入删除键列
表中 DeleteKeyList。 **
```

1.3.2.5.2.4.8.11.1 1、获得子特征图索引下的体素位置

```
VOXEL_LOCATION position = iter.first;
```

1.3.2.5.2.4.8.11.2 2、获得全局特征图在该体素位置索引

```
auto corre_voxel = feat_map.find(position);
unordered_map<VOXEL_LOCATION, VOXEL_POINTS *> feat_map; 定
义于 vio.h。（
```

1.3.2.5.2.4.8.11.3 3、若该索引有对应的体素点容器： if (corre\_voxel != feat\_map.end())

```
对该索引对应体素的所有体素点进行遍历，选出相机视野内，距离图像
深度最小的体素点作为候选点 retrieve_voxel_points，并记录深度
map_dist。不在视野内则将该体素位置推入删除键列表中
DeleteKeyList。
```

1.3.2.5.2.4.8.11.4 1、将体素在视野标志位 置 false

```
bool voxel_in_fov = false;
```

1.3.2.5.2.4.8.11.5 2、获得全局特征图中此体素位置对应的体素点容器

```
std::vector<VisualPoint *> &voxel_points = corre_voxel->second-
>voxel_points;
```

1.3.2.5.2.4.8.11.6 3、获得体素点容器中的点数: `int voxel_num = voxel_points.size();`

1.3.2.5.2.4.8.11.7 4、遍历该体素点容器的体素点:

```
for (int i = 0; i < voxel_num; i++)确保该体素点有观测它的参考块 obs_,
再确保该体素点在图像的正前方, 计算此点的栅格单元索引以及到图像的
深度, 选择最小的作为候选点 retrieve_voxel_points, 并记录深度
map_dist.**
```

1.3.2.5.2.4.8.11.8 1、取出一个体素点对象作为视觉点

```
VisualPoint *pt = voxel_points[i];
```

1.3.2.5.2.4.8.11.9 2、判断 是否为空指针

```
if (pt == nullptr) continue;
```

1.3.2.5.2.4.8.11.103、判断 obs\_是否为空:

```
if (pt->obs_.size() == 0) continue;obs_指的是观测到这个体素点的图像块
列表, 或者说附加到这个视觉地图点的图像块列表。
```

1.3.2.5.2.4.8.11.114、将视觉地图点的法向量同世界系转到当前帧相机系下

```
V3D norm_vec(new_frame_>T_f_w_.rotation_matrix() * pt->normal_);
```

1.3.2.5.2.4.8.11.125、将视觉地图点从世界系转到当前帧相机系下:

```
V3D dir(new_frame_>T_f_w_ * pt->pos_);
```

1.3.2.5.2.4.8.11.13判断 是否在图像正前方: `if (dir[2] < 0) continue;`

1.3.2.5.2.4.8.11.146、将视觉地图点从世界系转到当前图像的像素坐标系:

```
V2D pc(new_frame_>w2c(pt->pos_));
```

1.3.2.5.2.4.8.11.157、判断这个像素点是否在边界内:

```
if (new_frame_>cam_>isInFrame(pc.cast<int>(), border))找到该体素中每
个点对应的图像栅格单元, 更新单元类型 grid_num, 并找到体素中距
离图像深度最小的体素点, 作为候选点 retrieve_voxel_points 并记录此
时深度 map_dist
```

1.3.2.5.2.4.8.11.161、视觉地图点在当前帧视野的标志为设为真,

```
voxel_in_fov = true;
```

1.3.2.5.2.4.8.11.172、获得视觉地图点对应像素点的栅格单元索引:

```
int index = static_cast<int>(pc[1] / grid_size) * grid_n_width +
static_cast<int>(pc[0] / grid_size);类似下面的排布 1 2 3 4 5 6 7 8 9 10 11
12 13 14 ...
```

1.3.2.5.2.4.8.11.183、将栅格类型向量进行赋值

```
grid_num[index] = TYPE_MAP; //说明该栅格有对应的视觉地图点其中
TYPE_MAP 代表该栅格单元的类型。enum CellType {TYPE_MAP =
```



```
1,TYPE_POINTCLOUD,TYPE_UNKNOWN };
```

1.3.2.5.2.4.8.11.194、计算从视觉地图点到帧系的距离向量

```
Vector3d obs_vec(new_frame_ ->pos() - pt->pos_);
```

1.3.2.5.2.4.8.11.205、获得该向量的模长（点的帧系的距离）：float cur\_dist = obs\_vec.norm();

1.3.2.5.2.4.8.11.216、选出到当前图像 index 栅格距离最近的视觉地图点，并将此体素点赋值给该栅格位置的候选体素点。

```
if (cur_dist <= map_dist[index]){map_dist[index] = cur_dist;
retrieve_voxel_points[index] = pt;}
```

1.3.2.5.2.4.8.11.225、如果该体素不在这个视野内，将这个体素位置 **position** 推入到 删除键列表中 **DeleteKeyList**。

```
if (!voxel_in_fov) { DeleteKeyList.push_back(position); }
```

1.3.2.5.2.4.8.12 11、判断 射线投射是否使能 if (raycast\_en)，并遍历栅格单元----默认不使用，可略过

```
for (int i = 0; i < length; i++) 避开边缘栅格和有对应体素点的栅格，获得候选点或者平面属性：获得射线采样点在世界系的坐标，先从全局特征图找对应体素位置的体素点向量，更新单元类型 grid_num，找到体素中距离图像深度最小的体素点，作为候选点 retrieve_voxel_points 并记录此时深度 map_dist。如果 feat_map 中没有，则从点云体素图中找该位置索引，并获得对应的八叉树节点，将节点为真的平面属性推入到视觉子图 visual_submap 的 add_from_voxel_map 中
```

1.3.2.5.2.4.8.12.1 1、判断 栅格单元属性为 地图类型 1，或者 是边缘栅格单元：满足其一则直接跳出（如果这个栅格单元有投影到其上的体素点或者这个栅格单元位于边缘则不进行射线投射）

```
if (grid_num[i] == TYPE_MAP || border_flag[i] == 1) continue; border_flag
在 initializeVIO 中完成初始化，记录哪些栅格是位于图像边缘的。
```

1.3.2.5.2.4.8.12.2 2、遍历初始化就生成的该栅格单元的每个采样点：for (const auto &it :

```
rays_with_sample_points[i])
```

```
获得射线采样点在世界系的坐标，先从全局特征图找对应体素位置的体素点向量，更新单元类型 grid_num，找到体素中距离图像深度最小的体素点，作为候选点 retrieve_voxel_points 并记录此时深度 map_dist。如果全局特征图中没有，则从点云体素图中找该位置索引，并获得对应的八叉树节点，将节点为真的平面属性推入到视觉子图 visual_submap 的 add_from_voxel_map 中
```

1.3.2.5.2.4.8.12.3 1、将相机系的采样点转到世界系下：

```
V3D sample_point_w = new_frame_ ->f2w(it);
```

1.3.2.5.2.4.8.12.4 2、计算世界系射线采样点对应的体素坐标: **loc\_xyz**

```
for (int j = 0; j < 3; j++) {loc_xyz[j] = floor(sample_point_w[j] /
voxel_size);if (loc_xyz[j] < 0) { loc_xyz[j] -= 1.0; } }
```

1.3.2.5.2.4.8.12.5 3、转为哈希表的体素位置

```
VOXEL_LOCATION sample_pos(loc_xyz[0], loc_xyz[1], loc_xyz[2]);
```

1.3.2.5.2.4.8.12.6 4、子特征图中找到该位置的哈希索引 **corre\_sub\_feat\_map** (未使用)

```
auto corre_sub_feat_map = sub_feat_map.find(sample_pos);if
(corre_sub_feat_map != sub_feat_map.end()) break;
```

1.3.2.5.2.4.8.12.7 5、全局特征图中找到该位置的哈希索引 **corre\_feat\_map**

```
auto corre_feat_map = feat_map.find(sample_pos);
```

1.3.2.5.2.4.8.12.8 6、判断, 该索引是否为 **feat\_map** 的有效索引

```
if (corre_feat_map != feat_map.end())确保该体素点有被参考块观测到
obs_, 找到视野内 (若是视野内子特征图的该位置的值为 0) 该体素
中每个点对应的图像栅格单元, 更新单元类型 grid_num, 并找到体素
中距离图像深度最小的体素点, 作为候选点 retrieve_voxel_points 并记
录此时深度 map_dist
```

1.3.2.5.2.4.8.12.9 1、体素点位于视野标志位置 **false**

```
bool voxel_in_fov = false;
```

1.3.2.5.2.4.8.12.102、引用该位置全局特征图此索引下的所有体素点列表 **&voxel\_points** :

```
std::vector<VisualPoint *> &voxel_points = corre_feat_map->second-
>voxel_points;
```

1.3.2.5.2.4.8.12.113、判断 该列表是否有点

```
int voxel_num = voxel_points.size();if (voxel_num == 0) continue;
```

1.3.2.5.2.4.8.12.124、遍历体素列表中的所有点

```
for (int j = 0; j < voxel_num; j++)确保该体素点有被参考块观测到 obs_,
找到该体素中每个点对应的图像栅格单元, 更新单元类型 grid_num,
并找到体素中距离图像深度最小的体素点, 作为候选点
retrieve_voxel_points 并记录此时深度 map_dist
```

1.3.2.5.2.4.8.12.131、取出一个体素点对象

```
VisualPoint *pt = voxel_points[i];
```

1.3.2.5.2.4.8.12.142、判断 是否为空指针

```
if (pt == nullptr) continue;
```

1.3.2.5.2.4.8.12.153、判断 **obs\_** (观测到这个体素点的参考块列表) 是否为空

```
if (pt->obs_.size() == 0) continue;
```

1.3.2.5.2.4.8.12.164、将这体素点的法向量同世界系转到当前帧相机系下

```
V3D norm_vec(new_frame_ ->T_f_w_.rotation_matrix() * pt->normal_);
```

1.3.2.5.2.4.8.12.175、将体素点的位置从世界系转到当前帧相机系下

```
V3D dir(new_frame_ ->T_f_w_ * pt->pos_);
```

1.3.2.5.2.4.8.12.18判断 是否在图像正前方

```
if (dir[2] < 0) continue;
```

1.3.2.5.2.4.8.12.196、将当前体素点的位置从世界系转到像素坐标系

```
V2D pc(new_frame_ ->w2c(pt->pos_));
```

1.3.2.5.2.4.8.12.207、判断这个像素点是否在边界内

```
: if (new_frame_ ->cam_ ->isInFrame(pc.cast<int>(), border))找到该体素中
每个点对应的图像栅格单元，更新单元类型 grid_num，并找到体素中
距离图像深度最小的体素点，作为候选点 retrieve_voxel_points 并记录
此时深度 map_dist
```

1.3.2.5.2.4.8.12.211、视觉地图点在当前帧视野的标志为设为真，

```
voxel_in_fov = true;
```

1.3.2.5.2.4.8.12.222、获得视觉地图点对应像素点的栅格单元索引：

```
int index = static_cast<int>(pc[1] / grid_size) * grid_n_width +
static_cast<int>(pc[0] / grid_size);类似下面的排布 1 2 3 4 5 6 7 8 9 10 11
12 13 14 ...
```

1.3.2.5.2.4.8.12.233、将栅格类型向量进行赋值

```
grid_num[index] = TYPE_MAP; //说明该栅格有对应的视觉地图点其中
TYPE_MAP 代表该栅格单元的类型。enum CellType {TYPE_MAP =
1,TYPE_POINTCLOUD,TYPE_UNKNOWN };
```

1.3.2.5.2.4.8.12.244、计算从视觉地图点到帧系的距离向量

```
Vector3d obs_vec(new_frame_ ->pos() - pt->pos_);
```

1.3.2.5.2.4.8.12.255、获得该向量的模长（点的帧系的距离）： float cur\_dist = obs\_vec.norm();

1.3.2.5.2.4.8.12.266、选出到当前图像 index 栅格距离最近的视觉地图点，并将此体素点赋值给该栅格位置的候选体素点。

```
if (cur_dist <= map_dist[index]){map_dist[index] = cur_dist;
retrieve_voxel_points[index] = pt;}
```

1.3.2.5.2.4.8.12.275、判断 体素点在视野内 则 子特征图的该位置的值设为 0。

1.3.2.5.2.4.8.12.287、否则 该索引非全局特征图的有效索引

所全局特征图中没有可用索引，则从点云体素图中找该位置索引，并获得对应的八叉树节点，将节点为真的平面属性推入到视觉子图

**visual\_submap** 的 **add\_from\_voxel\_map** 中

1.3.2.5.2.4.8.12.291、又计算了一遍哈希位置形式（似乎没有必要） **sample\_pos**

`VOXEL_LOCATION sample_pos(loc_xyz[0], loc_xyz[1], loc_xyz[2]);`

1.3.2.5.2.4.8.12.302、从点云体素图中获得该位置的哈希索引： `auto iter =`

`plane_map.find(sample_pos);`

1.3.2.5.2.4.8.12.313、判断 该索引在点云体素图中是否有效： `if (iter != plane_map.end())`

找到位置对应的八叉树，并将为真的平面属性推入到视觉子图

**visual\_submap** 的 **add\_from\_voxel\_map** 中

1.3.2.5.2.4.8.12.321、声明一个 八叉树节点： `VoxelOctoTree *current_octo;`

1.3.2.5.2.4.8.12.332、找到点云体素图中该射线采样点对应的八叉树节点

`current_octo = iter->second->find_correspond(sample_point_w);`

1.3.2.5.2.4.8.12.343、判断 当前八叉树节点的平面属性是否为真

`if (current_octo->plane_ptr->is_plane_)`若是，将当前的平面属性推入到视觉子图 **visual\_submap** 的 **add\_from\_voxel\_map** 中

1.3.2.5.2.4.8.12.351、声明一个不确定点属性： `pointWithVar plane_center;`

1.3.2.5.2.4.8.12.362、获得该八叉树节点的平面属性： `VoxelPlane &plane = *current_octo->plane_ptr;`

1.3.2.5.2.4.8.12.373、将平面中心点（世界系）和法向量给上面的不确定点属性 **plane\_center**

`plane_center.point_w = plane.center_; plane_center.normal = plane.normal_;`

1.3.2.5.2.4.8.12.384、将这个不确定点属性 推入到 视觉子图中： **visual\_submap-**  
`>add_from_voxel_map.push_back(plane_center);`

`visual_submap。SubSparseMap *visual_submap;` 定义于 `vio.h`

1.3.2.5.2.4.8.13 12、遍历 `DeleteKeyList`，擦除子特征图中不在该帧图像视野内的体素位置对应的值 `sub_feat_map.erase(key);`

`for (auto &key : DeleteKeyList) {sub_feat_map.erase(key); }`

1.3.2.5.2.4.8.14 13、遍历整个栅格单元： `for (int i = 0; i < length; i++)`，并且必须栅格单元上有投影的视觉地图点。 `if (grid_num[i] == TYPE_MAP)`

用栅格单元对应的世界体素点坐标，获得相机系和像素系坐标，判断是否深度连续（ $> 0.5$  直接下个栅格单元），若连续则进行下面操作：先获得该视觉点的参考块 `pt->ref_patch`，`ref_ftr`，再根据参考块计算参考块与当前帧的仿射变换 `A_cur_ref_zero` 和最佳搜索层级 `search_level`，并且在无法向量时，对空缺的扭曲图进行补充 `warp_map[ref_ftr->id_]`根据上面计算的

**A\_cur\_ref** 计算 **A\_ref\_cur**，用于将当前块映射到参考块上。根据搜索层级和当前循环层级，对当前帧像素块进行放大，将当前帧像素块投影到参考图像上，获得以参考块中心为中心的投影块，判断投影块中的每个像素是否越界，越界的置 0，未越界的用参考图像对该位置进行插值，最终获得在参考图像上各层级扭曲块的像素值 **patch\_ptr/patch/patch\_wrap**。对当前帧块的每个像素进行插值填充。计算 **patch\_wrap** 与当前块的像素误差，需满足阈值，可选择地估计 Ncc，小于 ncc 阈值则跳过该栅格单元。（并没有计算分数）对视觉子图 **visual\_submap** 各属性进行更新

1.3.2.5.2.4.8.14.1 1、从候选点向量中取出视觉地图点：（retrieve\_voxel\_points 存的点的参考块列表都是  $\geq 1$ ）

```
VisualPoint *pt = retrieve_voxel_points[i];
```

1.3.2.5.2.4.8.14.2 2、获得视觉地图点在图像上的像素点坐标

```
: V2D pc(new_frame_ ->w2c(pt->pos_));
```

1.3.2.5.2.4.8.14.3 3、获得相机系下的该点坐标

```
V3D pt_cam(new_frame_ ->w2f(pt->pos_));
```

1.3.2.5.2.4.8.14.4 4、深度连续性标志位置 false

```
bool depth_continuous = false;
```

1.3.2.5.2.4.8.14.5 5、以视觉地图点在当前图像上投影像素坐标为中心构建 8x8 的图像块，遍历并

```
for (int u = -patch_size_half; u <= patch_size_half; u++) {for (int v = -patch_size_half; v <= patch_size_half; v++)以像素点为块中心并作为原点建系**uov**。起点从-patch_size_half 开始。（以该像素为中心，找到块中其他不确定性点投影的深度与之深度差>0.5 的，找到后置真标志位，退出双循环。）
```

1.3.2.5.2.4.8.14.6 1、如果坐标位于原点上（该像素点），直接跳过

```
if (u == 0 && v == 0) continue;
```

1.3.2.5.2.4.8.14.7 2、以这个像素点为中心，获得该块每个像素的深度值

```
float depth = it[width * (v + int(pc[1])) + u + int(pc[0])];
```

1.3.2.5.2.4.8.14.8 深度值为 0，跳出此次循环。

1.3.2.5.2.4.8.14.9 3、计算中心点与块中某点的深度差

```
double delta_dist = abs(pt_cam[2] - depth);
```

1.3.2.5.2.4.8.14.10 4、只要有深度差超过 0.5 米 if (delta\_dist > 0.5)，就将深度连续标志位置真

```
depth_continuous = true; 直接跳出双重循环 break;
```

1.3.2.5.2.4.8.14.11 6、若该栅格单元对应体素点的深度连续标志位为真（这里应该叫深度不连续标志位），跳过，计算下一个栅格

```
if (depth_continuous) continue; 只要深度差 > 0.5，下面 7 中的代码将不再
```

执行

#### 1.3.2.5.2.4.8.14.127、若深度差 $< 0.5$

先获得该视觉点的参考块 `pt->ref_patch`, `ref_ftr`, 再根据参考块计算参考块与当前帧的仿射变换帧 `A_cur_ref_zero` 和最佳搜索层级 `search_level`, 并且在无法向量时, 对空缺的扭曲图进行补充 `warp_map[ref_ftr->id_]` 根据上面计算的 `A_cur_ref` 计算 `A_ref_cur`, 用于下面双重循环将当前块映射到参考块上。根据搜索层级和当前循环层级, 对当前帧像素块进行放大, 将当前帧像素块投影到参考图像上, 获得以参考块中心为中心的投影块, 判断投影块中的每个像素是否越界, 越界的置 0, 未越界的用参考图像对该位置进行插值, 最终获得各层级扭曲块的像素值 `patch_ptr/patch/patch_wrap`。对块的每个像素进行插值填充。计算 `patch_wrap` 与当前块的误差, 需满足阈值, 可选择地估计 `Ncc`, 小于 `ncc` 阈值则跳过该栅格单元。(并没有计算分数) 对视觉子图 `visual_submap` 各属性进行更新

#### 1.3.2.5.2.4.8.14.131、声明一个参考特征:

`Feature *ref_ftr`; 描述一个在连续帧之间被跟踪的显著图像区域。

#### 1.3.2.5.2.4.8.14.142、声明一个块扭曲容器, 存放的是仿射到参考图像上的各金字塔层级图像块像素

`std::vector<float> patch_wrap(warp_len)`; 这里的扭曲长度为 `warp_len = patch_size_total * patch_pyramid_level`; 整个块的像素数乘以块的金字塔层数

#### 1.3.2.5.2.4.8.14.153、设置最佳搜寻层数

`int search_level`;

#### 1.3.2.5.2.4.8.14.164、声明一个仿射变换矩阵

`Matrix2d A_cur_ref_zero`;

#### 1.3.2.5.2.4.8.14.175、判断该视觉地图点的法向量是否已初始化

`if (!pt->is_normal_initialized_) continue`; 没初始化直接跳过。根据后面生成视觉地图点的代码, 视觉地图点法向量都必然初始化了。

#### 1.3.2.5.2.4.8.14.186、if (`normal_en`) 获得参考特征 `ref_ftr`

判断 是否法向量估计使能: 若该点未拥有参考块, 则通过计算视觉地图点的图像块列表中与其他块光度误差总和最小的图像块作为参考特征 `ref_ftr`。若该点已拥有参考块, 则直接将该块 `pt->ref_patch` 给参考块 `ref_ftr`

#### 1.3.2.5.2.4.8.14.191、初始化一个光度误差最小值

`float phtometric_errors_min = std::numeric_limits<float>::max()`;  
`std::numeric_limits` 是 来获取 `float` 类型在 C++ 中的最大值

#### 1.3.2.5.2.4.8.14.202、判断 观测到这个视觉点的图像块列表是否为 1

`if (pt->obs_.size() == 1)` 观测图像块列表只有一个图像块 `ref_ftr`, 直接将

该块给该点的参考块属性 `pt->ref_patch`

1.3.2.5.2.4.8.14.211、取出观测图像块列表的头部给参考特征

```
ref_ftr = *pt->obs_.begin();
```

1.3.2.5.2.4.8.14.222、直接将该参考特征给这个视觉地图点的参考块属性

```
pt->ref_patch = ref_ftr;
```

1.3.2.5.2.4.8.14.233、该点拥有参考块标志位 置真: `pt->has_ref_patch_ = true;`

1.3.2.5.2.4.8.14.243、否则判断, 该点是否还没有参考块:

```
else if (!pt->has_ref_patch_) 将该点对应的观测图像块列表中与其他块光度误差最小的块作为参考特征 ref_ftr, 并更新该点的参考块属性 pt->ref_patch 和标志位 pt->has_ref_patch_。
```

1.3.2.5.2.4.8.14.251、遍历该点的观测图像块列表

```
for (auto it = pt->obs_.begin(), ite = pt->obs_.end(); it != ite; ++it) 计算列表每个图像块与其他块的光度误差总和, 获得光度误差总和最小的临时块 ref_patch_temp 作为参考块 ref_ftr。
```

1.3.2.5.2.4.8.14.261、初始化一个临时参考块

```
Feature *ref_patch_temp = *it;
```

1.3.2.5.2.4.8.14.272、初始化一个指向图像上该块数据的指针 (指向浮点数数组)

```
float *patch_temp = ref_patch_temp->patch_;
```

1.3.2.5.2.4.8.14.283、初始化光度误差为 0

```
float phtometric_errors = 0.0;
```

1.3.2.5.2.4.8.14.294、初始化计数为 0

```
int count = 0;
```

1.3.2.5.2.4.8.14.305、再次遍历该点的观测图像块列表 (同时避开上层循环的临时块)

```
for (auto itm = pt->obs_.begin(), itme = pt->obs_.end(); itm != itme; ++itm) 计算列表当前参考块与列表其他参考块像素值差 (光度差) 的总和 phtometric_errors, 并计数 count
```

1.3.2.5.2.4.8.14.311、同时避开上层循环的临时块

```
f((*itm)->id_ == ref_patch_temp->id_) continue;
```

1.3.2.5.2.4.8.14.322、取图像块列表的其他块数据指针给

```
patch_cache : float *patch_cache = (*itm)->patch_;
```

1.3.2.5.2.4.8.14.333、遍历块像素计算当前点对应的两两参考块间像素值差的平方

```
for (int ind = 0; ind < patch_size_total; ind++) phtometric_errors += (patch_temp[ind] - patch_cache[ind]) * (patch_temp[ind] - patch_cache[ind]);
```

1.3.2.5.2.4.8.14.344、计数累加：count++;

1.3.2.5.2.4.8.14.356、判断 光度误差和是否小于光度误差最小值

```
if (phtometric_errors < phtometric_errors_min)更新光度误差最小值。将临
时参考块给参考块 phtometric_errors_min = phtometric_errors;
ref_ftr = ref_patch_temp;
```

1.3.2.5.2.4.8.14.362、将参考块赋值该视觉点的参考块属性

```
pt->ref_patch = ref_ftr;
```

1.3.2.5.2.4.8.14.373、将该视觉点的用于参考块标志位置真

```
pt->has_ref_patch_ = true;
```

1.3.2.5.2.4.8.14.384、否则 直接就将该点的参考块属性 pt->ref\_patch 给 参考块 ref\_ftr: ref\_ftr = pt->ref\_patch;

1.3.2.5.2.4.8.14.397、否则 该点是否无法获得最近观测参考块，跳出计算下个栅格单元：---可略过

```
if (!pt->getCloseViewObs(new_frame_>pos(), ref_ftr, pc)) continue; bool
VisualPoint::getCloseViewObs(const Vector3d &framepos, Feature *&ftr,
const Vector2d &cur_px) const 定义于 visual_point.cpp 通过计算点到当
前帧向量与点到列表中参考块向量的夹角确定参考块，只要最大的 cos
值，且角度不超过 60 度，则该块为参考块。
```

1.3.2.5.2.4.8.14.401、判断 参考块列表长度是否  $\leq 0$  : if (obs\_.size() <= 0) return false;

这里似乎没有意义 因为 pt 来自 retrieve\_voxel\_points , 而  
retrieve\_voxel\_points 选择的 pt, 都有长度不小于 1 的参考块列表。

1.3.2.5.2.4.8.14.412、世界系下 pt 点位置到当前帧的位置的向量: Vector3d obs\_dir(framepos - pos\_);

1.3.2.5.2.4.8.14.423、对向量进行归一化: obs\_dir.normalize();

1.3.2.5.2.4.8.14.434、获得参考块列表的头部: auto min\_it = obs\_.begin();

1.3.2.5.2.4.8.14.445、初始化一个最小 cos 角 (应该是最大吧) : double min\_cos\_angle = 0;

1.3.2.5.2.4.8.14.456、循环 观测参考块列表: for (auto it = obs\_.begin(), ite = obs\_.end(); it != ite; ++it)

找到参考块列表中 点到参考块向量与点到当前帧系向量的夹角最大的  
那个块 min\_it 。



1.3.2.5.2.4.8.14.461、（获得世界系下点到参考块的向量）这个块在世界系下的位置并减去当前点位置： `Vector3d dir((*it)->T_f_w_.inverse().translation() - pos_);`

1.3.2.5.2.4.8.14.472、对这个向量进行归一化： `dir.normalize();`

1.3.2.5.2.4.8.14.483、点到帧系向量与点到块向量之间的夹角 cos 值： `double cos_angle = obs_dir.dot(dir);`

1.3.2.5.2.4.8.14.494、判断 是否 cos 角 > 最小的 cos 角： `if (cos_angle > min_cos_angle), 并更新`  
`min_cos_angle = cos_angle; min_it = it;`

1.3.2.5.2.4.8.14.507、将这个块给 参考块 `Feature *&ftr: ftr = *min_it;`

1.3.2.5.2.4.8.14.518、如果 cos 值小于 0.5，返回 false: `if (min_cos_angle < 0.5) return false;`  
两个向量的夹角不能超过 60°。运动角度太大了。

1.3.2.5.2.4.8.14.529、返回 真： `return true;`

1.3.2.5.2.4.8.14.538、 `if (normal_en)`

利用相机模型，参考块第 0 层塔的像素，参考块系下视觉点的位置，参考块系下视觉点的法向量，参考块相对当前帧的变换阵 T，参考块层级等计算当前帧与参考块之间单应阵，再根据单应阵计算仿射变换 `A_cur_ref_zero`，并根据变换获得最佳的搜索层级 `search_level` 最大是 2.

1.3.2.5.2.4.8.14.541、计算 参考特征所在相机系下 该视觉点的法向量

`V3D norm_vec = (ref_ftr->T_f_w_.rotation_matrix() * pt->normal_).normalized();`

1.3.2.5.2.4.8.14.552、计算参考特征所在相机系下 该点的位置： `V3D pf(ref_ftr->T_f_w_ * pt->pos_);`

1.3.2.5.2.4.8.14.563、计算 参考特征相对于 当前帧的 位姿变换： `SE3 T_cur_ref = new_frame_->T_f_w_ * ref_ftr->T_f_w_.inverse();`

1.3.2.5.2.4.8.14.574、 `getWarpMatrixAffineHomography(*cam, ref_ftr->px_, pf, norm_vec, T_cur_ref, 0, A_cur_ref_zero);`

仿射变换矩阵（因为平面法向量可用，所以可根据单应阵获得仿射变换阵，无需假设平面上的点深度一致。）对应函数： `void VIOManager::getWarpMatrixAffineHomography(const vk::AbstractCamera &cam, const V2D &px_ref, const V3D &xyz_ref, const V3D &normal_ref, const SE3 &T_cur_ref, const int level_ref, Matrix2d &A_cur_ref)` 获得能够建立两个图像之间对应关系的仿射变换 `A_cur_ref_zero`

#### 1.3.2.5.2.4.8.14.581、构造一个单应性矩阵

#### 1.3.2.5.2.4.8.14.591、获得参考到当前变换逆的平移向量

```
const V3D t = T_cur_ref.inverse().translation();
```

#### 1.3.2.5.2.4.8.14.602、计算单应性阵 $H_{c_r} = R_{c_r} * (n_r * p_r) * I - t * n^T$ 。没有考虑内参，且两边都乘以了平面参数 $-d = n_r * p_r$

$$A_r^i = P \left( I_i R_{I_r} + I_i t_{I_r} \frac{1}{I_r n^T \cdot I_r p} I_r n^T \right) P^{-1}$$

```
const Eigen::Matrix3d H_cur_ref = T_cur_ref.rotation_matrix() *
(normal_ref.dot(xyz_ref) * Eigen::Matrix3d::Identity() - t *
normal_ref.transpose());
```

#### 1.3.2.5.2.4.8.14.612、初始化块尺寸的一半

```
const int kHalfPatchSize = 4;
```

#### 1.3.2.5.2.4.8.14.623、视觉地图点的投影点加上一半块偏移再投影到世界系下：最右边一个点和最下面一个点（偏移点）。

```
V3D f_du_ref(cam.cam2world(px_ref + Eigen::Vector2d(kHalfPatchSize, 0)
* (1 << level_ref))); V3D f_dv_ref(cam.cam2world(px_ref +
Eigen::Vector2d(0, kHalfPatchSize) * (1 << level_ref)));
```

#### 1.3.2.5.2.4.8.14.634、利用单应阵将参考块系下的点投影的当前帧下

```
const V3D f_cur(H_cur_ref * xyz_ref);
```

#### 1.3.2.5.2.4.8.14.645、同理偏移点也投影到当前帧下

```
const V3D f_du_cur = H_cur_ref * f_du_ref; const V3D f_dv_cur =
H_cur_ref * f_dv_ref;
```

#### 1.3.2.5.2.4.8.14.656、获得在当前帧下三个对应的像素

```
V2D px_cur(cam.world2cam(f_cur)); V2D
px_du_cur(cam.world2cam(f_du_cur)); V2D
px_dv_cur(cam.world2cam(f_dv_cur));
```

#### 1.3.2.5.2.4.8.14.667、计算仿射变换阵 $A_{cur\_ref}$

```
A_cur_ref.col(0) = (px_du_cur - px_cur) / kHalfPatchSize; A_cur_ref.col(1)
= (px_dv_cur - px_cur) / kHalfPatchSize;
```

#### 1.3.2.5.2.4.8.14.675、获得最佳的搜索层级

```
search_level = getBestSearchLevel(A_cur_ref_zero, 2); 对应函数： int
VIOManager::getBestSearchLevel(const Matrix2d &A_cur_ref, const int
max_level) 声明于 vio.h， 定义于 vio.cpp 根据仿射变换的行列式值来决定
搜索层级。最大是 2， 比如行列式值为 16 时（参考块到当前块需放大 16 倍），
search_level 为 2。若放大倍数小于 3， search_level 就为 0（原图像）。
```

#### 1.3.2.5.2.4.8.14.681、设置初始搜索层级为 0

```
int search_level = 0;
```

#### 1.3.2.5.2.4.8.14.692、计算仿射变换的行列式

`double D = A_cur_ref.determinant();`行列式在几何上反映了变换的缩放比例：如果 `[D]`(http:) 大于 1，表示该变换会放大区域；如果小于 1，则表示缩小；

#### 1.3.2.5.2.4.8.14.703、循环若当前行列式值大于 3.0 且 搜索层级没到达最大 2（直至有一方满足）

行列式  $\det(A_{cur\_ref})$  表示变换对局部面积的缩放比例：

- $\det > 0$ : 保持方向（无镜像），面积缩放比例为  $|\det|$ 。
- $\det < 0$ : 包含镜像翻转（方向反转），面积缩放比例为  $|\det|$ 。
- $\det = 0$ : 变换将空间压缩到低维（如直线或点）。

具体场景中的意义

1. 图像对齐：
  - 若  $\det \approx 1$ ，表示当前帧与参考帧的局部区域形状几乎不变（仅旋转或平移）。
  - 若  $\det \ll 1$  或  $\gg 1$ ，表示存在显著缩放（如相机远离或靠近平面）。
2. 退化检测：
  - 当  $\det \approx 0$ ，可能意味着视角过于倾斜或特征点远离平面假设，导致跟踪失败。

```
while (D > 3.0 && search_level < max_level) {search_level += 1; D *= 0.25; }
```

#### 1.3.2.5.2.4.8.14.719、否则 ----可略过

fast-livo 采用的吧。若在扭曲图 `warp_map` 中能找到参考块 `id` 对应扭曲的仿射阵和最佳层级就给 `A_cur_ref_zero` 和 `search_level`。所示没有，就假设深度一致，计算出一个仿射阵 `A_cur_ref_zero` 和最佳搜索层级 `search_level`，根据该层级获得一个新扭曲对象并赋予扭曲图中该参考块 `id` 位置 `warp_map[ref_ftr->id_]`。`warp_map[ref_ftr->id_]`在 `warp_map` 没有对应位置的扭曲时，需要新生成一个

#### 1.3.2.5.2.4.8.14.721、根据参考块对应的 id 找到扭曲图对应的哈希索引：auto iter\_warp =

```
warp_map.find(ref_ftr->id_);
```

#### 1.3.2.5.2.4.8.14.732、判断 该索引是否有效：if (iter\_warp != warp\_map.end())

根据扭曲对象 `iter_warp->second` 获得最佳搜索层级 `search_level` 和仿射阵 `A_cur_ref_zero`

#### 1.3.2.5.2.4.8.14.741、将该索引对应的扭曲对象的搜索层级给最佳搜索层级：search\_level =

```
iter_warp->second->search_level;
```

#### 1.3.2.5.2.4.8.14.752、将该索引对应的扭曲对象的仿射阵给仿射阵：A\_cur\_ref\_zero = iter\_warp->

```
second->A_cur_ref;
```

#### 1.3.2.5.2.4.8.14.763、否则 扭曲图没有该索引

没有法向量，没有计算单应阵，假设深度一致，计算出一个仿射阵和最佳搜索层级，根据该层级获得一个新扭曲对象并赋予扭曲图中该参考块 `id` 位置 `warp_map[ref_ftr->id_]`。

1.3.2.5.2.4.8.14.771、计算仿射变换阵（不依赖单应阵） `getWarpMatrixAffine(*cam, ref_ftr->px_, ref_ftr->f_, (ref_ftr->pos() - pt->pos_).norm(), new_frame_->T_f_w_ * ref_ftr->T_f_w_.inverse(),ref_ftr->level_, 0, patch_size_half, A_cur_ref_zero);` 传入相机模型，参考块在第 0 层金字塔上的像素，块的单位方向向量，（世界系）点到块的向量的模长（深度），参考块相对当前帧的变换阵，参考块层级（补丁特征被提取时所在的图像金字塔层级），金字塔层级设为 0，一半的块尺寸，待计算的仿射阵。

```
void VIOManager::getWarpMatrixAffine(const vk::AbstractCamera &cam,
const Vector2d &px_ref, const Vector3d &f_ref, const double
depth_ref,const SE3 &T_cur_ref, const int level_ref, const int
pyramid_level, const int halfpatch_size, Matrix2d &A_cur_ref)
```

1.3.2.5.2.4.8.14.781、获得在参考块下视觉点的位置： `const Vector3d xyz_ref(f_ref * depth_ref);`

1.3.2.5.2.4.8.14.792、计算两个偏移像素点对应的世界点 `xyz_du_ref`, `xyz_dv_ref`

```
Vector3d xyz_du_ref(cam.cam2world(px_ref + Vector2d(halfpatch_size, 0)
* (1 << level_ref) * (1 << pyramid_level))); Vector3d
xyz_dv_ref(cam.cam2world(px_ref + Vector2d(0, halfpatch_size) * (1 <<
level_ref) * (1 << pyramid_level)));
```

1.3.2.5.2.4.8.14.803、假设这两个偏移点与视觉点深度一致（有单应阵时可以得到对应的深度，无需假设）

```
xyz_du_ref *= xyz_ref[2] / xyz_du_ref[2]; xyz_dv_ref *= xyz_ref[2] /
xyz_dv_ref[2];
```

1.3.2.5.2.4.8.14.814、将这三个点都转到当前帧下

```
const Vector2d px_cur(cam.world2cam(T_cur_ref * (xyz_ref)));const
Vector2d px_du(cam.world2cam(T_cur_ref * (xyz_du_ref)));const Vector2d
px_dv(cam.world2cam(T_cur_ref * (xyz_dv_ref)));
```

1.3.2.5.2.4.8.14.825、计算仿射阵

```
A_cur_ref.col(0) = (px_du - px_cur) / halfpatch_size;A_cur_ref.col(1) =
(px_dv - px_cur) / halfpatch_size;
```

1.3.2.5.2.4.8.14.832、根据仿射阵获得最佳的搜索层级： `search_level =`

```
getBestSearchLevel(A_cur_ref_zero, 2);
```

1.3.2.5.2.4.8.14.843、根据 仿射阵和最佳搜索层级 生成一个新的扭曲类对象： `Warp *ot = new Warp(search_level, A_cur_ref_zero);`

1.3.2.5.2.4.8.14.854、将这个对象赋予到扭曲图该参考块对应 id 位置中： `warp_map[ref_ftr->id_] = ot;`

1.3.2.5.2.4.8.14.8610、遍历，从 0 层到最高层（总计四层）

```
for (int pyramid_level = 0; pyramid_level <= patch_pyrimid_level - 1;
pyramid_level++) { warpAffine(A_cur_ref_zero, ref_ftr->img_, ref_ftr->px_,
ref_ftr->level_, search_level, pyramid_level, patch_size_half,
```

```
patch_wrap.data()); } 对应函数: ``void VIOManager::warpAffine(const Matrix2d &A_cur_ref, const cv::Mat &img_ref, const Vector2d &px_ref, const int level_ref, const int search_level, const int pyramid_level, const int halfpatch_size, float *patch)``根据搜索层级和当前金字塔层级, 对当前帧像素块进行放大, 将当前帧像素块投影到参考图像上, 获得以参考块中心为中心的投影块, 判断投影块中的每个像素是否越界, 越界的置 0, 未越界的用参考图像对该位置进行插值, 最终获得在参考图像上各层级扭曲块的像素值 patch_ptr、patch。patch_wrap。
```

#### 1.3.2.5.2.4.8.14.871、获得图像块尺寸

```
const int patch_size = halfpatch_size * 2;
```

#### 1.3.2.5.2.4.8.14.882、获得当前帧到参考块的仿射变换

```
const Matrix2f A_ref_cur = A_cur_ref.inverse().cast<float>();
```

#### 1.3.2.5.2.4.8.14.893、判断仿射变换是否正常: if (isnan(A\_ref\_cur(0, 0)))

```
if (isnan(A_ref_cur(0, 0))) {printf("Affine warp is NaN, probably camera has no translation\n"); // TODOreturn; }
```

#### 1.3.2.5.2.4.8.14.904、初始化指向块扭曲向量 **patch\_wrap** 第一元素的指针: float \***patch\_ptr** = patch;

#### 1.3.2.5.2.4.8.14.915、遍历整个块:

```
for (int y = 0; y < patch_size; ++y) {for (int x = 0; x < patch_size; ++x) //, ++patch_ptr) {根据搜索层级和当前循环层级, 对当前帧像素块进行放大, 将当前帧像素块投影到参考图像上, 获得以参考块中心为中心的投影块, 判断投影块中的每个像素是否越界, 越界的置 0, 未越界的用参考图像对该位置进行插值, 最终获得在参考图像上各层级扭曲块的像素值 patch_ptr、patch。
```

#### 1.3.2.5.2.4.8.14.921、从左上角开始往右遍历块偏移像素坐标

```
Vector2f px_patch(x - halfpatch_size, y - halfpatch_size);
```

#### 1.3.2.5.2.4.8.14.932、块偏移像素坐标放大到对应的搜索层级

```
px_patch *= (1 << search_level);先按搜索层级进行放大, 参考块相对于当前图像块的第 search_level, 放大后获得了当前图像块对应的 px_patch 坐标。
```

#### 1.3.2.5.2.4.8.14.943、再将块偏移像素坐标按当前金字塔层级进行放大

```
px_patch *= (1 << pyramid_level);假设当前图像块是当前帧的第 pyramid_level 层, 按当前层再次放大, 获得在第 0 层上的 px_patch 坐标。
```

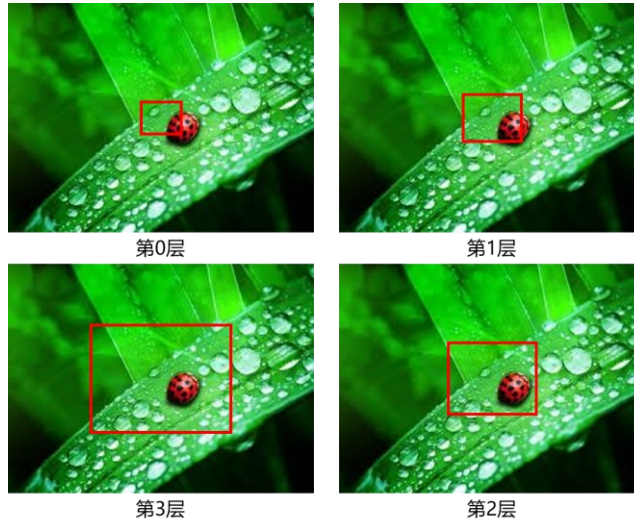
#### 1.3.2.5.2.4.8.14.954、参考块的中心为原点, 参考块中心坐标+映射到参考块上的偏移

```
const Vector2f px(A_ref_cur * px_patch + px_ref.cast<float>());相当于将第 0 层 px_patch 映射到参考块上, 作为参考块中心的偏移。(笔者还有一种理解, A_ref_cur * px_patch 时, 前面做的 (1 << search_level)相当于又 (1 >> search_level)了, 但 px_patch 还有(1 << pyramid_level)部分, 相当于参考块为第 pyramid_level, 按当前层数 pyramid_level 进行放大, 还原到参考图像 0 层上。)
```

1.3.2.5.2.4.8.14.965、如果映射过来的块中像素超出了参考图像边界，则当前层像素赋值为 0。

```
if (px[0] < 0 || px[1] < 0 || px[0] >= img_ref.cols - 1 || px[1] >= img_ref.rows - 1) patch_ptr[patch_size_total * pyramid_level + y * patch_size + x] = 0; patch_ptr 是一个一维的向量，因此 patch_size_total * pyramid_level + y * patch_size + x，表示当前层还原到 0 层后在 patch_ptr 中对应的像素索引。
```

1.3.2.5.2.4.8.14.976、若未超出参考图像边界则在参考图像该像素位置插值像素值



```
patch_ptr[patch_size_total * pyramid_level + y * patch_size + x] = (float)vk::interpolateMat_8u(img_ref, px[0], px[1]);
```

Fast-LIVO2 似乎并没有采用传统金字塔的采样方式，而是在不损害图像分辨率的基础上在原图上进行图像块范围放大，如图。

1.3.2.5.2.4.8.14.9811、`getImagePatch(img, pc, patch_buffer.data(), 0);`

传入当前图像 **img**，当前栅格视觉地图点对应的像素坐标 **pc**，块缓存器（64 个像素），第 0 层（原图像）。对块的每个像素进行插值填充（第 0 层图像块）。对应函数：`void VIOManager::getImagePatch(cv::Mat img, V2D pc, float *patch_tmp, int level)` `patch_buffer` 长度与一个块的像素数量一致，在 VIO 初始化函数中初始化。

1.3.2.5.2.4.8.14.991、图像块中心的横 **u\_ref** 纵 **v\_ref** 浮点坐标。

```
const float u_ref = pc[0]; const float v_ref = pc[1];
```

1.3.2.5.2.4.8.14.1002、金字塔层数对应的缩放因子：

```
const int scale = (1 << level);
```

1.3.2.5.2.4.8.14.1013、获得当前层数的整数中心坐标 **u\_ref\_i** **v\_ref\_i**

```
const int u_ref_i = floorf(pc[0] / scale) * scale; const int v_ref_i = floorf(pc[1] / scale) * scale;
```

1.3.2.5.2.4.8.14.1024、获得实际浮点坐标与整数坐标的偏移：`subpix_u_ref` `subpix_v_ref`

```
const float subpix_u_ref = (u_ref - u_ref_i) / scale; const float subpix_v_ref = (v_ref - v_ref_i) / scale;
```

1.3.2.5.2.4.8.14.1035、左上、右上、左下、右下四个邻域像素的插值权重，确保四个权重之和为 1

```
const float w_ref_tl = (1.0 - subpix_u_ref) * (1.0 - subpix_v_ref);const float
w_ref_tr = subpix_u_ref * (1.0 - subpix_v_ref);const float w_ref_bl = (1.0 -
subpix_u_ref) * subpix_v_ref;const float w_ref_br = subpix_u_ref *
subpix_v_ref;
```

1.3.2.5.2.4.8.14.1046、对块进行循环（双重遍历）for (int x = 0; x < patch\_size; x++) 这个是块的每行  
基于左上、右上、左下、右下四个邻域像素对块的每个像素进行插值填充。

1.3.2.5.2.4.8.14.1051、以图像第一个像素为起点，获得当前遍历块第一行最左边像素位于图像数据  
向量中的位置指针

```
uint8_t *img_ptr = (uint8_t *)img.data + (v_ref_i - patch_size_half * scale +
x * scale) * width + (u_ref_i - patch_size_half * scale);
```

1.3.2.5.2.4.8.14.1062、for (int y = 0; y < patch\_size; y++, img\_ptr += scale) 遍历块的每列

逐个将块的像素填充。基于左上、右上、左下、右下四个邻域像素值及其权重插值。patch\_tmp[patch\_size\_total \* level + x \* patch\_size + y]  
=w\_ref\_tl \* img\_ptr[0] + w\_ref\_tr \* img\_ptr[scale] + w\_ref\_bl \*  
img\_ptr[scale \* width] + w\_ref\_br \* img\_ptr[scale \* width + scale];

1.3.2.5.2.4.8.14.10712、初始化一个误差：float error = 0.0;

1.3.2.5.2.4.8.14.10813、从 patch\_wrap 取出第 0 层的参考块像素值与当前帧对应的块像素值求差平方和，且考虑逆曝光时间的影响 **error**

```
for (int ind = 0; ind < patch_size_total; ind++), { error += (ref_ftr->
inv_expo_time_ * patch_wrap[ind] - state->inv_expo_time_ *
patch_buffer[ind]) * (ref_ftr->inv_expo_time_ * patch_wrap[ind] - state->
inv_expo_time_ * patch_buffer[ind]);}
```

1.3.2.5.2.4.8.14.10914、判断 如果 ncc 使能 则计算 ncc 值 if (ncc\_en)-----可略过

用于计算参考块与当前帧的 ncc。小于 ncc 阈值则跳过该栅格单元。  
(并没有计算分数)

1.3.2.5.2.4.8.14.1101、计算 ncc 传入各层扭曲块的像素（参考图像），当前图像块像素，块的总像素数。

$$\text{NCC}(\mathbf{f}, \mathbf{g}) = \frac{\sum_{x,y} [\mathbf{f}(x,y) - \bar{\mathbf{f}}] [\mathbf{g}(x,y) - \bar{\mathbf{g}}]}{\sqrt{\sum_{x,y} [\mathbf{f}(x,y) - \bar{\mathbf{f}}]^2 \sum_{x,y} [\mathbf{g}(x,y) - \bar{\mathbf{g}}]^2}}$$
$$c = \frac{\mathbf{n} \cdot \mathbf{p}}{\|\mathbf{p}\|}, \quad \omega_1 = \frac{1}{1 + e^{u(\Sigma_n)}}$$
$$S = (1 - \omega_1) \cdot \frac{1}{n} \sum_{i=1}^n \text{NCC}(\mathbf{f}, \mathbf{g}_i) + \omega_1 \cdot c \quad (12)$$

```
double ncc = calculateNCC(patch_wrap.data(), patch_buffer.data(),
patch_size_total);
```

1.3.2.5.2.4.8.14.1111、计算参考块第 0 层的平均像素值 **mean\_ref**

```
double sum_ref = std::accumulate(ref_patch, ref_patch + patch_size,
0.0);double mean_ref = sum_ref / patch_size;
```

1.3.2.5.2.4.8.14.1122、计算当前块的平均像素值 `mean_curr`

```
double sum_cur = std::accumulate(cur_patch, cur_patch + patch_size, 0.0); double mean_curr = sum_cur / patch_size;
```

1.3.2.5.2.4.8.14.1133、初始化三个量：

```
double numerator = 0, demoniator1 = 0, demoniator2 = 0;
```

1.3.2.5.2.4.8.14.1144、循环 块中所有像素 `for (int i = 0; i < patch_size; i++)`

1.3.2.5.2.4.8.14.1151、计算 与各自均值的差再相乘：`double n = (ref_patch[i] - mean_ref) * (cur_patch[i] - mean_curr);`

1.3.2.5.2.4.8.14.1162、将其累加到 `numerator`：`numerator += n;`

1.3.2.5.2.4.8.14.1173、各自均值差的平方再累加到：`demoniator1, demoniator2`

```
demoniator1 += (ref_patch[i] - mean_ref) * (ref_patch[i] - mean_ref); demoniator2 += (cur_patch[i] - mean_curr) * (cur_patch[i] - mean_curr);
```

1.3.2.5.2.4.8.14.1185、计算 `NCC`：`return numerator / sqrt(demoniator1 * demoniator2 + 1e-10);`

1.3.2.5.2.4.8.14.1192、判断 小于 `ncc` 阈值则跳过该栅格单元。（并没有计算分数）

1.3.2.5.2.4.8.14.12015、判断 误差是否大于离群点阈值\*块总像素数 是则跳过该栅格单元

```
if (error > outlier_threshold * patch_size_total) continue;
```



1.3.2.5.2.4.8.14.12116、对视觉子图 **visual\_submap** 各属性进行更新

1.3.2.5.2.4.8.14.1221、推入该栅格对应的体素点到 voxel\_points: visual\_submap-  
>voxel\_points.push\_back(pt);

1.3.2.5.2.4.8.14.1232、推入该栅格对应的误差到 传播误差? propa\_errors: visual\_submap-  
>propa\_errors.push\_back(error);

1.3.2.5.2.4.8.14.1243、推入该栅格对应的搜索层到 search\_levels: visual\_submap-  
>search\_levels.push\_back(search\_level);

1.3.2.5.2.4.8.14.1254、推入该栅格对应的误差到 errors: visual\_submap->errors.push\_back(error);

1.3.2.5.2.4.8.14.1265、推入该栅格对应的各层的扭曲块像素值到 warp\_patch: visual\_submap-  
>warp\_patch.push\_back(patch\_wrap);

1.3.2.5.2.4.8.14.1276、推入该栅格对应的参考特征的逆曝光时间到 inv\_expo\_list: visual\_submap-  
>inv\_expo\_list.push\_back(ref\_ftr->inv\_expo\_time\_);

1.3.2.5.2.4.8.15 14、该图所有栅格的总点数: **total\_points** = visual\_submap-  
>voxel\_points.size();

**1.3.2.5.2.4.9 记录时间: double t2 = omp\_get\_wtime();**

**1.3.2.5.2.4.10 8、computeJacobianAndUpdateEKF(img);**

基于当前图像计算雅可比并更新 EKF: 对应函数: void  
VIOManager::computeJacobianAndUpdateEKF(cv::Mat img)如果第一次运行  
retrieveFromVisualSparseMap 因 vio 管理器的 feat\_map 未赋值, 而直接返回了,  
那么 total\_points 就是 0, 这个函数也会返回。循环计算多层直接法的光度误差  
雅可比阵 H, 测量向量, 并根据 EKF 更新公式对状态, 协方差阵进行更新, 更  
新一下世界系到当前帧的旋转和平移。new\_frame\_ ->T\_f\_w\_ = SE3(Rcw, Pcw);

1.3.2.5.2.4.10.1 1、判断 所有栅格对应体素点是否为空

if (total\_points == 0) return;

1.3.2.5.2.4.10.2 2、初始化计算所有时间

compute\_jacobian\_time = update\_ekf\_time = 0.0;

1.3.2.5.2.4.10.3 3、遍历金字塔, 从最顶层开始到原图: for (int level =  
patch\_pyramid\_level - 1; level >= 0; level--)

循环计算多层直接法的光度误差雅可比阵 H, 测量向量, 并根据 EKF 更新  
公式对状态进行更新, 协方差阵没有更新

1.3.2.5.2.4.10.3.1 1、判断是否 逆补偿使能 if (inverse\_composition\_en)-----可略过

1.3.2.5.2.4.10.3.2 1、拥有参考块缓存标志位设为 false: has\_ref\_patch\_cache = false;

1.3.2.5.2.4.10.3.3 2、更新状态逆: updateStateInverse(img, level);

与 updateState 区别在于残差计算那里, 用四个像素 01/23 插值出来的像素值-该块实际值, 雅可比的计算也非直接法, 未包含逆曝光时间。JdR = J\_dR \* Rwi + J\_dt \* P\_wi\_hat \* Rwi; Jdt = J\_dt \* Rwi;

1.3.2.5.2.4.10.3.4 2、更新状态: updateState(img, level);

void VIOManager::updateState(cv::Mat img, int level)采用稀疏直接法----多层直接法 (从粗到细的计算策略)

1.3.2.5.2.4.10.3.5 1、一些量的初始化, 每个变量的含义

```
if (total_points == 0) return; // 判断是否有可用当前视野内的视觉地图点
(与栅格单元关联的) StatesGroup old_state = (*state); // 用当前状态初始一个旧状态
VectorXd z; // 代表观测量 MatrixXd H_sub; // 代表观测的 H 阵
bool EKF_end = false; // EKF 结束标志位 float last_error = std::numeric_limits<float>::max(); // 最终误差
const int H_DIM = total_points * patch_size_total; // H 阵维度 (所有块加到一起的像素数): 总点数*块像素量
z.resize(H_DIM); // 观测量容器大小设置 z.setZero(); // 观测量结果设置
H_sub.resize(H_DIM, 7); // 观测阵 大小设置 H_sub.setZero(); // 初始化观测阵
```

1.3.2.5.2.4.10.3.6 \*关于稀疏直接法的理解

下面第 2 项的代码实现可以参考该理解中的

1.3.2.5.2.4.10.3.7 1、直接法的逻辑

特征点匹配法是知道对应点, 然后用重投影误差来优化位姿。直接法是不知道对应点的, 因此逻辑上是反过来的: 先给一个位姿初始值 (LIO 提供的状态), 当前图像基于这个状态估计它在另一个图像上的对应点, 计算两个点的光度误差来优化位姿 (基于光度不变假设)。如下: 如果先认为相机右转了 15°, 那么光流能不能以这个 15°运动作为初始值的假设, 调整光流的计算结果呢?直接法就是遵循这样的思路得到的结果。

1.3.2.5.2.4.10.3.8 2、考虑一个空间三维点

$$e = I_1(p_1) - I_2(p_2)$$

已知空间三维点及其对应的当前图像像素 p1, 和当前图像状态。根据当前图像状态取寻找对应点 p2, 如果状态不精确, 就最小化两个点的光度误差 e 来精细化状态。

1.3.2.5.2.4.10.3.9 3、考虑 N 个空间三维点

$$\min_T J(T) = \sum_{i=1}^N e_i^T e_i, \quad e_i = I_1(p_{1,i}) - I_2(p_{2,i})$$

基于位姿 T 获得 N 个点之间的光度误差这里的优化变量就是相机位姿 T

1.3.2.5.2.4.10.3.104、位姿  $T$  与误差存在什么样的变化关系（误差是如何随相机位姿  $T$  变化的--导数）

1.3.2.5.2.4.10.3.111、根据状态获得当前图像系下的三维点在参考系下的三维坐标  $q$  和相应的像素坐标  $u$

$$u = \frac{1}{Z_2} K q.$$

1.3.2.5.2.4.10.3.122、对误差进行一阶泰勒展开, 获得关于位姿误差的一阶导数（链式法则）

$$e(T) = I_1(p_1) - I_2(u),$$

$$\frac{\partial e}{\partial T} = \frac{\partial I_2}{\partial u} \frac{\partial u}{\partial q} \frac{\partial q}{\partial \delta \xi} \delta \xi,$$

1.3.2.5.2.4.10.3.131、像素对该处坐标  $u$  的导数：像素梯度

$$\partial I_2 / \partial u$$

1.3.2.5.2.4.10.3.142、投影得到的像素点  $u$  对相机坐标系下三维点  $q$  的导数

$$\partial u / \partial q$$

1.3.2.5.2.4.10.3.15具体形式

$$\frac{\partial u}{\partial q} = \begin{bmatrix} \frac{\partial u}{\partial X} & \frac{\partial u}{\partial Y} & \frac{\partial u}{\partial Z} \\ \frac{\partial v}{\partial X} & \frac{\partial v}{\partial Y} & \frac{\partial v}{\partial Z} \end{bmatrix} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} \end{bmatrix}$$

1.3.2.5.2.4.10.3.163、当前相机系下的三维点  $q$  对误差状态的导数

$$\partial q / \partial \delta \xi$$

1.3.2.5.2.4.10.3.17具体形式

$$\frac{\partial q}{\partial \delta \xi} = [I, -q^\wedge].$$

1.3.2.5.2.4.10.3.184、对后两项进行整合（因为只与三维点  $\mathbf{q}$  有关）

$$\frac{\partial \mathbf{u}}{\partial \delta \xi} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & -\frac{f_x XY}{Z^2} & f_x + \frac{f_x X^2}{Z^2} & -\frac{f_x Y}{Z} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & -f_y - \frac{f_y Y^2}{Z^2} & \frac{f_y XY}{Z^2} & \frac{f_y X}{Z} \end{bmatrix}.$$

1.3.2.5.2.4.10.3.193、基于 2，可得光度误差关于状态误差的雅可比阵

$$\mathbf{J} = -\frac{\partial I_2}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \delta \xi}.$$

1.3.2.5.2.4.10.3.202、进行迭代更新，最大迭代次数 5:

```
for (int iteration = 0; iteration < max_iterations; iteration++) 计算光度误差
雅可比阵 H，测量向量，并根据 EKF 更新公式对状态进行更新，协方
差阵没有更新
```

1.3.2.5.2.4.10.3.21 记录时间: double t1 = omp\_get\_wtime();

1.3.2.5.2.4.10.3.221、IMU 系相对世界系的位姿: Rwi, Pwi

```
M3D Rwi(state->rot_end); V3D Pwi(state->pos_end);
```

1.3.2.5.2.4.10.3.232、世界系到当前图像相机系的位姿变换:

```
Rcw = Rci * Rwi.transpose(); Pcw = -Rci * Rwi.transpose() * Pwi + Pci;
```

1.3.2.5.2.4.10.3.243、雅可比阵--用于后续

```
M3D Jdp_dt = Rci * Rwi.transpose();
```

1.3.2.5.2.4.10.3.254、初始化误差 error 和均值 n\_meas :

```
float error = 0.0; int n_meas = 0;
```

1.3.2.5.2.4.10.3.265、设置并行运算执行下面的 for

```
#ifndef MP_ENomp_set_num_threads(MP_PROC_NUM); #pragma omp
parallel for reduction(+:error, n_meas) #endif
```

1.3.2.5.2.4.10.3.276、遍历所有栅格单元的体素点

```
for (int i = 0; i < total_points; i++) 用于计算每个视觉地图点对应块的每个
像素光度观测雅可比矩阵 H**, 光度误差测量向量 z**, 每个块的误差
平方和 visual_submap->errors[i] = patch_error; **, 所有体素的块光度误
差和 error
```

1.3.2.5.2.4.10.3.281、初始化六个雅可比系数阵:

```
MD(1, 2) Jim; MD(2, 3) Jdpi; // 像素对三维点的导数 MD(1, 3) Jdphi, Jdp,
JdR, Jdt;
```

1.3.2.5.2.4.10.3.292、初始化块误差

```
float patch_error = 0.0;
```

1.3.2.5.2.4.10.3.303、获得该点对应的最佳搜索层级（在参考图像块相对与当前图像的层级）

```
int search_level = visual_submap->search_levels[i];
```

1.3.2.5.2.4.10.3.314、整个金字塔层级：当前循环的金字塔层级+最佳搜索层级

```
int pyramid_level = level + search_level;
```

1.3.2.5.2.4.10.3.325、获得缩放尺度

```
int scale = (1 << pyramid_level);
```

1.3.2.5.2.4.10.3.336、逆缩放尺度

```
float inv_scale = 1.0f / scale;
```

1.3.2.5.2.4.10.3.347、获得该视觉点

```
VisualPoint *pt = visual_submap->voxel_points[i];
```

1.3.2.5.2.4.10.3.358、转到当前相机系和当前图像像素系下

```
V3D pf = Rcw * pt->pos_ + Pcw; V2D pc = cam->world2cam(pf);
```

1.3.2.5.2.4.10.3.369、computeProjectionJacobian(pf, Jdpi);

$$\partial \mathbf{u} / \partial \mathbf{q}$$

计算投影雅可比矩阵，传入相机系 **pf** 下坐标和一个 2\*3 雅可比阵 **Jdpi**。  
对应函数：void VIOManager::computeProjectionJacobian(V3D p, MD(2, 3) & J)声明于 vio.h，定义于 vio.cpp

1.3.2.5.2.4.10.3.371、获得相机系下该点的 **xy** 坐标：

```
const double x = p[0];const double y = p[1];
```

1.3.2.5.2.4.10.3.382、获得该点的逆深度及平方

```
const double z_inv = 1. / p[2];const double z_inv_2 = z_inv * z_inv;
```

1.3.2.5.2.4.10.3.393、计算雅可比

```
J(0, 0) = fx * z_inv;J(0, 1) = 0.0;J(0, 2) = -fx * x * z_inv_2;J(1, 0) = 0.0;J(1, 1) = fy * z_inv;J(1, 2) = -fy * y * z_inv_2;
```

1.3.2.5.2.4.10.3.4010、计算当前相机系下三维点的反对称矩阵：**p\_hat**

```
M3D p_hat; p_hat << SKEW_SYM_MATRX(pf);
```

1.3.2.5.2.4.10.3.4111、将该体素点对应的像素点作为块中间，获得实际像素坐标：**u\_ref, v\_ref**

```
float u_ref = pc[0];float v_ref = pc[1];
```

1.3.2.5.2.4.10.3.4212、计算得到 像素平面坐标：**u\_ref\_i, v\_ref\_i**

```
int u_ref_i = floorf(pc[0] / scale) * scale;int v_ref_i = floorf(pc[1] / scale) *
```

scale;

1.3.2.5.2.4.10.3.4313、subpix\_u\_ref , subpix\_v\_ref 用于下面的权重计算

```
float subpix_u_ref = (u_ref - u_ref_i) / scale; float subpix_v_ref = (v_ref - v_ref_i) / scale;
```

1.3.2.5.2.4.10.3.4414、计算左上，右上，左下，右下权重

```
float w_ref_tl = (1.0 - subpix_u_ref) * (1.0 - subpix_v_ref); float w_ref_tr = subpix_u_ref * (1.0 - subpix_v_ref); float w_ref_bl = (1.0 - subpix_u_ref) * subpix_v_ref; float w_ref_br = subpix_u_ref * subpix_v_ref;
```

1.3.2.5.2.4.10.3.4515、获得第 i 个栅格单元对应体素点的参考块像素向量。

```
vector<float> P = visual_submap->warp_patch[i];
```

1.3.2.5.2.4.10.3.4616、参考块的逆曝光时间

```
double inv_ref_expo = visual_submap->inv_expo_list[i];
```

1.3.2.5.2.4.10.3.4717、遍历整个块像素 首先遍历行： for (int x = 0; x < patch\_size; x++)

与一般像素梯度计算（十字形）不同的是，SVO 和 Fast-livo 系列采用的是 品，利用四个像素插值出一个特征点然后再计算。 获得光度误差的观测雅可比矩阵  $\mathbf{H}$ ，光度误差测量向量  $\mathbf{z}$ ，每个块的误差平方和 patch\_error

1.3.2.5.2.4.10.3.481、获得块 每行最左边的像素

```
uint8_t *img_ptr = (uint8_t *)img.data + (v_ref_i + x * scale - patch_size_half * scale) * width + u_ref_i - patch_size_half * scale;
```

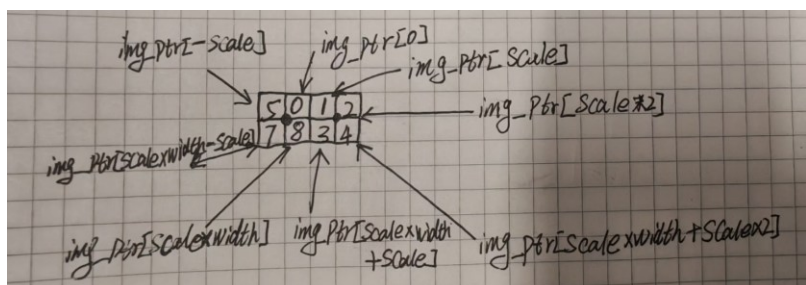
1.3.2.5.2.4.10.3.492、遍历列

$$e(\mathbf{T}) = \mathbf{I}_1(\mathbf{p}_1) - \mathbf{I}_2(\mathbf{u}),$$

$$\frac{\partial e}{\partial \mathbf{T}} = \frac{\partial \mathbf{I}_2}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{q}} \frac{\partial \mathbf{q}}{\partial \delta \xi} \delta \xi,$$

for (int y = 0; y < patch\_size; ++y, img\_ptr += scale) 这部分数学推导可以参考论文解析中的光度误差雅可比推导。

1.3.2.5.2.4.10.3.501、计算在原图横向上的像素变化 du （提前乘以 0.5，类似将 SVO 论文中最小化函数前的 0.5 拿了进来）



```
float du = 0.5f * ((w_ref_tl * img_ptr[scale] + w_ref_tr *
```

```
img_ptr[scale * 2] + w_ref_bl * img_ptr[scale * width + scale] +
w_ref_br * img_ptr[scale * width + scale * 2]) - (w_ref_tl *
img_ptr[-scale] + w_ref_tr * img_ptr[0] + w_ref_bl * img_ptr[scale * width -
scale] + w_ref_br * img_ptr[scale * width]));
```

1.3.2.5.2.4.10.3.512、计算原图纵向上的像素变化  $dv$  -----与第一项类似。

```
float dv = 0.5f * ((w_ref_tl * img_ptr[scale * width] + w_ref_tr *
img_ptr[scale + scale * width] + w_ref_bl * img_ptr[width * scale * 2] +
w_ref_br * img_ptr[width * scale * 2 + scale]) - (w_ref_tl *
img_ptr[-scale * width] + w_ref_tr * img_ptr[-scale * width + scale] +
w_ref_bl * img_ptr[0] + w_ref_br * img_ptr[scale]));
```

1.3.2.5.2.4.10.3.523、将原图梯度变化给

```
Jimg << du, dv;
```

1.3.2.5.2.4.10.3.534、将原图梯度变化乘上逆曝光时间变化，并缩放到当前所在层数 Jimg：

$$\partial I_2 / \partial u$$

```
Jimg = Jimg * state->inv_expo_time; Jimg = Jimg * inv_scale;
```

1.3.2.5.2.4.10.3.545、将上面两种导数 乘以 相机系下的三维点  $q$  对误差状态的导数，与单位阵相乘的部分为 Jdphi，与点反对称阵相乘的阵为 Jdp

$$\frac{\partial q}{\partial \delta \xi} = [I, -q^\wedge].$$

```
Jdphi = Jimg * Jdpi * p_hat; //旋转 Jdp = -Jimg * Jdpi; // 平移
```

1.3.2.5.2.4.10.3.556、对旋转阵的导数：JdR = Jdphi \* Jdphi\_dR + Jdp \* Jdp\_dR;

```
Jdphi_dR = Rci; Jdp_dR = -Rci * tmp; tmp << SKEW_SYM_MATRX(Pic);
第 6 和第 7 项可以参考论文解析以了解。
```

1.3.2.5.2.4.10.3.567、对平移的导数：Jdt = Jdp \* Jdp\_dt;

```
Jdp_dt = Rci * Rwi.transpose();
```

1.3.2.5.2.4.10.3.578、计算当前的像素值：cur\_value

```
double cur_value = w_ref_tl * img_ptr[0] + w_ref_tr * img_ptr[scale]
+ w_ref_bl * img_ptr[scale * width] + w_ref_br * img_ptr[scale * width +
scale];
```

1.3.2.5.2.4.10.3.589、计算光度误差：res

```
double res = state->inv_expo_time * cur_value - inv_ref_expo *
P[patch_size_total * level + x * patch_size + y];
```

1.3.2.5.2.4.10.3.5910、将此残差值存入观测值向量

```
z(i * patch_size_total + x * patch_size + y) = res; 每个栅格单元对应体素点的
块每个像素值与参考之差。
```

1.3.2.5.2.4.10.3.6011、残差的平方作为块误差累加

```
patch_error += res * res;
```

1.3.2.5.2.4.10.3.6112、记录计算了残差的像素数

```
n_meas += 1;
```

1.3.2.5.2.4.10.3.6213、判断 曝光时间估计使能将当前插值出的像素值作为观测阵的最后一个元素。

```
if (exposure_estimate_en) { H_sub.block<1, 7>(i * patch_size_total + x *
patch_size + y, 0) << JdR, Jdt, cur_value; } ,
```

1.3.2.5.2.4.10.3.6314、否则 就只要 对旋转和平移的雅可比阵

```
else { H_sub.block<1, 6>(i * patch_size_total + x * patch_size + y, 0) <<
JdR, Jdt; }
```

1.3.2.5.2.4.10.3.6418、将每个栅格对应视觉地图点的图像块的 patch\_error 推入该体素点索引对应的  
误差属性

```
visual_submap->errors[i] = patch_error;
```

1.3.2.5.2.4.10.3.6519、累计每个体素点对应的块光度误差： error += patch\_error;

1.3.2.5.2.4.10.3.667、计算所有块每个像素的光度误差均值

```
error = error / n_meas;
```

1.3.2.5.2.4.10.3.67 记录时间; compute\_jacobian\_time += omp\_get\_wtime() - t1;

1.3.2.5.2.4.10.3.688、判断是否小于上次的误差： if (error <= last\_error)

1.3.2.5.2.4.10.3.691、将当前状态给旧状态，将误差给上次误差

```
old_state = (*state); last_error = error;
```

1.3.2.5.2.4.10.3.702、执行 EKF 的更新，包括 K 的计算和状态迭代更新可参考 LIO 更新部分

1.3.2.5.2.4.10.3.713、判断 增量部分的旋转，平移和曝光时间是否足够小，则 EKF 完成（实际上曝光被注释了）： if ((rot\_add.norm() \* 57.3f < 0.001f) && (t\_add.norm() \* 100.0f <  
0.001f)) EKF\_end = true;

1.3.2.5.2.4.10.3.729、否则 误差大于上次，则使用上次迭代或 LIO 的旧状态，EKF 结束。

```
(*state) = old_state; EKF_end = true;
```

1.3.2.5.2.4.10.3.7310、判断 迭代达到最大 或者 EKF 完成，退出迭代

1.3.2.5.2.4.10.4 4、更新状态协方差

```
state->cov -= G * state->cov;
```



1.3.2.5.2.4.10.5 5、更新帧的状态：updateFrameState(\*state);

```
new_frame_ ->T_f_w_ = SE3(Rcw, Pcw); // 就是更新一下世界系到当前帧的旋转和平移。
```

1.3.2.5.2.4.11 记录时间：double t3 = omp\_get\_wtime();

1.3.2.5.2.4.12 9、generateVisualMapPoints(img, pg);

用当前帧不确定性点生成视觉地图点对应函数：void VIOManager::generateVisualMapPoints(cv::Mat img, vector<pointWithVar> &pg)声明于 vio.h，定义于 vio.cpp1、遍历所有不确定点确定其对应的栅格索引，推入附加体素向量 add\_from\_voxel\_map// 候选视觉地图点的向量；2、可略过-----同时考虑射线投射采样的点，确定其对应的栅格索引，推入附加体素向量；3、对该栅格单元的附加体素点处理：1、先获得以该附加体素点 pt\_var 对应像素点为中心的块；2、使用 pt\_var 初始化一个新视觉地图点 pt\_new 和新特征 ftr\_new；3、对这两个属性进行元素更新；4、调用 insertPointIntoVoxelMap 将新视觉地图点插入到特征图 feat\_map 的视觉点属性向量中。

1.3.2.5.2.4.12.1 1、不确定点列表有点

```
if (pg.size() <= 10) return;
```

1.3.2.5.2.4.12.2 2、遍历所有不确定点

```
for (int i = 0; i < pg.size(); i++)在边缘范围内，根据像素角点值，更新对应栅格的 scan_value[index] 栅格角点值，附加体素点容器
append_voxel_points[index] 体素点 pg, grid_num[index]栅格类型
=TYPE_POINTCLOUD。
```

1.3.2.5.2.4.12.2.1 1、判断 不确定点的法向量为 0 向量 下个点

```
if (pg[i].normal == V3D(0, 0, 0)) continue;
```

1.3.2.5.2.4.12.2.2 2、获得不确定点的世界坐标系 pt 和当前帧的像素坐标系 pc

```
V3D pt = pg[i].point_w; V2D pc(new_frame_ ->w2c(pt));
```

1.3.2.5.2.4.12.2.3 3、判断 是否超出边界

```
if (new_frame_ ->cam_ ->isInFrame(pc.cast<int>(), border))
```

1.3.2.5.2.4.12.2.4 1、获得当前点对应的栅格坐标

```
int index = static_cast<int>(pc[1] / grid_size) * grid_n_width +
static_cast<int>(pc[0] / grid_size);
```

1.3.2.5.2.4.12.2.5 2、判断当前栅格的类型是否还没有点

```
if (grid_num[index] != TYPE_MAP)
```

1.3.2.5.2.4.12.2.6 1、使用 vk::shiTomasScore 函数，计算图像中某个像素点的 Shi-Tomas 角点分数

```
float cur_value = vk::shiTomasScore(img, pc[0], pc[1]);
```

#### 1.3.2.5.2.4.12.2.7 2、判断 该角点值是否大于该栅格记录的值

if (cur\_value > scan\_value[index])更新 scan\_value[index] 栅格角点值, 附加体素点容器 append\_voxel\_points[index] 体素点 pg, grid\_num[index] 栅格类型

#### 1.3.2.5.2.4.12.2.8 1、更新该栅格积累的值

scan\_value[index] = cur\_value;

#### 1.3.2.5.2.4.12.2.9 2、将该体素点 添加到 附加体素点容器 中

append\_voxel\_points[index] = pg[i];

#### 1.3.2.5.2.4.12.2.103、更新栅格类型为点云

grid\_num[index] = TYPE\_POINTCLOUD;

#### 1.3.2.5.2.4.12.3 3、考虑射线采样点: 遍历视觉体素子图 visual\_submap 中的平面中心点 (add\_from\_voxel\_map 是体素射线采样的容器) -----可略过

for (int j = 0; j < visual\_submap->add\_from\_voxel\_map.size(); j++)计算 角点值, 更新对应栅格的 scan\_value[index] 栅格角点值, 附加体素点容器 append\_voxel\_points[index] 体素点 pg, grid\_num[index] 栅格类型 =TYPE\_POINTCLOUD。

#### 1.3.2.5.2.4.12.3.1 1、获得该体素点的世界坐标 **pt** 和当前帧的像素坐标 **pc**

V3D pt = visual\_submap->add\_from\_voxel\_map[j].point\_w; V2D pc(new\_frame\_->w2c(pt));

#### 1.3.2.5.2.4.12.3.2 2、判断该体素点是否在范围内: if (new\_frame\_->cam\_->isInFrame(pc.cast<int>(), border))

#### 1.3.2.5.2.4.12.3.3 1、计算该栅格索引: int **index** = static\_cast<int>(pc[1] / grid\_size) \* grid\_n\_width + static\_cast<int>(pc[0] / grid\_size);

#### 1.3.2.5.2.4.12.3.4 2、判断 栅格类型是否不为地图 if (grid\_num[index] != TYPE\_MAP)

计算 角点值, 更新对应栅格的 scan\_value[index] 栅格角点值, 附加体素点容器 append\_voxel\_points[index] 体素点 pg, grid\_num[index] 栅格类型=TYPE\_POINTCLOUD。

#### 1.3.2.5.2.4.12.4 4、初始化计数变量: int add = 0;

记录被插入到 feat\_map 中的新视觉点 (每个栅格对应一个) 数量。

#### 1.3.2.5.2.4.12.5 5、遍历每个栅格单元: for (int i = 0; i < length; i++), 且只处理栅格类型为点云的栅格: if (grid\_num[i] == TYPE\_POINTCLOUD)

对该栅格单元的附加体素点处理: 1、先获得以该附加体素点 **pt\_var** 对应像素点为中心的块; 2、使用 **pt\_var** 初始化一个新视觉点属性 **pt\_new** 和新特征属性 **fttr\_new**; 3、对这两个属性进行元素更新; 4、将新视觉点属性插入

**insertPointIntoVoxelMap** 到特征图 **feat\_map** 的视觉点属性向量中。

1.3.2.5.2.4.12.5.1 1、取出该栅格的附加体素点 **pt\_var** 及其世界坐标 **pt** :

```
pointWithVar pt_var = append_voxel_points[i]; V3D pt =
pt_var.point_w;
```

1.3.2.5.2.4.12.5.2 2、将世界点法向量及其坐标转到当前相机系下 **norm\_vec**, **dir**, 并计算相机系下的归一化 **dir.normalize()** :

```
V3D norm_vec(new_frame_ ->T_f_w_.rotation_matrix() *
pt_var.normal); V3D dir(new_frame_ ->T_f_w_ * pt);dir.normalize();
```

1.3.2.5.2.4.12.5.3 3、到相机系原点向量与法向量的夹角 cos 值: double cos\_theta =  
dir.dot(norm\_vec);

1.3.2.5.2.4.12.5.4 4、获得该体素点的像素点坐标: V2D pc(new\_frame\_ ->w2c(pt));

1.3.2.5.2.4.12.5.5 5、初始化一个指向块像素数组的指针 float \*patch = new float[patch\_size\_total];

1.3.2.5.2.4.12.5.6 6、获得该图像的块: **getImagePatch**(img, pc, patch, 0);

通过权重插值, 获得以 pc 中心的块像素

1.3.2.5.2.4.12.5.7 7、新初始化一个视觉点(世界系的)。VisualPoint \*pt\_new = new VisualPoint(pt);

```
VisualPoint::VisualPoint(const Vector3d &pos): pos_(pos),
previous_normal_(Vector3d::Zero()),
normal_(Vector3d::Zero()),is_converged_(false),
is_normal_initialized_(false), has_ref_patch_(false){}
```

1.3.2.5.2.4.12.5.8 8、将像素点转到世界系下: Vector3d f = cam->cam2world(pc);

1.3.2.5.2.4.12.5.9 9、初始化一个新特征对象 **ftr\_new** : Feature \*ftr\_new = new Feature(pt\_new, patch, pc, f, new\_frame\_ ->T\_f\_w\_, 0); // 当前栅格单元对应的视觉点**属性** (世界系), 视觉点对应的块像素值, 视觉点对应的像素坐标, 视觉点对应的世界坐标系, 当前帧的世界系到相机系变换阵。

```
Feature(VisualPoint *_point, float *_patch, const Vector2d &_px, const
Vector3d &_f, const SE3 &_T_f_w, int _level) : type_(CORNER),
px_(px), f_(f), T_f_w_(T_f_w), mean_(0), score_(0), level_(level),
patch_(patch), point_(point){}
```

1.3.2.5.2.4.12.5.10 10、将图像 img, 新帧 id, 状态逆曝光时间 state->inv\_expo\_time, 给新特征对象  
ftr\_new

```
ftr_new->img_ = img;ftr_new->id_ = new_frame_ ->id_;ftr_new->
>inv_expo_time_ = state->inv_expo_time;
```

1.3.2.5.2.4.12.5.11 11、把新生成的这个特征对象 addFrameRef(ftr\_new) 作为新视觉点 pt\_new 的特征属性，获得新视觉点的协方差 pt\_new->covariance\_，该视觉点法向量标志位置真 pt\_new->is\_normal\_initialized\_：

```
pt_new->addFrameRef(ftr_new); // pt_new->covariance_ =
pt_var.var;pt_new->is_normal_initialized_ = true;
```

1.3.2.5.2.4.12.5.12 12、判断 夹角值是否小于 0（钝角）：if (cos\_theta < 0) { pt\_new->normal\_ = -pt\_var.normal; } 将新视觉点的法向量 pt\_new->normal\_ 设为附加体素点法向量的反方向 -pt\_var.normal。

1.3.2.5.2.4.12.5.13 13、否则 直接等于：pt\_new->normal\_ = pt\_var.normal

1.3.2.5.2.4.12.5.14 14、用新视觉点的法向量更新新视觉点之前的法向量（在 updateReferencePatch 会用到）：pt\_new->previous\_normal\_ = pt\_new->normal\_;

1.3.2.5.2.4.12.5.15 15、将该视觉点属性插入到特征图中的视觉点属性向量中：feat\_map, insertPointIntoVoxelMap(pt\_new);

```
void VIOManager::insertPointIntoVoxelMap(VisualPoint *pt_new)
```

1.3.2.5.2.4.12.5.16 1、获得该视觉点的位置属性（世界系坐标）

1.3.2.5.2.4.12.5.17 2、计算该点对应的体素坐标：loc\_xyz

```
double voxel_size = 0.5;float loc_xyz[3];for (int j = 0; j < 3; j++) {loc_xyz[j]
= pt_w[j] / voxel_size;if (loc_xyz[j] < 0) { loc_xyz[j] -= 1.0; } }
```

1.3.2.5.2.4.12.5.18 3、查找该位置的特征图是否已经有视觉点：feat\_map.find(position);

```
VOXEL_LOCATION position((int64_t)loc_xyz[0], (int64_t)loc_xyz[1],
(int64_t)loc_xyz[2]);auto iter = feat_map.find(position);
```

1.3.2.5.2.4.12.5.19 4、判断 已经有视觉点了 if (iter != feat\_map.end()): 将该视觉点推入该位置视觉点属性的视觉点向量，并计数。

```
iter->second->voxel_points.push_back(pt_new);iter->second->count++;
```

1.3.2.5.2.4.12.5.20 5、还没有视觉点：生成一个视觉点属性并将视觉点推到视觉点属性的视觉点向量，将该属性给特征图 feat\_map[position] = ot。

```
VOXEL_POINTS *ot = new VOXEL_POINTS(0);ot->
voxel_points.push_back(pt_new);feat_map[position] = ot;
```

1.3.2.5.2.4.12.5.2116、计数加1: `add += 1;`

1.3.2.5.2.4.12.6 6、打印新增加的点数: `printf("[ VIO ] Append %d new visual map points\n", add);`

**1.3.2.5.2.4.13 记录时间: `double t4 = omp_get_wtime();`**

**1.3.2.5.2.4.14 10、`plotTrackedPoints();`**

绘制跟踪点第一次运行这里应该会直接返回 `total_points = visual_submap->voxel_points.size()`, 应该是 0, 根据光度误差

1.3.2.5.2.4.14.1 1、第一次运行时这里应该是 0: 视觉子图的候选视觉点 `visual_submap->voxel_points`。

```
int total_points = visual_submap->voxel_points.size(); if (total_points == 0)
return;
```

1.3.2.5.2.4.14.2 2、遍历所有候选视觉点: `for (int i = 0; i < total_points; i++)`

1.3.2.5.2.4.14.2.1 1、取出候选视觉点: `VisualPoint *pt = visual_submap->voxel_points[i];`

1.3.2.5.2.4.14.2.2 2、获得对应的像素坐标: `V2D pc(new_frame->w2c(pt->pos_));`

1.3.2.5.2.4.14.2.3 3、判断视觉子图误差是否小于传播误差: `if (visual_submap->errors[i] <= visual_submap->propa_errors[i])`, 画一个绿色圆到该像素处

1.3.2.5.2.4.14.2.4 4、大于传播误差: 画一个蓝色圆到该像素处

```
cv::circle(img_cp, cv::Point2f(pc[0], pc[1]), 7, cv::Scalar(255, 0, 0), -1, 8);
```

**1.3.2.5.2.4.15 11、判断 若绘制标志为真 投影块从参考到当前图-----可略过**

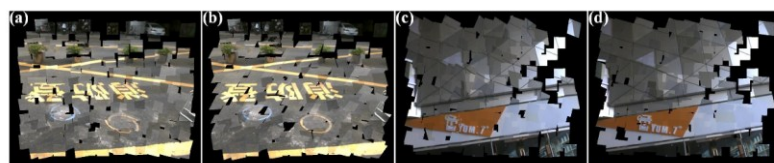


Fig. S1: (a) and (c) depict the warped patches derived from the constant depth assumption, while (b) and (d) represent those based on the plane prior.

`if (plot_flag) projectPatchFromRefToCur(feats_map); plot_flag` 只有在 `fabs((LidarMeasures.last_lidar_update_time - _first_lidar_time) - plot_time) < (frame_cnt / 2 * 0.1)` 才为真, `x+10 < 0.3` 不可能为真。这个是用来做实验对比的, 就是将参考块投影到当前块, 看看错位是否严重。

**1.3.2.5.2.4.16 记录时间: `double t5 = omp_get_wtime();`**

**1.3.2.5.2.4.17 12、`updateVisualMapPoints(img);`**

给视觉子地图中的视觉地图点添加新图像块到图像块列表中。第一次运行到这

里也会返回对应函数：void VIOManager::updateVisualMapPoints(cv::Mat img)声明于 vio.h，定义于 vio.cpp 遍历所有候选点属性 若该视觉点已收敛则只保留一个参考块，否则获取当前帧图像上的图像块，特征，并添加到参考块列表中  
\*\*obs\_.push\_front(ftr)\*\*。

#### 1.3.2.5.2.4.17.1 1、视觉子地图中的点数为 0 直接返回

```
if (total_points == 0) return;
```

#### 1.3.2.5.2.4.17.2 2、初始化更新计数 update\_num 以及当前帧的世界系到当前系的变换阵 pose\_cur：

```
int update_num = 0; SE3 pose_cur = new_frame_ ->T_f_w_;
```

#### 1.3.2.5.2.4.17.3 3、遍历所有视觉子地图中的所有点属性

```
for (int i = 0; i < total_points; i++)若该视觉点已收敛则只保留一个参考块，否则继续在当前帧图像上获取块，特征，并添加到参考块列表中
obs_.push_front(ftr)。
```

##### 1.3.2.5.2.4.17.3.1 1、获得一个候选视觉点属性 pt

```
VisualPoint *pt = visual_submap->voxel_points[i];
```

##### 1.3.2.5.2.4.17.3.2 2、判断该视觉点属性是否被生成：if (pt == nullptr) continue;

##### 1.3.2.5.2.4.17.3.3 3、判断 该视觉点属性是否收敛

```
if (pt->is_converged_)若收敛了，就将参考块列表中其他块给删除了，只保留 ref_patch，然后计算下一个视觉点属性。pt->deleteNonRefPatchFeatures(); continue;对应函数：void
VisualPoint::deleteNonRefPatchFeatures()遍历图像块列表，并将列表中不是指定参考块 ref_patch 的其他块删除
```

##### 1.3.2.5.2.4.17.3.4 1、判断该块是否是指定参考块 if (\*it != ref\_patch)，不是则删除该块

```
delete *it; it = obs_.erase(it);
```

##### 1.3.2.5.2.4.17.3.5 2、否则 遍历下一个 ++it;

##### 1.3.2.5.2.4.17.3.6 4、该视觉点属性尚未收敛

继续添加参考特征。1、获得该视觉点在当前帧的块 2、与上一个参考特征的位姿变换是否足够 3、若参考块列表太多则删除分数最小的 4、生成一个新的参考特征，并将该特征推入参考块列表中。

##### 1.3.2.5.2.4.17.3.7 1、获得该点到当前图像的像素坐标 pc，并初始化一个添加标志位 add\_flag：

```
V2D pc(new_frame_ ->w2c(pt->pos_));bool add_flag = false;
```

##### 1.3.2.5.2.4.17.3.8 2、初始化一个块指针

```
float *patch_temp = new float[patch_size_total];
```

1.3.2.5.2.4.17.3.9 3、获取该视觉点在当前图像上的图像块

```
getImagePatch(img, pc, patch_temp, 0);
```

1.3.2.5.2.4.17.3.104、获得该视觉点属性的参考块列表的最后一个参考特征

```
Feature *last_feature = pt->obs_.back();
```

1.3.2.5.2.4.17.3.115、获得该参考特征的世界系到参考帧系的变换阵

```
SE3 pose_ref = last_feature->T_f_w_;
```

1.3.2.5.2.4.17.3.126、获得当前帧到参考帧的位姿变换

```
SE3 delta_pose = pose_ref * pose_cur.inverse();
```

1.3.2.5.2.4.17.3.137、两帧之间的平移距离

```
double delta_p = delta_pose.translation().norm();
```

1.3.2.5.2.4.17.3.148、通过旋转矩阵的迹 (trace) 推导出旋转角度

```
double delta_theta = (delta_pose.rotation_matrix().trace() > 3.0 - 1e-6) ? 0.0 :
std::acos(0.5 * (delta_pose.rotation_matrix().trace() - 1));如果迹非常接近 3,
说明旋转角度接近 0, 直接将[delta_theta](http:)设为 0。否则, 利用旋转
矩阵的迹与旋转角度的关系公式直接计算出旋转角度 theta (弧度制)
```

1.3.2.5.2.4.17.3.159、若平移距离超过 0.5 米, 旋转角度大于 0.3 弧度, 将添加标志位 **add\_flag** 设为真

```
if (delta_p > 0.5 || delta_theta > 0.3) add_flag = true;
```

1.3.2.5.2.4.17.3.1610、参考块的视觉点像素位置与视觉点在当前帧像素位置距离

```
double pixel_dist = (pc - last_px).norm();
```

1.3.2.5.2.4.17.3.1711、若像素距离超过 40 则 添加标志位 **add\_flag** 为真

```
if (pixel_dist > 40) add_flag = true;
```

1.3.2.5.2.4.17.3.1812、若当前点的图像块列表超出了 30 个

```
if (pt->obs_.size() >= 30)
```

1.3.2.5.2.4.17.3.191、找到最小分数的特征

```
Feature *ref_ftr;pt->findMinScoreFeature(new_frame_->pos(), ref_ftr);
```

1.3.2.5.2.4.17.3.202、删除这个特征: pt->deleteFeatureRef(ref\_ftr);

1.3.2.5.2.4.17.3.2113、判断 标志位 **add\_flag** 为真 if (add\_flag)

1.3.2.5.2.4.17.3.221、更新计数 update\_num 加 1, 该位置的更新标志 update\_flag[i] 位置 1

```
update_num += 1;update_flag[i] = 1;
```

1.3.2.5.2.4.17.3.232、根据该视觉点，对应的块，该帧像素位置，该帧下此点位置，该帧世界系到当前帧的变换阵，最佳搜索层级，生成一个新特征

```
Feature *ftr_new = new Feature(pt, patch_temp, pc, f, new_frame_->T_f_w_, visual_submap->search_levels[i]);
```

1.3.2.5.2.4.17.3.243、添加图像，id 和逆曝光时间：

```
ftr_new->img_ = img;ftr_new->id_ = new_frame_->id_;ftr_new->inv_expo_time_ = state->inv_expo_time;
```

1.3.2.5.2.4.17.3.254、将该特征添加到该点的图像块列表中：pt->addFrameRef(ftr\_new);

```
void VisualPoint::addFrameRef(Feature *ftr){obs_.push_front(ftr);}
```

1.3.2.5.2.4.17.4 4、打印更新了多少个视觉点属性到 视觉子图中。

**1.3.2.5.2.4.18 记录时间：double t6 = omp\_get\_wtime();**

**1.3.2.5.2.4.19 13、updateReferencePatch(feat\_map);**

更新视觉子地图中视觉地图点的参考图像块对应函数：void VIOManager::updateReferencePatch(const unordered\_map<VOXEL\_LOCATION, VoxelOctoTree\*> &plane\_map)声明于 vio.h，定义于 vio.cpp 第一次运行到这里也会返回，计算 NCC 分数，选出每个视觉点 分数最高的参考块作为指定参考块

1.3.2.5.2.4.19.1 1、视觉子地图中视觉地图点为 0 直接返回

```
if (total_points == 0) return;
```

1.3.2.5.2.4.19.2 2、遍历视觉子地图中视觉地图点

```
for (int i = 0; i < visual_submap->voxel_points.size(); i++)
```

1.3.2.5.2.4.19.2.1 1、取出 视觉点属性

```
VisualPoint *pt = visual_submap->voxel_points[i];
```

1.3.2.5.2.4.19.2.2 2、判断 法向量是否已初始化 || 该视觉点属性是否已收敛 || 该视觉点属性的参考块列表是否 ≤ 5 || 该点对应的更新标志位为 0 。满足任一项则跳过

```
if (!pt->is_normal_initialized_) continue;if (pt->is_converged_) continue;if (pt->obs_.size() <= 5) continue;if (update_flag[i] == 0) continue;
```

1.3.2.5.2.4.19.2.3 3、获得该视觉点属性的世界坐标系

```
const V3D &p_w = pt->pos_;
```

1.3.2.5.2.4.19.2.4 4、计算其体素坐标 loc\_xyz

```
float loc_xyz[3];for (int j = 0; j < 3; j++) {loc_xyz[j] = p_w[j] / 0.5;if (loc_xyz[j] < 0) { loc_xyz[j] -= 1.0; } } VOXEL_LOCATION position((int64_t)loc_xyz[0], (int64_t)loc_xyz[1], (int64_t)loc_xyz[2]);
```



#### 1.3.2.5.2.4.19.2.5 5、从体素图中找到该位置的八叉树节点索引

```
auto iter = plane_map.find(position);
```

#### 1.3.2.5.2.4.19.2.6 6、判断 该索引有效

if (iter != plane\_map.end())构建关于这个点到平面距离的期望和协方差，点到平面距离够小，计算该视觉点属性的法向量增量是否足够小，更新之前法向量，足够小则任务该视觉点已收敛 pt->is\_converged\_ = true;。  
主要是对该视觉点收敛进行判断

#### 1.3.2.5.2.4.19.2.7 1、找到该点对应的子叶节点 current\_octo

```
VoxelOctoTree *current_octo; current_octo = iter->second->find_correspond(p_w);
```

#### 1.3.2.5.2.4.19.2.8 2、判断该子叶节点的平面属性是否为真

```
if (current_octo->plane_ptr_->is_plane_)
```

#### 1.3.2.5.2.4.19.2.9 1、计算该点到平面中心点的向量在平面上的投影长度 range\_dis

```
VoxelPlane &plane = *current_octo->plane_ptr_; float dis_to_plane = plane.normal_(0) * p_w(0) + plane.normal_(1) * p_w(1) + plane.normal_(2) * p_w(2) + plane.d_; float dis_to_plane_abs = fabs(dis_to_plane); float dis_to_center = (plane.center_(0) - p_w(0)) * (plane.center_(0) - p_w(0)) + (plane.center_(1) - p_w(1)) * (plane.center_(1) - p_w(1)) + (plane.center_(2) - p_w(2)) * (plane.center_(2) - p_w(2)); float range_dis = sqrt(dis_to_center - dis_to_plane * dis_to_plane);
```

#### 1.3.2.5.2.4.19.2.10 2、判断这个投影模长是否小于 3 倍的平面协方差的最大特征值

$$d_i \sim \mathcal{N}(0, \Sigma_{\mathbf{w}_i}), \quad \Sigma_{\mathbf{w}_i} = \mathbf{J}_{\mathbf{w}_i} \Sigma_{\mathbf{n}_i, \mathbf{q}_i, \mathbf{w}_{\mathbf{p}_i}} \mathbf{J}_{\mathbf{w}_i}^T$$

if (range\_dis <= radius\_k \* plane.\*\*radius\_\*\*)构建关于这个点到平面距离的期望和协方差，点到平面距离够小，计算该视觉点属性的法向量增量是否足够小，更新之前法向量，足够小则任务该视觉点已收敛 pt->is\_converged\_ = true;。

#### 1.3.2.5.2.4.19.2.11 1、将点到平面距离整理为期望加一阶项的形式：计算得到点到平面距离协方差（方差） sigma\_l。

$$d_i = (\mathbf{n}_i^{gt} \oplus \delta_{\mathbf{n}_i})^T [({}^W \mathbf{p}_i^{gt} + \delta_{\mathbf{w}_{\mathbf{p}_i}}) - \mathbf{q}_i^{gt} - \delta_{\mathbf{q}_i}]$$

$$\approx \underbrace{\mathbf{n}_i^{gtT} ({}^W \mathbf{p}_i^{gt} - \mathbf{q}_i^{gt})}_0 + \underbrace{\mathbf{J}_{\mathbf{n}_i} \delta_{\mathbf{n}_i} + \mathbf{J}_{\mathbf{q}_i} \delta_{\mathbf{q}_i} + \mathbf{J}_{\mathbf{w}_{\mathbf{p}_i}} \delta_{\mathbf{w}_{\mathbf{p}_i}}}_{\mathbf{w}_i}$$

#### 1.3.2.5.2.4.19.2.12 1、声明一个 1\*6 的雅可比矩阵：Eigen::Matrix<double, 1, 6> J\_nq;

#### 1.3.2.5.2.4.19.2.13 2、构建这个 J 阵（只构建了前两项）这里将平面不确定性与世界点不确定性分开实现 J。这里关于 J 的代码只将平面不确定性考虑在内。

$$\mathbf{J}_{\mathbf{w}_i} = [\mathbf{J}_{\mathbf{n}_i}, \mathbf{J}_{\mathbf{q}_i}, \mathbf{J}_{\mathbf{w}_{\mathbf{p}_i}}] = [({}^W \mathbf{p}_i - \mathbf{q}_i)^T, -\mathbf{n}_i^T, \mathbf{n}_i^T]$$

J\_nq.block<1, 3>(0, 0) = p\_w - plane.center\_; J\_nq.block<1, 3>(0, 3) = -

```
plane.normal_;
```

1.3.2.5.2.4.19.2.143、基于平面不确定性构建协方差第一项和第二项+点不确定性构建第三项=点到平面距离协方差

$$\Sigma_{\mathbf{w}_i} = \mathbf{J}_{\mathbf{w}_i} \Sigma_{\mathbf{n}_i, \mathbf{q}_i, \mathbf{w}_{\mathbf{p}_i}} \mathbf{J}_{\mathbf{w}_i}^T$$

```
double sigma_l = J_nq * plane.plane_var_ * J_nq.transpose(); sigma_l +=
plane.normal_.transpose() * pv.var * plane.normal_;
```

1.3.2.5.2.4.19.2.152、判断 点到平面距离是否小于 3 倍 平方差

```
if (dis_to_plane < sigma_num * sqrt(sigma_l))
```

1.3.2.5.2.4.19.2.161、判断 这个视觉点属性之前的法向量与当前体素节点平面法向量夹角是否钝角  
是则 将当前平面法向量取反给这个点当前法向量（让当前视觉点的法向量与之前的一致）

```
if (pt->previous_normal_.dot(plane.normal_) < 0) { pt->normal_ = -
plane.normal_; }
```

1.3.2.5.2.4.19.2.172、否则 锐角 直接赋值法向量

```
else { pt->normal_ = plane.normal_; }
```

1.3.2.5.2.4.19.2.183、计算 视觉点属性当前法向量与之前法向量的增量

```
double normal_update = (pt->normal_ - pt->previous_normal_).norm();
```

1.3.2.5.2.4.19.2.194、将当前法向量赋给之前法向量

```
pt->previous_normal_ = pt->normal_;
```

1.3.2.5.2.4.19.2.205、如果增量 比较小，且 该视觉点属性的参考块列表大小不低于 10 个，则认为该视觉点已收敛。

```
if (normal_update < 0.0001 && pt->obs_.size() > 10) {pt-
>is_converged_ = true; //
visual_converged_point.push_back(pt); }
```

1.3.2.5.2.4.19.2.217、初始化一个最大分数

```
float score_max = -1000.;
```

1.3.2.5.2.4.19.2.228、遍历该视觉地图点的图像块列表（并未计算 w1）。

$$\text{NCC}(\mathbf{f}, \mathbf{g}) = \frac{\sum_{x,y} [\mathbf{f}(x,y) - \bar{\mathbf{f}}] [\mathbf{g}(x,y) - \bar{\mathbf{g}}]}{\sqrt{\sum_{x,y} [\mathbf{f}(x,y) - \bar{\mathbf{f}}]^2 \sum_{x,y} [\mathbf{g}(x,y) - \bar{\mathbf{g}}]^2}}$$

$$c = \frac{\mathbf{n} \cdot \mathbf{p}}{\|\mathbf{p}\|}, \quad \omega_1 = \frac{1}{1 + e^{\text{tr}(\Sigma_{\mathbf{n}})}}$$

$$S = (1 - \omega_1) \cdot \frac{1}{n} \sum_{i=1}^n \text{NCC}(\mathbf{f}, \mathbf{g}_i) + \omega_1 \cdot c \quad (12)$$

```
for (auto it = pt->obs_.begin(), ite = pt->obs_.end(); it != ite; ++it)通过计算
```

参考块灰度均值，计算 NCC，及分数，并通过分数获得指定参考块，更新相关标志位及属性。

1.3.2.5.2.4.19.2.231、初始化一个临时参考特征

```
Feature *ref_patch_temp = *it;
```

1.3.2.5.2.4.19.2.242、获得该临时参考特征的块数组指针

```
float *patch_temp = ref_patch_temp->patch_;
```

1.3.2.5.2.4.19.2.253、初始化一些 NCC 参数

```
float NCC_up = 0.0;float NCC_down1 = 0.0;float NCC_down2 = 0.0;float
NCC = 0.0;float score = 0.0;int count = 0;
```

1.3.2.5.2.4.19.2.264、获得该视觉点在临时参考特征系下的三维点坐标，并做归一化处理

```
V3D pf = ref_patch_temp->T_f_w_ * pt->pos_; pf.normalize();
```

1.3.2.5.2.4.19.2.275、同样，将视觉点属性的法向量也转到临时参考特征系下

```
V3D norm_vec = ref_patch_temp->T_f_w_.rotation_matrix() * pt->normal_;
```

1.3.2.5.2.4.19.2.286、计算临时参考系到点的向量与该点法向量的夹角 COS 值

```
double cos_angle = pf.dot(norm_vec);
```

1.3.2.5.2.4.19.2.297、计算 该临时参考特征对应块的平均灰度值

```
ref_patch_temp->mean_
```

1.3.2.5.2.4.19.2.301、初始化一个变量记录平均灰度值

```
float ref_mean;
```

1.3.2.5.2.4.19.2.312、判断 该参考特征的平均灰度值是否很小：是，则累加整个块的像素值并求平均 ref\_mean。

```
if (abs(ref_patch_temp->mean_) < 1e-6){float ref_sum =
std::accumulate(patch_temp, patch_temp + patch_size_total, 0.0);
ref_mean = ref_sum / patch_size_total;ref_patch_temp->mean_ = ref_mean;}
```

1.3.2.5.2.4.19.2.328、遍历 该视觉地图点图像块列表的其他图像块：for (auto itm = pt->obs\_.begin(),  
itme = pt->obs\_.end(); itm != itme; ++itm)

计算当前图像块与其他图像块之间的 NCC 总和

1.3.2.5.2.4.19.2.331、避开临时参考特征

```
if ((*itm)->id_ == ref_patch_temp->id_) continue;
```

1.3.2.5.2.4.19.2.342、获得其他块：float \*patch\_cache = (\*itm)->patch\_;

1.3.2.5.2.4.19.2.353、计算其他块的灰度均值：(\*itm)->mean\_

```
float other_mean;if (abs((*itm)->mean_) < 1e-6) {float other_sum =
std::accumulate(patch_cache, patch_cache + patch_size_total, 0.0);
```

```
other_mean = other_sum / patch_size_total; (*itm)->mean_ =
other_mean; }
```

1.3.2.5.2.4.19.2.364、计算 NCC 的上面一项 NCC\_up 和下面两项 NCC\_down1, NCC\_down2

$$NCC(f, g) = \frac{\sum_{x,y} [f(x, y) - \bar{f}] [g(x, y) - \bar{g}]}{\sqrt{\sum_{x,y} [f(x, y) - \bar{f}]^2 \sum_{x,y} [g(x, y) - \bar{g}]^2}}$$

```
for (int ind = 0; ind < patch_size_total; ind++) { NCC_up +=
(patch_temp[ind] - ref_mean) * (patch_cache[ind] - other_mean);
NCC_down1 += (patch_temp[ind] - ref_mean) * (patch_temp[ind] -
ref_mean); NCC_down2 += (patch_cache[ind] - other_mean) *
(patch_cache[ind] - other_mean); }
```

1.3.2.5.2.4.19.2.375、计算 NCC 并计数

```
NCC += fabs(NCC_up / sqrt(NCC_down1 * NCC_down2)); count++;
```

1.3.2.5.2.4.19.2.389、计算当前参考块的均值: NCC = NCC / count;

1.3.2.5.2.4.19.2.3910、计算当前参考块的分数 (并未计算  $w_1$ ) 。

$$c = \frac{\mathbf{n} \cdot \mathbf{p}}{\|\mathbf{p}\|}, \quad \omega_1 = \frac{1}{1 + e^{\text{tr}(\Sigma_n)}}$$

$$S = (1 - \omega_1) \cdot \frac{1}{n} \sum_{i=1}^n NCC(f, g_i) + \omega_1 \cdot c$$

```
score = NCC + cos_angle;
```

1.3.2.5.2.4.19.2.4011、将分数给当前这个参考特征

```
ref_patch_temp->score_ = score;
```

1.3.2.5.2.4.19.2.4112、判断 分数是否是最大的, if (score > score\_max), 是则更新该参考块为视觉点属性指定参考块, 并更新拥有参考块标志位为真:

```
score_max = score;pt->ref_patch = ref_patch_temp;pt->has_ref_patch_ =
true;
```

**1.3.2.5.2.4.20 记录时间: double t7 = omp\_get\_wtime();**

**1.3.2.5.2.4.21 14、判断 colmap 格式, 产生 colmap 格式数据: if(colmap\_output\_en) dumpDataForColmap();**

1.3.2.5.2.5 5、判断是否以 IMU 频率传播使能: if(imu\_prop\_enable)----可略过

```
ekf_finish_once = true; //EKF 完成一次为真 latest_ekf_state = _state; //获得状态
latest_ekf_time = LidarMeasures.last_lidar_update_time; //当前图像捕获时间为最新 ekf
时间 state_update_flg = true; //状态更新位为真。
```

1.3.2.5.2.6 6、发布有 RGB 颜色的点云:

```
publish_frame_world(pubLaserCloudFullRes, vio_manager);
```

1.3.2.5.2.7 7、发布带有标注点的图像 img\_cp :

```

publish_img_rgb(pubImage, vio_manager);
1.3.2.5.2.8 8、将姿态存入 fout_out 文件中
euler_cur = RotMtoEuler(_state.rot_end);fout_out << std::setw(20) <<
LidarMeasures.last_lid_update_time - _first_lidar_time << " " << euler_cur.transpose() *
57.3 << " " << _state.pos_end.transpose() << " " << _state.vel_end.transpose() << " " <<
_state.bias_g.transpose() << " " << _state.bias_a.transpose() << " " <<
V3D(_state.inv_expo_time, 0, 0).transpose() << " " << feats_undistort->points.size() <<
std::endl;
1.3.3 3、系统运行结束后将缓存的点云保存为 PCD
savePCD();

```